



LA PERSISTENZA

Il presente documento è di fondamentale importanza.

In esso saranno contenuti numerosi frammenti di codice da analizzare, tutti tratti dalle lezioni del ricercatore G. De Angelis.

Si tratta di una sezione che non tutti gli studenti hanno affrontato pienamente, poiché molti appartenenti all'indirizzo di Robotica non hanno seguito il corso di Basi di Dati tenuto dalla prof.ssa V. De Nitto e dal prof. ing. A. Pellegrini.

Pertanto, questo documento ha l'obiettivo di riprendere l'intero corso di Basi di Dati, sintetizzarlo e integrarlo con gli insegnamenti del ricercatore G. De Angelis.

Si segnala che è stata redatta una cartella dedicata al corso di Basi di Dati all'interno di questo drive.

Ogni applicazione che si rispetti deve essere accompagnata da un sistema di persistenza a supporto del proprio funzionamento: tale persistenza può consistere in un file oppure in una base di dati, ossia un contenitore deputato alla memorizzazione delle informazioni.

In questo documento verrà spiegato tutto in modo chiaro e graduale, presupponendo che il lettore non abbia alcuna conoscenza pregressa del corso di Basi di Dati.

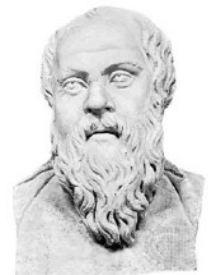
Inoltre, si assumerà che il lettore non abbia ancora superato il corso di Sistemi Operativi, nel quale vengono introdotti i concetti di stream e gestione dell'I/O.

L'obiettivo è che questo testo rappresenti una guida introduttiva "for dummies", in grado di accompagnare il lettore passo dopo passo, aumentando progressivamente il livello di complessità.

La consapevolezza è il punto di partenza fondamentale.

Come affermava Socrate: «Io so di non sapere».

Ecco, partiamo da qui: dalle basi, dalla consapevolezza di non sapere ancora, ma di essere coscienti di ciò, e questo è ciò che conta davvero.



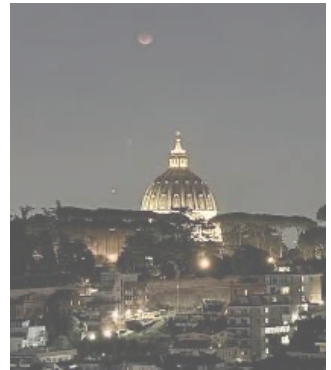
Il materiale contenuto in questo documento è stato redatto a fini didattici e può contenere semplificazioni o adattamenti rispetto ai contenuti ufficiali dei corsi universitari.

LA PERSISTENZA - COMINCIAMO

Immaginiamo una scena.

Esco di casa, decido di andare al **Parco di Monte Ciocchi**, uno splendido punto panoramico a Roma Ovest, non lontano dal Vaticano.

La benzina costa, quindi scelgo di muovermi con i **mezzi pubblici**.



Prendo il **Cotral**, arrivo alla fermata della **metropolitana**, scendo a **Termini**, magari faccio un **cambio di linea**, sto attento ai **borseggiatori**, e infine arrivo alla fermata **Valle Aurelia**, a pochi minuti dal parco. Fin qui tutto normale, un tragitto come tanti.

Ma proviamo a guardarlo da un punto di vista “informatico”.

Ogni azione che compio — prendere il Cotral, scendere a Termini, fare il cambio, arrivare a Valle Aurelia — è un **dato**.

Insieme, questi dati descrivono **la mia esperienza, il mio percorso, le mie scelte**. Se però, arrivato al parco, **nessuno di questi dati viene memorizzato**, il sistema non avrà alcuna traccia del mio viaggio.

È come se non fossi **mai uscito di casa**.

La persistenza dei dati serve proprio a questo: fare in modo che le informazioni non vadano perdute una volta terminata l'esecuzione del programma.

Nel nostro esempio, il “diario del viaggio” può assumere diverse forme:

- può essere un **file di testo** su cui l'applicazione scrive i dati del viaggio;
- oppure può essere una **base di dati**, un sistema strutturato e affidabile in grado di gestire milioni di viaggi, utenti, fermate e linee.

La **persistenza** è, quindi, ciò che dà **memoria a lungo termine** ai sistemi informatici.

È ciò che permette a un'app come Google Maps di ricordare i tuoi luoghi preferiti, o a un sito di biglietteria di sapere che hai già comprato un biglietto.



Netflix deve ricordare che serie stai guardando, dove ti sei fermato, cosa ti è piaciuto, cosa consigliarti dopo.

Quando salvi una playlist o metti un brano tra i preferiti, stai utilizzando un sistema di persistenza.

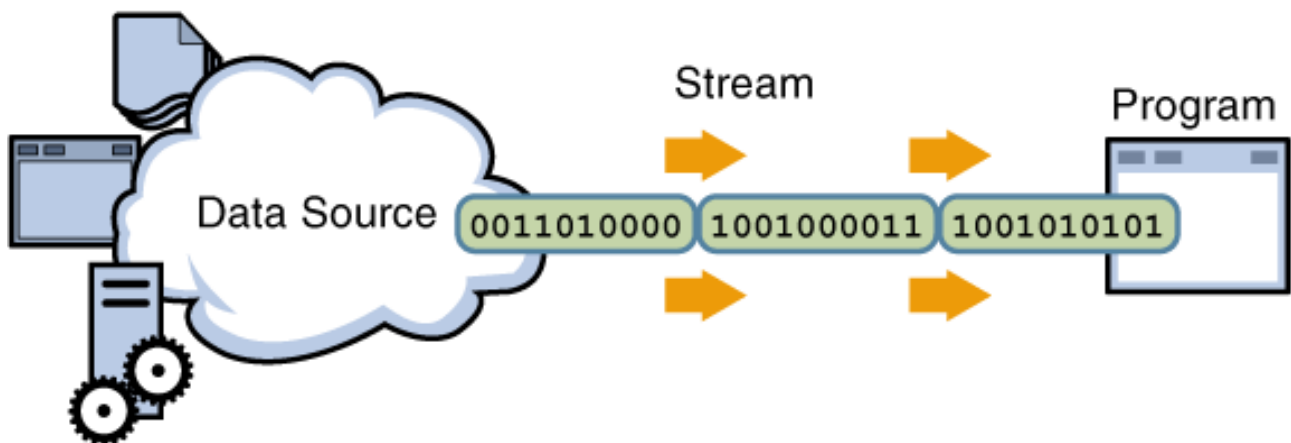
Ogni video, like, iscrizione o cronologia di visione è **persistita**.

STREAM E JAVA

In un programma, l'**Input/Output (I/O)** riguarda tutto ciò che entra ed esce: leggere da file, scrivere su disco, ricevere dati da rete, stampare su schermo, leggere da tastiera, ecc.

Java deve gestire **una grande varietà di dispositivi e formati**: file di testo, file binari, rete, dispositivi esterni, ecc.

Tutta la gestione I/O in Java si basa sul concetto di **stream**, cioè un **flusso continuo di dati**.



Immaginalo come un tubo d'acqua:

- all'ingresso (*input stream*) fluiscono dati **che arrivano da una sorgente**,
- all'uscita (*output stream*) fluiscono dati **che vanno verso un destinatario**.

Quando il programma deve **ricevere dati**, apre un *input stream* sulla sorgente.

```
InputStream in = new FileInputStream("dati.txt");
```

Poi legge i dati **uno dopo l'altro**, in modo sequenziale:

1. open(stream)
2. while (ci sono dati)
3. read(informazione)

4. close(stream)

Quando il programma deve **inviare dati**, apre un *output stream* verso la destinazione.

```
OutputStream out = new FileOutputStream("output.txt");
```

Segue lo stesso schema:

1. open(stream)

2. while (ci sono dati)

3. write(informazione)

4. close(stream)

La libreria standard della piattaforma Java per la gestione degli stream è localizzato nel package java.io!

Esempio lettura di un file con FileInputStream (byte stream)

```
import java.io.FileInputStream;
import java.io.IOException;

public class LeggiFileByte {
    public static void main(String[] args) {
        try {
            // 1. Apro uno stream in ingresso sul file
            FileInputStream input = new FileInputStream("testo.txt");

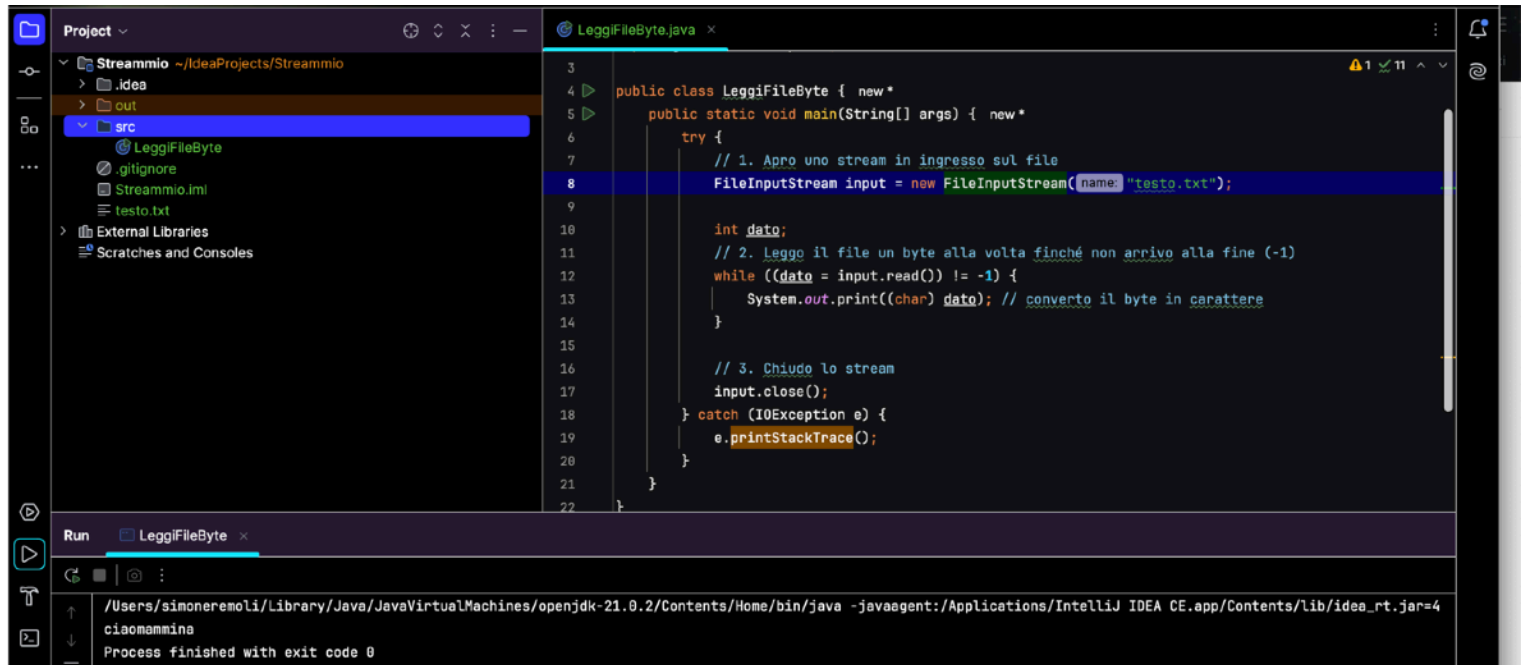
            int dato;
            // 2. Leggo il file un byte alla volta finché non arrivo alla
            fine (-1)
            while ((dato = input.read()) != -1) {
                System.out.print((char) dato); // converto il byte in
            carattere
            }

            // 3. Chiudo lo stream
            input.close();
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

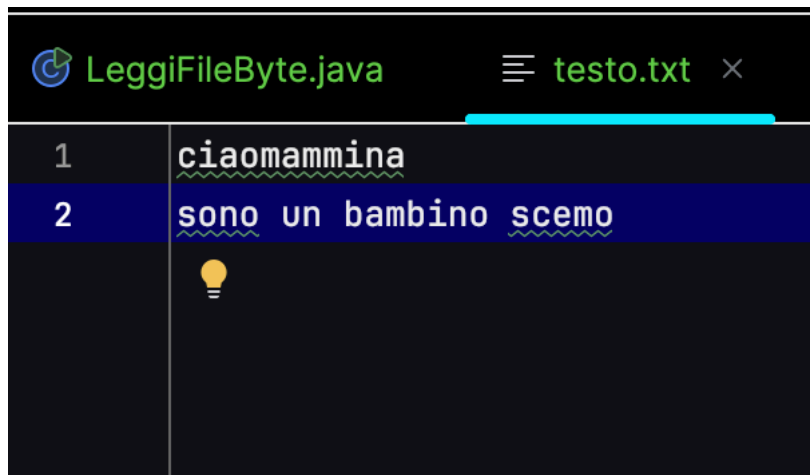
Apre uno input stream sul file testo.txt.

Legge byte per byte. Stampa tutto su console.

Se nel file ho scritto “ciaomamma” l’output è “ciaomamma”.



Se nel file aggiungo una linea sotto:



Output:

ciaomamma

sono un bambino scemo

Esempio scrittura di un file con FileOutputStream (byte stream)

```
import java.io.FileOutputStream;  
import java.io.IOException;  
  
public class ScriviFileByte {  
    public static void main(String[] args) {  
        String testo = "Me so mbriacato";  
    }  
}
```

```

try {
    // 1. Apro uno stream in uscita verso il file
    FileOutputStream output = new FileOutputStream("testo.txt");

    // 2. Scrivo la stringa convertita in array di byte
    output.write(testo.getBytes());

    // 3. Chiudo lo stream
    output.close();

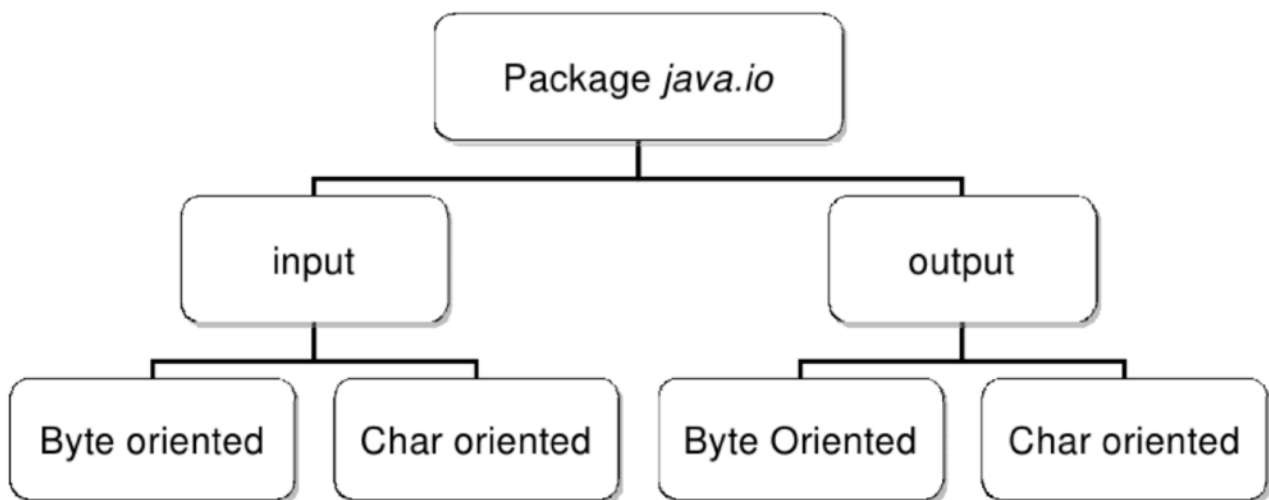
    System.out.println("Scrittura completata!");
} catch (IOException e) {
    e.printStackTrace();
}
}

```

Crea (o sovrascrive) il file output.txt.

Scrivo il contenuto come flusso di byte. Chiudo il file in modo sicuro.

Attenzione: come è organizzato tutto il sistema di Input/Output in Java?



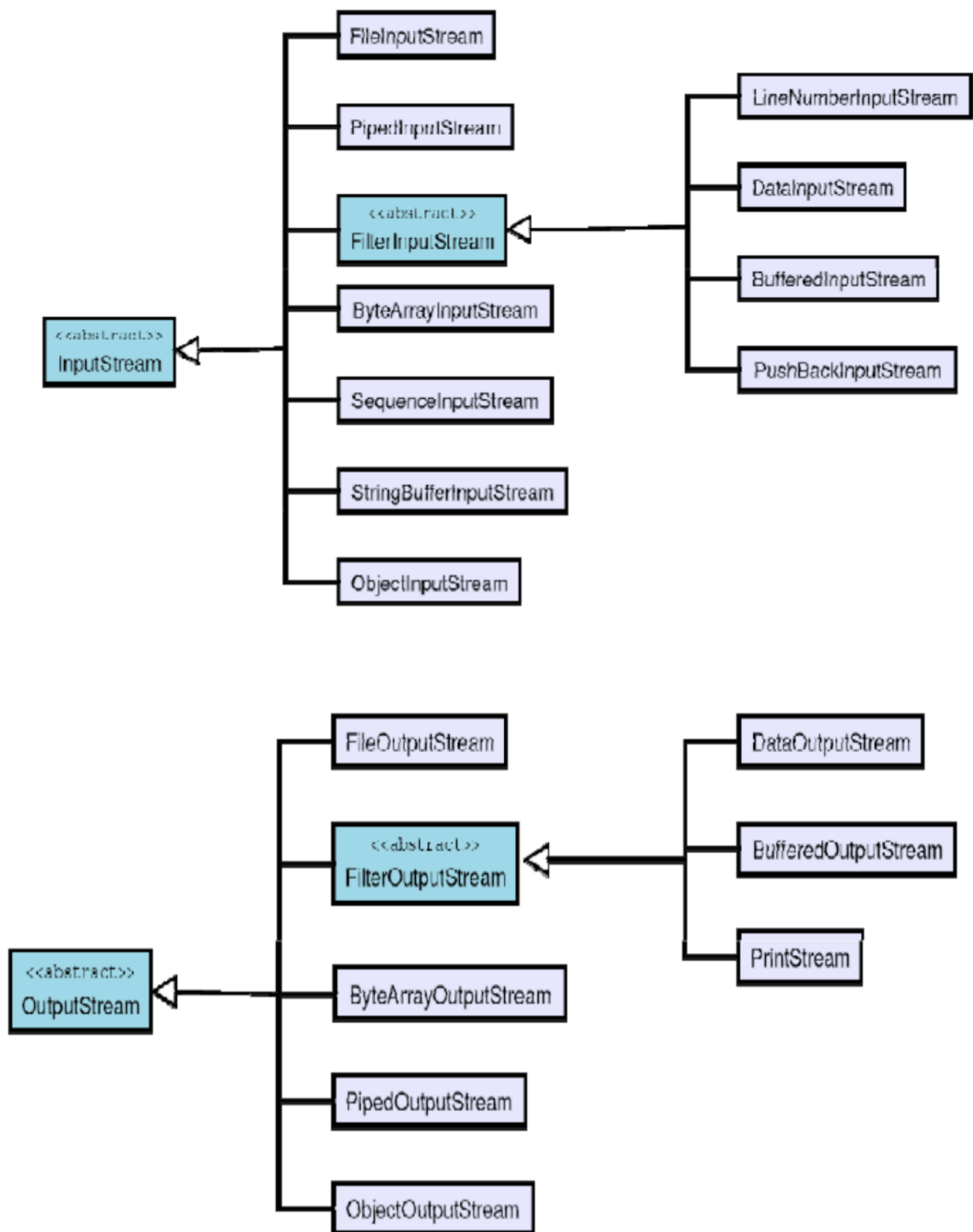
Byte-oriented vs Char-oriented: dopo aver deciso se leggo o scrivo, devo capire che tipo di dati sto gestendo.

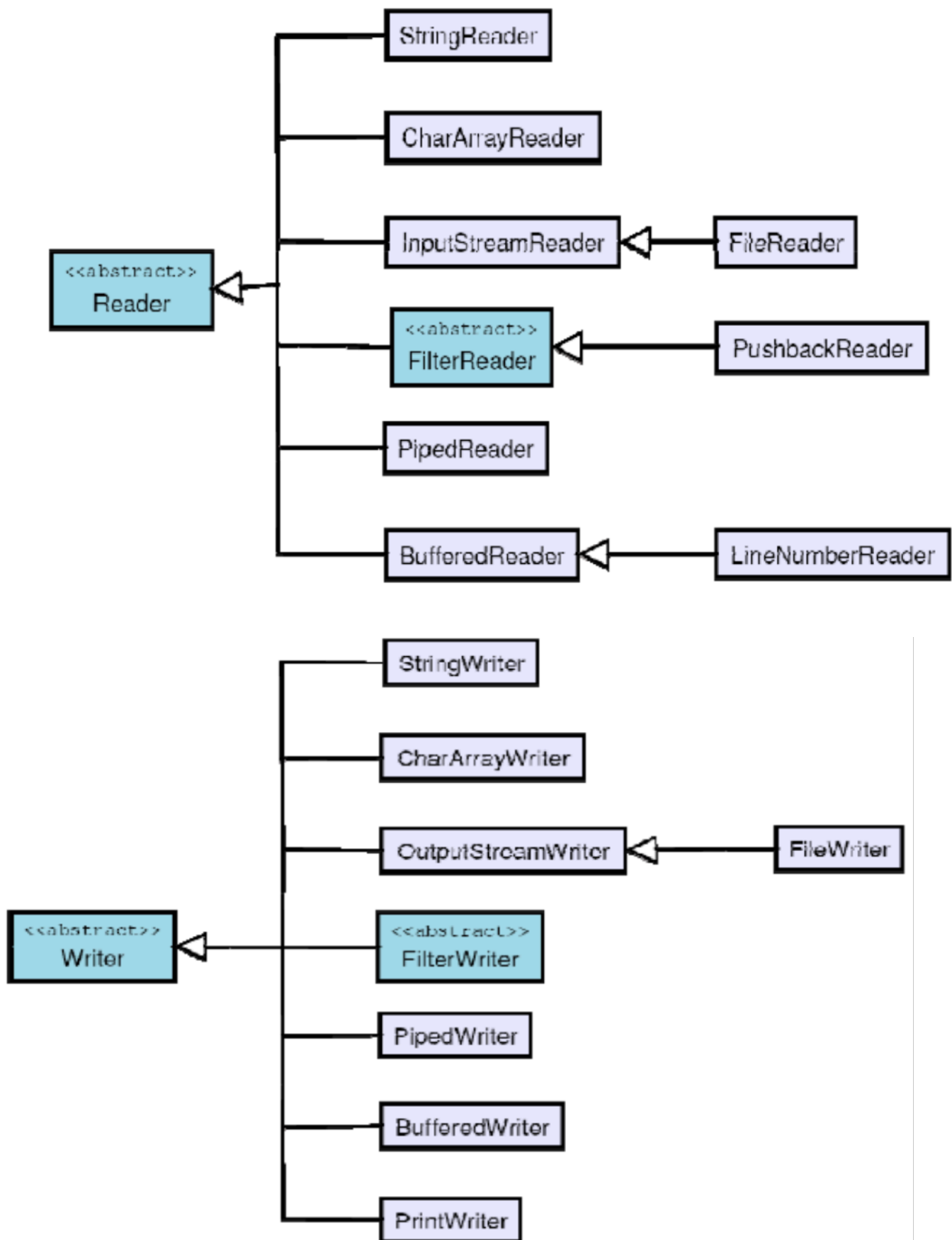
Byte-oriented gestisce i dati a livello binario, cioè byte grezzi (8 bit). Serve per file binari: immagini, audio, PDF, file compressi, ecc. Le classi principali sono:

- `InputStream` : per leggere byte.
- `OutputStream` : per scrivere byte.

Char-oriented gestisce i dati a livello di caratteri (char), cioè testo leggibile. Converte automaticamente i byte in caratteri secondo una codifica (UTF-8, ASCII...). Le classi principali sono:

- **Reader** : per leggere testo.
- **Writer** : per scrivere testo.





Si noti che, nei due esempi di codice precedenti, compaiono le classi `FileInputStream` e `FileOutputStream`.

La loro presenza non è casuale: è dovuta al fatto che in entrambi i casi si stanno gestendo flussi di byte (*byte stream*).

Quando vuoi leggere o scrivere **testo** (non byte grezzi), si usano i *character stream*.

Lettura e scrittura con `FileReader` e `FileWriter` (character stream).

```
import java.io.FileReader;
import java.io.FileWriter;
import java.io.IOException;

public class CopiaFileTesto {
    public static void main(String[] args) {
        try {
            FileReader reader = new FileReader("input.txt");
            FileWriter writer = new FileWriter("copia.txt");

            int carattere;
            while ((carattere = reader.read()) != -1) {
                writer.write(carattere);
            }

            reader.close();
            writer.close();

            System.out.println("File copiato con successo!");
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

Usa `Reader` e `Writer` per leggere/scrivere caratteri Unicode.

Perfetto per file di testo. Crea un file chiamato “copia.txt” con il contenuto di “input.txt”.

BUFFER

Un **buffer** è una zona di memoria *temporanea* che raccoglie un blocco di dati prima di passarli (in lettura o scrittura) al flusso effettivo.

In **lettura**, il buffer legge grandi quantità di dati in un colpo solo dal file, e poi il programma pesca da lì un carattere alla volta.

In **scrittura**, il buffer accumula i dati in memoria e li scrive sul file solo quando il buffer è pieno o quando lo si chiude.

Quindi non cambia il tipo di dati (sono sempre caratteri), ma **cambia il modo in cui avviene l'accesso fisico**.

Le classi `BufferedReader` e `BufferedWriter`

`BufferedReader`

È una classe di input bufferizzato per caratteri, che avvolge (wrappa) un `Reader` esistente. Serve per leggere testo in modo più efficiente e comodo.

```
import java.io.*;

public class LetturaBuffer {
    public static void main(String[] args) throws IOException {
        FileReader fr = new FileReader("testo.txt");
        BufferedReader br = new BufferedReader(fr); // Avvolge il
FileReader - occhio eh

        String linea;
        while ((linea = br.readLine()) != null) {
            System.out.println(linea);
        }

        br.close();
    }
}
```

Cosa cambia rispetto a `FileReader`:

- puoi leggere intere righe con `.readLine()`;
- meno accessi fisici al file: **più veloce**;
- meno codice: **più leggibile**.

`BufferedWriter`

È la controparte in output, che bufferizza la scrittura dei caratteri.

```
import java.io.*;
```

```

public class ScritturaBuffer {
    public static void main(String[] args) throws IOException
    {
        FileWriter fw = new FileWriter("output.txt");
        BufferedWriter bw = new BufferedWriter(fw);

        bw.write("Ciao a tutti!");
        bw.newLine(); // va a capo
        bw.write("Questa è una seconda riga.");
        bw.close(); // forza la scrittura del buffer
    }
}

```

Crea un file con scritto dentro:

Ciao a tutti!
Questa è una seconda riga.

Cosa cambia rispetto a `FileWriter`:

- scrive in memoria prima, poi sul disco in blocco;
- evita molte chiamate costose di I/O;
- può usare metodi comodi come `.newLine()`.

`BufferedReader` e `BufferedWriter` non sostituiscono `FileReader` e `FileWriter`, ma li **avvolgono**, aggiungendo un livello di efficienza e comodità.

Immagina una serie di bambole matrioske, una dentro l'altra. La più piccola è nascosta al centro, e ogni bambola più grande **avvolge** la precedente, aggiungendole qualcosa magari una decorazione o una funzione in più.

In Java, la logica è la stessa. Ogni *stream* può essere **avvolto da un altro stream** per aggiungere funzionalità, esattamente come una bambola più grande contiene quella più piccola.

```

FileReader file = new FileReader("testo.txt");
BufferedReader buffer = new BufferedReader(file);

```

In questo caso:

- la **bambola più piccola** è il `FileReader`: si occupa di leggere i caratteri dal file, in modo semplice e diretto;
- la **bambola più grande** è il `BufferedReader`: avvolge il `FileReader` e aggiunge la capacità di leggere **più velocemente**, con metodi più comodi (`readLine()`, ad esempio).

Le classi `StringReader` e `StringWriter`

`StringReader` è una classe del package `java.io` che permette di leggere dati da una stringa come se fosse un file.

```
import java.io.*;

public class EsempioStringReader {
    public static void main(String[] args) throws IOException
    {
        String testo = "Ciao!\nMe chiamano er pupazzo\nMa nun  
so mai l'andazzo!";

        StringReader reader = new StringReader(testo);
        int c;
        while ((c = reader.read()) != -1) {
            System.out.print((char) c);
        }
        reader.close();
    }
}
Out:
Ciao!
Me chiamano er pupazzo
Ma nun so mai l'andazzo!
```

In pratica, ti permette di **simulare un file di testo** direttamente da una variabile `String`.

`StringReader` crea un flusso di caratteri **a partire dalla stringa** testo.

Il programma la legge **uno alla volta**, proprio come farebbe leggendo da un file con `FileReader`.

Non serve nessun file fisico: **tutto avviene in memoria**.

Esempio con BufferedReader + StringReader

```
import java.io.*;

public class BufferSuStringa {
    public static void main(String[] args) throws IOException
    {
        String testo = ""
            Prima riga
            Seconda riga
            Terza riga

        \""; //ce deve sta il punto e virgola

        BufferedReader reader = new BufferedReader(new
StringReader(testo));

        String linea;
        while ((linea = reader.readLine()) != null) {
            System.out.println(">> " + linea);
        }
        reader.close();
    }
}
```

Out:

```
>> Prima riga
>> Seconda riga
>> Terza riga
```

StringWriter è una classe del package `java.io` che ti permette di scrivere testo in memoria, dentro una `stringa`, invece che su un file o su un dispositivo esterno. In altre parole: mentre `FileWriter` scrive su un file fisico, `StringWriter` scrive dentro una `String` in RAM.

```
import java.io.StringWriter;

import java.io.IOException;
```

```

public class EsempioStringWriter {
    public static void main(String[] args) throws IOException
    {
        StringWriter sw = new StringWriter();

        sw.write("Ciao mondo!");
        sw.write("\n");
        sw.write("Questo testo non è su un\n file, ma in
memoria.");

        // ottengo la stringa completa
        String risultato = sw.toString();
        System.out.println("Contenuto del buffer:");
        System.out.println(risultato);

        sw.close(); // opzionale, ma buona abitudine (tu
fallo)
    }
}

```

Out:
Ciao mondo!
Questo testo non è su un
file, ma in memoria.

Esempio con **BufferedWriter** + **StringWriter**

```

import java.io.*;

public class BufferSuStringWriter {
    public static void main(String[] args) throws IOException
    {
        StringWriter sw = new StringWriter();
        BufferedWriter bw = new BufferedWriter(sw);

        bw.write("Prima riga");
        bw.newLine();
        bw.write("Seconda riga");
        bw.newLine();
        bw.write("Terza riga");

        bw.flush(); // forza la scrittura nel buffer interno
        bw.close();
    }
}

```

```

        // Ottengo la stringa finale
        System.out.println("Risultato finale:");
        System.out.println(sw.toString());
    }
}

```

Out:

Risultato finale:

Prima riga

Seconda riga

Terza riga

StringReader + StringWriter insieme

```

import java.io.*;

public class StringReaderWriterDemo {
    public static void main(String[] args) throws IOException
    {
        String input = "Linea 1\nLinea 2\nLinea 3";

        StringReader reader = new StringReader(input);
        StringWriter writer = new StringWriter();

        int c;
        while ((c = reader.read()) != -1) {
            writer.write(Character.toUpperCase(c)); // scrivo
            la versione maiuscola
        }

        System.out.println("Risultato trasformato:");
        System.out.println(writer.toString());
    }
}

```

Out:

Risultato trasformato:

LINEA 1

LINEA 2

LINEA 3

Puoi anche usarli **insieme**, per simulare un'operazione di lettura/scrittura completa **senza mai toccare il disco**.

Adesso entriamo in una parte **fondamentale e molto interessante** del mondo degli stream in Java: quelli che lavorano su **dati binari strutturati**, cioè numeri, boolean, double, char, stringhe in formato binario, ecc.

È qui che entra in gioco la classe **DataInputStream** (e la sua “gemella **DataOutputStream**”).

DataInputStream e DataOutputStream

DataInputStream è una classe del package `java.io` che **permette di leggere dati primitivi** (`int`, `float`, `boolean`, `char`, `double`, ecc.) **da uno stream di byte** in modo comodo e strutturato.

In pratica, è una *decorazione* (una bambola Matrioska) che si mette sopra un `InputStream` (es. `FileInputStream`) per leggere **valori binari con significato**, non solo byte grezzi.

DataInputStream avvolge un flusso di byte (come `FileInputStream`) e fornisce **metodi di alto livello** per leggere tipi di dati primitivi:

- `readInt()`
- `readDouble()`
- `readBoolean()`
- `readUTF()` → legge stringhe codificate in UTF-8
- `readChar()`
- `readFloat()`, ecc.

Ma come si scrivono quei dati primitivi?

Con la classe gemella: **DataOutputStream**, che avvolge un `FileOutputStream`.

```
import java.io.*;

public class EsempioDataOutputStream {
    public static void main(String[] args) throws IOException
    {
        FileOutputStream fos = new
FileOutputStream("dati.bin");
        DataOutputStream dos = new DataOutputStream(fos);

        dos.writeInt(42);
        dos.writeDouble(3.14159);
        dos.writeBoolean(true);

        dos.close();

        System.out.println("Dati scritti correttamente!");
    }
}
```

Cosa è successo? I dati sono stati scritti correttamente su un file con estensione *.bin*. Un file binario cos'è?

Un file binario è un file che contiene dati codificati in formato binario puro (0 e 1), invece che testo leggibile. In un file binario, il contenuto è formato da **sequenze di byte** che rappresentano informazioni strutturate: numeri, immagini, suoni, oggetti, ecc. Nel file `dati.bin`, non troverai parole o numeri leggibili, ma una sequenza di byte codificati. Non è pensato per essere letto da un umano, ma da un programma. È più compatto, veloce da leggere/scrivere e perfetto per memorizzare numeri, immagini, oggetti o dati strutturati.

Output sul file binario dopo l'esecuzione di questo snippet:



Immagina di dover spedire una valigia da Roma a Milano.

Dentro ci metti: un libro (un numero intero), una bottiglia d'acqua (un numero decimale), un biglietto con scritto "vero" o "falso" (un valore boolean).

Tu **non scrivi sul coperchio** della valigia "Questo è un libro, questo è un numero, questo è un booleano"; semplicemente **metti dentro gli oggetti** così come sono,

ben confezionati e ordinati. Quando la valigia arriva a destinazione, chi la riceve **non la può aprire a caso**: deve sapere **in quale ordine** hai messo le cose. Se tu hai messo prima il libro, poi la bottiglia, poi il biglietto, anche lui dovrà estrarli nello stesso ordine.

Ecco, il file binario funziona esattamente così: la “valigia” è il file **.bin**, gli “oggetti” sono i dati (int, double, boolean, ecc.), l’“ordine” con cui li hai scritti è fondamentale per rileggerli correttamente.

Ed è quello che ora farò per leggere i dati in chiaro da un file binario .

```
import java.io.*;

public class EsempioDataInputStream {
    public static void main(String[] args) throws IOException
    {
        // Apro uno stream di byte sul file
        FileInputStream fis = new
FileInputStream("dati.bin");
        // Lo avvolgo con un DataInputStream, dentro alla
culla sto scemo
        DataInputStream dis = new DataInputStream(fis);

        // Leggiamo i dati nel giusto ordine
        int numero = dis.readInt();
        double valore = dis.readDouble();
        boolean flag = dis.readBoolean();

        dis.close();

        System.out.println("Numero: " + numero);
        System.out.println("Valore: " + valore);
        System.out.println("Flag: " + flag);
    }
}

Out:
Numero: 42
Valore: 3.14159
Flag: true
```

Sono i dati di prima nella scrittura.

Se adesso nella lettura inverte l'ordine, quindi leggo prima il valore double, succede un undefined behaviour.

L'ordine e il tipo dei dati devono essere identici in scrittura e in lettura.

Chi legge il file deve conoscere esattamente l'ordine e il tipo dei dati scritti.

ByteArrayInputStream

ByteArrayInputStream è una classe del package java.io che permette di leggere byte da un array di byte in memoria, come se quell'array fosse un file.

In altre parole: è un *InputStream virtuale* che non legge da un disco, ma da un blocco di memoria (RAM).

```
import java.io.ByteArrayInputStream;
import java.io.DataInputStream;
import java.io.EOFException;
import java.io.IOException;

public static void main(String[] args) {
    String s2 = "this is foo!!";

    try {
        // Creiamo uno stream che legge byte dalla stringa
        // (in memoria)
        DataInputStream in3 = new DataInputStream(new
        ByteArrayInputStream(s2.getBytes()));

        // Leggiamo byte per byte e li stampiamo come
        caratteri
        while (true)
            System.out.print((char) in3.readByte());

    } catch (EOFException e) {
        System.err.print("End of stream");
    } catch (IOException e) {
        e.printStackTrace();
    }
}

Out:
this is foo!!End of stream
```

PrintWriter

PrintWriter è una classe di alto livello (del package java.io) che serve per scrivere testo in modo semplice e comodo, molto più leggibile di FileWriter o BufferedWriter.

In sintesi, il suo scopo è fornire un modo semplice e diretto per scrivere testo su file, rete, o console, con la stessa sintassi di System.out. Diciamo che è una classe molto bella :)

```
import java.io.*;

public class EsempioPrintWriter {
    public static void main(String[] args) {
        String s2 = "this is foo!!";

        try {
            // Lettura da memoria tramite StringReader
            BufferedReader in3 = new BufferedReader(new
StringReader(s2));

            // Scrittura su file con PrintWriter (avvolto da
BufferedWriter e FileWriter)
            PrintWriter out1 = new PrintWriter(new BufferedWriter(
                new FileWriter("file.txt")));

            int lineCount = 0;
            String s;

            // Leggiamo riga per riga dalla stringa (simulando un file)
            while ((s = in3.readLine()) != null) {
                lineCount++;
                out1.println(lineCount + ": " + s); // scrive su file
            }

            out1.println(); // va a capo finale
            out1.close();   // chiude il flusso

            System.out.println("Scrittura completata! Controlla il
file.txt");

        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

Il file.txt avrà:

```
1: this is foo!!
```


Il programma Crea una stringa in memoria (s2 = "this is foo!!").

Usa un `StringReader` e un `BufferedReader` per leggere la stringa come se fosse un file di testo. Crea un `PrintWriter` (avvolto dentro un `BufferedWriter` e un `FileWriter`) per scrivere su file. Legge le righe una per una, le numerizza e le scrive in `file.txt`.

A questo punto la domanda diventa legittima.

Esiste anche un `PrintReader` per leggere testo così facilmente come `PrintWriter` lo scrive? no, `PrintReader` non esiste in Java.

Contesto: “Output (e Input) di byte su file”

```
import java.io.*;

public class EsempioDataStream {
    public static void main(String[] args) {
        try {
            // =====
            //  SCRITTURA BINARIA SU FILE
            //  =====

            // Creiamo un DataOutputStream "a matrioska"
            // (ogni livello aggiunge una funzione)
            DataOutputStream out2 = new DataOutputStream(
                new BufferedOutputStream(
                    new
FileOutputStream("data.dat")));

            // Scriviamo coppie di valori: (double, stringa)
            out2.writeDouble(3.14159);                // Pi
greco
            out2.writeUTF("That was pi");            //
Messaggio testuale
            out2.writeDouble(1.41413);                //
Radice di 2
            out2.writeUTF("Square root of 2");        //
Messaggio testuale
            out2.close();                             //
Chiusura del flusso (scrive il buffer)
```

```

        System.out.println(" Scrittura completata su
data.dat");

        // =====
        //  LETTURA BINARIA DA FILEEEDAJE
        //  =====

        // Creiamo il DataInputStream corrispondente
        DataInputStream in5 = new DataInputStream(
            new BufferedInputStream(
                new
FileInputStream("data.dat")));

        // Lettura nello stesso ordine in cui i dati sono
stati scritti
        double valore1 = in5.readDouble();
        String testo1 = in5.readUTF();
        double valore2 = in5.readDouble();
        String testo2 = in5.readUTF();

        in5.close(); // chiude il flusso di input

        // =====
        //  OUTPUT A VIDEOOOLLÉ
        //  =====
        System.out.println("\n Contenuto letto da
file:");

        System.out.println("Valore 1: " + valore1);
        System.out.println("Testo 1:  " + testo1);
        System.out.println("Valore 2: " + valore2);
        System.out.println("Testo 2:  " + testo2);

    } catch (IOException e) {
        e.printStackTrace();
    }
}

Out.
Contenuto letto da file:
Valore 1: 3.14159
Testo 1:  That was pi
Valore 2: 1.41413
Testo 2:  Square root of 2

```

Questo esempio riguarda proprio i **byte stream**, ma “**intelligenti**”, cioè in grado di leggere e scrivere **valori primitivi** (**int**, **double**, **String**) in formato binario.

Per farlo si usano **DataOutputStream** e **DataInputStream**.

Come prima, è fondamentale mantenere lo stesso ordine e tipo di lettura e scrittura, altrimenti i byte vengono interpretati male.

ANALOGIA E BREVE RIFLESSIONE

Il sistema di I/O in Java è l'esempio perfetto del **Decorator Pattern**.

ogni stream è un fluido che scorre attraverso condotti,
e ogni condotto aggiunge una funzione: filtrare, bufferizzare, interpretare.

Alla fine, il flusso resta lo stesso, cambiano solo gli ‘strati’ che lo decorano.



Attenzione:

Il pattern Decorator è da sapere come l'Ave Maria: “grazia plena, Dominus tecum”.

È il cuore dell'intero sistema di I/O di Java, tutto il package `java.io` si basa su di lui.

Capirlo significa comprendere perché gli stream si “incapsulano” a matryoska, come si compongono e come aggiungono funzionalità senza modificare la struttura originaria. Vedere miei documenti sul pattern decorator.

java.io.RandomAccessFile

Questa è una delle lezioni più importanti del Prof. G. De Angelis, perché introduce un concetto che rompe la logica classica dello *stream sequenziale*: il **Random Access File**, cioè un file ad **accesso casuale**, dove puoi *saltare direttamente* a qualsiasi posizione e leggere o scrivere lì.

La classe `java.io.RandomAccessFile` è una **classe indipendente** (non appartiene né alla gerarchia `InputStream` né `Reader`), ma serve per leggere e scrivere file binari **in qualunque punto**, non solo in modo sequenziale.

Sintassi base

```
RandomAccessFile rf = new RandomAccessFile("file.dat", "rw");
```

Il primo parametro è il **nome del file**.

Il secondo parametro indica la **modalità di accesso**:

- "r" → *read only* (solo lettura),
- "rw" → *read & write* (lettura e scrittura).

Ora prepariamoci a viaggiare verso le stelle e i pianeti, userò un'analogia cosmica.

Quello che voglio lasciarvi con questo documento non è la capacità di ripetere i codici a memoria, ma di comprendere davvero i concetti che li muovono.



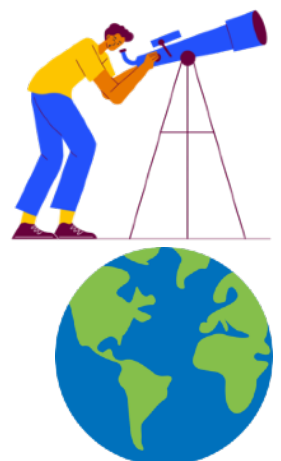
Immagina l'universo come un enorme cielo stellato, pieno di punti luminosi, ognuno rappresenta un byte nel nostro file.

Se usiamo gli stream classici (FileInputStream, DataInputStream, ecc.), possiamo solo guardare il cielo in linea retta, stella dopo stella, come un raggio di luce che attraversa l'universo da un punto all'altro.

Ci muoviamo sequenzialmente, dal primo byte all'ultimo, senza possibilità di tornare indietro o saltare una zona.

Poi arriva RandomAccessFile. Con RandomAccessFile, invece, diventi un astronomo con un telescopio interstellare.

Non devi più seguire un percorso unico nel cielo dei dati,



ora puoi puntare il telescopio verso qualunque stella, in qualunque punto dell'universo del file.

Puoi dire: `rf.seek(5 * 8);` e in un istante ti **teletrasporti** alla “quinta stella”, al byte numero 40 del cosmo binario.

Il `RandomAccessFile` è fuori dal tempo e dallo spazio sequenziale.

È un astronomo digitale che osserva e modifica il firmamento dei byte senza muoversi lungo un percorso. In un istante può guardare la prima stella o l'ultima, come se il file fosse un universo senza direzione privilegiata.

Ogni tipo primitivo ha una **dimensione** fissa, definita dallo standard Java:

Tipo di dato	Dimensione	Cosa rappresenta
boolean	1 byte	vero/falso
byte	1 byte	numero intero da -128 a 127
char	2 byte	un carattere Unicode
short	2 byte	intero corto (-32.768 → 32.767)
int	4 byte	intero (-2 miliardi → +2 miliardi)
float	4 byte	numero reale (precisione singola)
long	8 byte	intero molto grande
double	8 byte	numero reale (precisione doppia)

Di seguito un esempio di codice.

```
import java.io.IOException;
import java.io.RandomAccessFile;

public class UniversoRandomAccess {
    public static void main(String[] args) {
        try {
            // =====
            // Creazione del "file universo"
            // =====
            RandomAccessFile universo = new
RandomAccessFile("galassia.dat", "rw");
```

```

        // Creiamo 10 stelle (ognuna rappresentata da un
valore double)
        for (int i = 0; i < 10; i++) {
            universo.writeDouble(i * 1.414); // ogni stella ha
una "luminosità" diversa
        }
        universo.close();
        System.out.println(" Universo creato con 10 stelle
(galassia.dat)");

        // =====
        //  Teletrasporto alla stella n°5
        //  =====
        universo = new RandomAccessFile("galassia.dat", "rw");

        // Ogni double occupa 8 byte → la stella n°5 inizia a
byte 40
        universo.seek(5 * 8);                // spostamento
interstellare
        universo.writeDouble(47.0001);        // trasformo quella
stella in una supernova
        universo.close();
        System.out.println(" Stella n°5 trasformata in una
supernova!");

        // =====
        //  Osserviamo di nuovo tutte le stelle
        //  =====
        universo = new RandomAccessFile("galassia.dat", "r");

        System.out.println("\n Catalogo stellare
aggiornato:");
        for (int i = 0; i < 10; i++) {
            System.out.println("Stella " + i + ": " +
universo.readDouble());
        }

        universo.close();
        System.out.println("\n Esplorazione completata con
successo!");

    } catch (IOException e) {
        e.printStackTrace();
    }
}
}
Out:

```

```

Universo creato con 10 stelle (galassia.dat)
Stella n°5 trasformata in una supernova!

```

```

Catalogo stellare aggiornato:

```

```

Stella 0: 0.0
Stella 1: 1.414
Stella 2: 2.828
Stella 3: 4.242
Stella 4: 5.656
Stella 5: 47.0001
Stella 6: 8.484
Stella 7: 9.898
Stella 8: 11.312
Stella 9: 12.725999999999999

```

Esplorazione completata con successo!

Java si sposta direttamente al byte n°40, cioè all’inizio del 6° valore (indice 5), e lì sovrascrive i successivi 8 byte con il nuovo double 47.0001.

Per chi ha superato l’esame di Sistemi Operativi con il Prof.Quaglia non ha problemi con la seek().

Ogni double occupa 8 byte, quindi:

Stella	Byte iniziale	Byte finale	Valore scritto
0	0	7	0.0
1	8	15	1.414
2	16	23	2.828
3	24	31	4.242
4	32	39	5.656
5	40	47	7.07 ← (questa la modifico)
6	48	55	8.484
7	56	63	9.898
8	64	71	11.312
9	72	79	12.726

Se avessi scritto seek(5) mi sarei spostato solo di 5 byte dall’inizio del file, cioè a metà del primo valore, e avrei “sporcato” il file binario.

I byte non corrisponderebbero più a double completi e readDouble() non capirebbe più niente. Mi uscirebbero valori “senza senso” (es. 3.427E-302), o addirittura un’eccezione EOFException.

Quando scrivo `seek(5 * 8)` non sto andando “alla quinta stella”,
ma **saltando avanti di 40 anni luce**, la distanza precisa che separa la stella n°0 dalla n°5.

Se mi fermassi prima, **finirei nel mezzo del vuoto quantico**, dove i byte non formano più
nessuna struttura comprensibile

`RandomAccessFile` è una classe a sé stante, non eredita da `InputStream` o `Reader`.

Tuttavia, **implementa le interfacce `DataInput` e `DataOutput`**,
quindi **offre gli stessi metodi** di `DataInputStream` e `DataOutputStream`:
`writeDouble()`, `readDouble()`, `writeUTF()`, `readInt()`, ecc.

La persistenza degli oggetti e la serializzazione in Java.

In Java (e più in generale in programmazione orientata agli oggetti), ogni **oggetto** ha un **ciclo di vita ben preciso**. Un oggetto nasce e muore **nella memoria volatile (RAM)**. Più precisamente: **viene allocato** spazio in memoria (`new NomeClasse()`). **Viene costruito** tramite un costruttore (che inizializza i campi). Vive fintanto che è referenziato nel programma. **Muore** quando viene eliminato esplicitamente (in C++ con `delete`); oppure, in Java, quando il **Garbage Collector** lo rileva come non più usato e ne libera la memoria; o infine, alla terminazione dell'applicazione che lo aveva creato.

In altre parole, quando chiudi il programma, tutti gli oggetti spariscono. Finché vivi in RAM, nulla è persistente.

Ma la realtà del software è diversa.

Esistono casi in cui vorremmo che un oggetto “sopravvivesse” alla chiusura del programma. Questo bisogno si chiama **persistenza degli oggetti**: conservare lo stato di un oggetto oltre la durata del processo che lo ha creato.

Il prof. De Angelis distingue due grandi strade “classiche” per gestire la persistenza:

Metodo	Vantaggi
File tradizionali	Facili da usare, leggeri
Database (DBMS)	Strutturati, potenti, multiutente

Java introduce un meccanismo **nativo e automatico** per salvare e recuperare oggetti: la **serializzazione** (serialization) e la **deserializzazione** (deserialization).



Cos'è la serializzazione?

Serializzare significa tradurre un oggetto in una sequenza di byte binaria in modo che possa essere:

- salvato su un file;
- inviato su rete;
- o archiviato in qualsiasi supporto persistente.

Questa sequenza binaria contiene lo stato interno dell'oggetto, e durante la deserializzazione può essere ricostruita in un nuovo oggetto identico all'originale.

Durante la serializzazione, Java salva tutti i **campi primitivi** (int, double, boolean...) e tutti gli **oggetti referenziati** che siano anch'essi serializzabili (ricorsivamente). È una **fotografia completa** dello stato di quell'oggetto nel momento della scrittura.

Cos'è la deserializzazione?

Deserializzare significa ricostruire l'oggetto originale leggendo i byte salvati e ricreando i campi e i riferimenti come erano al momento della scrittura.

Come sempre in Java, anche qui il concetto passa attraverso gli **stream**.

- Per **scrivere** (serializzare) si usa: **ObjectOutputStream**;
- Per **leggere** (deserializzare) si usa: **ObjectInputStream**;

TAGGING INTERFACE

Condizione essenziale: implementare Serializable!

Affinché una classe sia serializzabile, deve implementare l'interfaccia **Serializable**, ossia un'interfaccia senza metodi (una tagging interface).

Inoltre, questa interfaccia **non richiede la definizione di nessun metodo** dalla classe che la realizza. Serve solo a dire alla JVM: “Carissima JVM, occhio che questa classe può essere serializzata”.

```
class Pianeta implements Serializable { ... }
```

Cosa succede se una classe non è serializzabile?



Se provi a scrivere su file un oggetto di una classe che non implementa `Serializable`, Java lancerà un'eccezione.

L'eccezione proviene da: [`java.io.NotSerializableException`](#)!

Serializable e ereditarietà.

Capiamo bene.

In Java, la serializzazione è **ereditaria**.

Significa che se una classe è serializzabile, anche tutte le sue sottoclassi lo sono automaticamente.

Se dichiariamo:

```
class Pianeta implements Serializable { ... }
```

```
class Terra extends Pianeta { ... }
```

allora anche `Terra` sarà automaticamente serializzabile. Non serve riscrivere `implements Serializable`.

COSA DICE LA DOCUMENTAZIONE UFFICIALE JAVADOC?

1. All subtypes of a serializable class are themselves serializable.

Tutte le sottoclassi di una classe serializzabile sono automaticamente serializzabili.

2. To allow subtypes of non-serializable classes to be serialized, the subtype may assume responsibility for saving and restoring the state of the supertype's fields...!

Se la superclasse non è serializzabile, la sottoclasse può comunque esserlo. Ma deve **farsi carico lei** di salvare e ripristinare i campi ereditati. E può farlo **solo** se la superclasse ha un costruttore “vuoto” (**no-arg**) e accessibile (public o protected).

```
class CorpoCeleste {  
    double massa;  
}
```

```
class Pianeta extends CorpoCeleste implements Serializable {  
    String nome;  
}
```

In questo caso CorpoCeleste non è serializzabile; Pianeta lo è.

Allora la sottoclasse (Pianeta) diventa responsabile di salvare lo stato ereditato dalla superclasse (massa) e ripristinarlo durante la deserializzazione. E può farlo solo se la superclasse ha un costruttore senza argomenti (no-arg) pubblico o protetto, che la JVM può chiamare per inizializzare correttamente i suoi campi.

3. During deserialization, the fields of non-serializable classes will be initialized using the public or protected no-arg constructor.

In pratica quando deserializzi, la JVM chiama prima **il costruttore vuoto** della superclasse **non serializzabile**, poi ricostruisce i campi della sottoclasse leggendo dal flusso binario. Questo garantisce che **l'oggetto venga inizializzato in modo coerente** anche se la superclasse non è stata salvata nel file.

4. Classes that require special handling during serialization must implement special methods with these exact signatures.

Se vuoi **personalizzare** la serializzazione o deserializzazione, puoi aggiungere alla tua classe tre metodi privati “speciali”, che la JVM riconosce **automaticamente** se esistono.

```
private void writeObject(ObjectOutputStream out) throws IOException
private void readObject(ObjectInputStream in) throws IOException, ClassNotFoundException
private void readObjectNoData() throws ObjectStreamException
```

Questi servono per **controllare manualmente** come viene salvato e ricostruito l’oggetto.

Esempio completo.

```
import java.io.*;

class CorpoCeleste {
    double massa;
    public CorpoCeleste() {
        //massa = 0;
        System.out.println("Costruttore CorpoCeleste chiamato");
    }
}

class Pianeta extends CorpoCeleste implements Serializable {
    String nome;
    public Pianeta(String n, double m) {
        nome = n;
        massa = m;
    }

    public String toString() {
        return "Pianeta: " + nome + " (massa = " + massa + ")";
    }
}

public class EreditarietaSerializzazione {
    public static void main(String[] args) throws Exception {
        Pianeta terra = new Pianeta("Terra", 5.97e24);

        // SERIALIZZAZIONE
        ObjectOutputStream out = new ObjectOutputStream(new
FileOutputStream("pianeta.dat"));
        out.writeObject(terra);
        out.close();

        // DESERIALIZZAZIONE
```

```

        ObjectInputStream in = new ObjectInputStream(new
FileInputStream("pianeta.dat"));
        Pianeta copia = (Pianeta) in.readObject();
        in.close();

        System.out.println(copia);
    }
}

```

Out:

```

Costruttore CorpoCeleste chiamato
Costruttore CorpoCeleste chiamato
Pianeta: Terra (massa = 0.0)

```

La superclasse `CorpoCeleste` non è serializzabile, i suoi campi non vengono salvati nel flusso!

Durante la deserializzazione, la JVM chiama il costruttore vuoto della superclasse per inizializzarla, perciò massa torna al valore predefinito.

La massa è tornata a 0.0.

Java scrive sul file solo i campi appartenenti a classi serializzabili.

Quindi i campi di `Pianeta` (cioè nome) vengono salvati. I campi ereditati da

`CorpoCeleste` (cioè massa) vengono ignorati, perché la superclasse non è serializzabile.

Come si può risolvere?

Sei tu programmatore a gestire la serializzazione personalizzata aggiungendo i metodi “speciali”. Così la sottoclasse `Pianeta` diventa davvero responsabile anche dei campi ereditati dalla superclasse non serializzabile.

```

private void writeObject(ObjectOutputStream out) throws IOException {
    out.defaultWriteObject(); // serializza normalmente i campi
    out.writeDouble(massa);   // aggiungi la massa manualmente
}

private void readObject(ObjectInputStream in)
    throws IOException, ClassNotFoundException {
    in.defaultReadObject();   // deserializza i campi standard
    massa = in.readDouble();  // rilegge la massa
}

```

Quindi, a questo punto il codice completo risolto:

```

import java.io.*;
/* Simone Remoli */
// SUPERCLASSE NON SERIALIZZABILE
class CorpoCeleste {
    protected double massa;

    // Costruttore senza argomenti (necessario per la deserializzazione)
    public CorpoCeleste() {
        System.out.println("Costruttore CorpoCeleste chiamato");
    }

    public CorpoCeleste(double massa) {
        this.massa = massa;
    }
}

// SOTTOCLASSE SERIALIZZABILE
class Pianeta extends CorpoCeleste implements Serializable {
    private String nome;

    public Pianeta(String nome, double massa) {
        this.nome = nome;
        this.massa = massa;
    }

    public String toString() {
        return "Pianeta: " + nome + " (massa = " + massa + ")";
    }

    // --- Metodo speciale per personalizzare la serializzazione ---
    private void writeObject(ObjectOutputStream out) throws IOException {
        System.out.println(" Scrittura personalizzata di Pianeta");
        out.defaultWriteObject();    // serializza i campi di Pianeta
(nome)
        out.writeDouble(massa);    // salva manualmente il campo
ereditato
    }

    private void readObject(ObjectInputStream in)
        throws IOException, ClassNotFoundException {
        System.out.println(" Lettura personalizzata di Pianeta");
        in.defaultReadObject();    // deserializza i campi di Pianeta
(nome)
        massa = in.readDouble();    // ripristina il campo ereditato
    }
}

public class EsempioSerializzazioneEreditarieta {
    public static void main(String[] args) throws Exception {
        Pianeta terra = new Pianeta("Terra", 5.97e24);

        // --- SERIALIZZAZIONE ---
        ObjectOutputStream out = new ObjectOutputStream(
            new FileOutputStream("pianeta.dat"));
        out.writeObject(terra);
        out.close();
    }
}

```

```

        System.out.println("\n Oggetto serializzato con successo!\n");

        // --- DESERIALIZZAZIONE ---
        ObjectInputStream in = new ObjectInputStream(
            new FileInputStream("pianeta.dat"));
        Pianeta copia = (Pianeta) in.readObject();
        in.close();

        System.out.println("\n Oggetto deserializzato:");
        System.out.println(copia);
    }
}

```

Out:

Costruttore CorpoCeleste chiamato
Scrittura personalizzata di Pianeta

Oggetto serializzato con successo!

Costruttore CorpoCeleste chiamato
Lettura personalizzata di Pianeta

Oggetto deserializzato:
Pianeta: Terra (massa = 5.97E24)

Nota che: System.out.println(copia) = System.out.println(copia.toString());

CorpoCeleste **non è serializzabile** (non implementa Serializable); contiene un campo massa; ha un costruttore vuoto pubblico, fondamentale per la deserializzazione.

Pianeta è **serializzabile**, perché implementa Serializable; eredita massa da CorpoCeleste, ma poiché la superclasse non è serializzabile, deve occuparsi manualmente di salvare e ripristinare quel campo; lo fa tramite i metodi privati writeObject() e readObject().

Metodo defaultWriteObject(), che cosa fa?

Serializza automaticamente tutti i campi dell'oggetto che appartengono alla classe corrente, escludendo static.

Il “gemello” di defaultWriteObject() è defaultReadObject(), l'analogo nella lettura. defaultWriteObject() è il pilota automatico della serializzazione: si occupa di tutti i campi standard, così tu puoi concentrarti sui dettagli che fanno la differenza.

CAMPI STATIC IN SERIALIZAZIONE

I campi static vengono esclusi durante la serializzazione: la serializzazione riguarda l'oggetto, non la definizione della classe.

I campi di istanza rappresentano lo *stato individuale* di un oggetto. Hanno senso da salvare (fanno parte di *quell'oggetto*).

I campi statici, invece, sono *globali alla classe*. Non appartengono a un oggetto specifico, ma all'intera definizione della classe.

ESEMPIO DI CODICE

```
import java.io.*;

class Pianeta implements Serializable {
    String nome;
    static int numeroPianeti = 0; // campo statico condiviso

    public Pianeta(String n) {
        nome = n;
        numeroPianeti++;
    }

    public String toString() {
        return nome + " (pianeti creati finora = " + numeroPianeti + ")";
    }
}

public class StaticSerializzazione {
    public static void main(String[] args) throws Exception {
        Pianeta terra = new Pianeta("Terra");
        ObjectOutputStream out = new ObjectOutputStream(new
FileOutputStream("pianeta.dat"));
        out.writeObject(terra);
        out.close();

        // Cambiamo il valore statico dopo la serializzazione
        Pianeta.numeroPianeti = 99;

        ObjectInputStream in = new ObjectInputStream(new
FileInputStream("pianeta.dat"));
        Pianeta copia = (Pianeta) in.readObject();
        in.close();

        System.out.println(copia);
    }
}

Out:
Terra (pianeti creati finora = 99)
```

Anche se l'oggetto terra era stato serializzato quando numeroPianeti = 1, il valore statico rimane 99 dopo la deserializzazione. Questo perché numeroPianeti non è mai stato salvato nel file.

Quindi, ricordate che i campi static rappresentano lo stato globale della classe, e non dovrebbero cambiare quando serializzi o deserializzi un singolo oggetto.

Ma questo perché? Perché a logica uno può pensare che lo stato globale della classe non è parte dello stato individuale dell'oggetto. Nient'altro da aggiungere.

Esempio: serializzare due oggetti **Persona** (e poi leggerli entrambi).

```
import java.io.*;

// Classe serializzabile
class Persona implements Serializable {
    private String nome;
    private String cognome;
    private int eta;
    private double altezza;
    private boolean studente;

    public Persona(String nome, String cognome, int eta, double altezza,
boolean studente) {
        this.nome = nome;
        this.cognome = cognome;
        this.eta = eta;
        this.altezza = altezza;
        this.studente = studente;
    }

    public String toString() {
        return " Persona:\n" +
            "Nome: " + nome + "\n" +
            "Cognome: " + cognome + "\n" +
            "Età: " + eta + "\n" +
            "Altezza: " + altezza + " m\n" +
            "Studente: " + (studente ? "Sì" : "No") + "\n";
    }
}

public class SerializzaDuePersone {
    public static void main(String[] args) {
        // Creiamo due oggetti Persona
        Persona persona1 = new Persona("Simone", "Remoli", 24, 1.91,
true);
        Persona persona2 = new Persona("Laura", "Bianchi", 22, 1.68,
false);

        // SERIALIZZAZIONE (salvataggio su file) ---
        try (ObjectOutputStream out = new ObjectOutputStream(new
FileOutputStream("persone.dat"))) {
```

```

        out.writeObject(persona1);
        out.writeObject(persona2);
        System.out.println(" Oggetti salvati correttamente su
'persone.dat'\n");
    } catch (IOException e) {
        e.printStackTrace();
    }

    // --- DESERIALIZAZIONE (lettura da file) ---
    try (ObjectInputStream in = new ObjectInputStream(new
FileInputStream("persone.dat"))) {
        Persona p1 = (Persona) in.readObject();
        Persona p2 = (Persona) in.readObject();

        System.out.println(" Oggetti letti dal file:\n");
        System.out.println(p1);
        System.out.println(p2);
    } catch (IOException | ClassNotFoundException e) {
        e.printStackTrace();
    }
}

```

Out:

Oggetti salvati correttamente su 'persone.dat'

Oggetti letti dal file:

Persona:

Nome: Simone
Cognome: Remoli
Età: 24
Altezza: 1.91 m
Studente: Sì

Persona:

Nome: Laura
Cognome: Bianchi
Età: 22
Altezza: 1.68 m
Studente: No

ESEMPIO CLASSE PADRE SERIALIZZABILE

```
import java.io.*;
```

```
// CLASSE PADRE — è serializzabile
```

```
class CorpoCeleste implements Serializable {
    protected String nome;
    protected double massa;
}
```

```

    public CorpoCeleste(String nome, double massa) {
        this.nome = nome;
        this.massa = massa;
    }

    public String toString() {
        return "CorpoCeleste: " + nome + " (massa = " + massa
+ ")";
    }
}

// CLASSE FIGLIA – NON è serializzabile
class Pianeta extends CorpoCeleste {
    private double distanzaDalSole; // attributo extra

    public Pianeta(String nome, double massa, double
distanzaDalSole) {
        super(nome, massa);
        this.distanzaDalSole = distanzaDalSole;
    }

    public String toString() {
        return "Pianeta: " + nome +
            " (massa = " + massa +
            ", distanza dal sole = " + distanzaDalSole +
" AU)";
    }
}

// PROGRAMMA PRINCIPALE
public class SerializzazioneSoloPadre {
    public static void main(String[] args) {
        Pianeta terra = new Pianeta("Terra", 5.97e24, 1.0);

        // --- SERIALIZZAZIONE ---
        try (ObjectOutputStream out = new
ObjectOutputStream(new FileOutputStream("pianeta.dat"))) {
            out.writeObject(terra);
            System.out.println(" Oggetto scritto su file.");
        } catch (IOException e) {
            e.printStackTrace();
        }

        // --- DESERIALIZZAZIONE ---

```

```

        try (ObjectInputStream in = new ObjectInputStream(new
FileInputStream("pianeta.dat"))) {
            Pianeta copia = (Pianeta) in.readObject();
            System.out.println("\n Oggetto letto dal file:");
            System.out.println(copia);
        } catch (IOException | ClassNotFoundException e) {
            e.printStackTrace();
        }
    }
}

```

OUT:

Oggetto scritto su file.

Oggetto letto dal file:

Pianeta: Terra (massa = 5.97E24, distanza dal sole = 1.0 AU)

Nota.

Serializable in Java riguarda il singolo oggetto e il suo stato.

SERIALIZABLE nel corso di Basi di Dati riguarda **transazioni concorrenti e la loro coerenza**. Ugual nome, **mondi completamente diversi**: uno parla di *byte*, l'altro di *transazioni*.

Adesso procediamo con la lettura e la relativa spiegazione dei codici del Ricercatore
G.De Angelis.

```

import java.io.ByteArrayInputStream;
import java.io.ByteArrayOutputStream;
import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.io.IOException;
import java.io.ObjectInputStream;
import java.io.ObjectOutputStream;
import java.io.Serializable;
import java.util.Random;
import java.util.Vector;

public class ComplexSerializableClass implements Serializable
{
    private static Random RAND = new Random();
    private static int SIZE = 8;

```

```

    private Vector<SimpleSerializableClass> ssc;

// The following attribute is not serializable.
// If line 43 becomes uncommented together with the statement
at line 59,
// an error will be detected at runtime.
//     private Vector<SimpleClass> sc;

    private String instanceName;

    public ComplexSerializableClass(){
        this("NoName");
    }

    public ComplexSerializableClass(String instanceName){
        this.instanceName = instanceName;

        this.ssc = this.configureSimpleSerializableClasses();
// The following attribute "this.sc" is not serializable.
// If line 59 becomes uncommented together with the statement
at line 43,
// an error will be detected at runtime.
//     this.sc = this.configureSimpleClasses();

        System.out.println("instance of
ComplexSerializableClass has been created : " +
this.instanceName);
    }

    private Vector<SimpleSerializableClass>
configureSimpleSerializableClasses(){
        int size = RAND.nextInt(10)+1;

        Vector<SimpleSerializableClass> localSSC = new
Vector<SimpleSerializableClass>();
        for (int i = size; i>0; i--){
            int j = RAND.nextInt(10);
            String label = this.generateRandomString(SIZE);
            localSSC.add(new
SimpleSerializableClass(j,label));
        }

        return localSSC;
    }

```

```

    }

    private Vector<SimpleClass> configureSimpleClasses(){
        int size = RAND.nextInt(10)+1;

        Vector<SimpleClass> localSC = new
Vector<SimpleClass>();
        for (int i = size; i>0; i--){
            int j = RAND.nextInt(10);
            String label = this.generateRandomString(SIZE);
            localSC.add(new SimpleClass(j,label));
        }

        return localSC;
    }

    private String generateRandomString(int length)
    {
        String charsPool =
"ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789";
        char[] text = new char[length];
        for (int i = 0; i < length; i++)
        {
            text[i] =
charsPool.charAt(RAND.nextInt(charsPool.length()));
        }
        return new String(text);
    }

    public String toString() {
        StringBuilder result = new StringBuilder(":");
        result.append(this.instanceName);
        result.append("(");
        for(SimpleSerializableClass s : this.ssc)
            result.append(s);
        result.append(")");
        return result.toString();
    }

    public static void main(String[] args) throws
ClassNotFoundException, IOException {
        ComplexSerializableClass c1 = new
ComplexSerializableClass("c1");
        ObjectOutputStream out = new ObjectOutputStream(new
FileOutputStream("storage.out"));

```

```

        System.out.println("Serializing header message and
instance of c1 = "+ c1.toString()+ "\n");
        out.writeObject("Storage Example\n");
        out.writeObject(c1);
        out.close(); // Also flushes the output

        System.out.println("Retreiving the Serialized Object
(i.e. header message + c1 instance) from the file system");
        ObjectInputStream in = new ObjectInputStream(new
FileInputStream("storage.out"));
        String s = (String)in.readObject();
        ComplexSerializableClass c2 =
(ComplexSerializableClass)in.readObject();
        System.out.println(s + "c2 = " + c2.toString());
        in.close();

        System.out.println();

        System.out.println("Passing the Serialized Object
(i.e. header message + c1 instance) throughout another
stream");
        ByteArrayOutputStream bout = new
ByteArrayOutputStream();
        ObjectOutputStream out2 = new
ObjectOutputStream(bout);
        out2.writeObject("Storage Example\n");
        out2.writeObject(c2);
        out2.flush();
        //
        ObjectInputStream in2 = new ObjectInputStream(new
ByteArrayInputStream(bout.toByteArray()));
        s = (String)in2.readObject();
        ComplexSerializableClass c3 =
(ComplexSerializableClass)in2.readObject();
        System.out.println(s + "c3 = " + c3);
    }
}

```

```

public class SimpleClass {

    private int integerAttr;
    private String strAttr;

```



```

public SimpleClass() {
    this(-888, "**default**");
}

public SimpleClass(int n) {
    this(n, "**default**");
}

public SimpleClass(String str) {
    this(-999, str);
}

public SimpleClass(int n, String str) {
    this.integerAttr = n;
    this.strAttr = str;
    System.out.println("instance of SimpleClass has been
created");
}

public String toString() {
    String value = Integer.toString(this.integerAttr) + "
@ "+ this.strAttr;
    return value;
}
}

```

```

import java.io.Serializable;

```

```

public class SimpleSerializableClass implements Serializable
{
    private int integerAttr;
    private String strAttr;

    public SimpleSerializableClass() {
        this(-888, "**default**");
    }

    public SimpleSerializableClass(int n) {
        this(n, "**default**");
    }

    public SimpleSerializableClass(String str) {
        this(-999, str);
    }
}

```

```

    public SimpleSerializableClass(int n, String str) {
        this.integerAttr = n;
        this.strAttr = str;
        System.out.println("instance of
SimpleSerializableClass has been created");
    }

    public String toString() {
        String value = "#" +
Integer.toString(this.integerAttr) + " @ " + this.strAttr + "#
";
        return value;
    }
}

```

Out:

```

instance of SimpleSerializableClass has been created
instance of ComplexSerializableClass has been created : c1
Serializing header message and instance of c1 = :c1(#6 @
QKR2JS1F# )

```

Retreiving the Serialized Object (i.e. header message + c1 instance) from the file system

Storage Example

```
c2 = :c1(#6 @ QKR2JS1F# )
```

Passing the Serialized Object (i.e. header message + c1 instance) throughout another stream

Storage Example

```
c3 = :c1(#6 @ QKR2JS1F# )
```

SimpleClass (non serializzabile)

È una classe semplice, con due attributi (int e String), **ma non implementa**

Serializable. Quindi **non può essere salvata** tramite ObjectOutputStream. Serve solo per mostrare cosa accadrebbe se provassimo a salvare un oggetto non serializzabile.

SimpleSerializableClass (serializzabile) è la versione “sicura” di SimpleClass, ma con: implements Serializable.

ComplexSerializableClass è la classe padre della dimostrazione. Contiene

`private Vector<SimpleSerializableClass> ssc`, cioè un vettore di oggetti serializzabili.

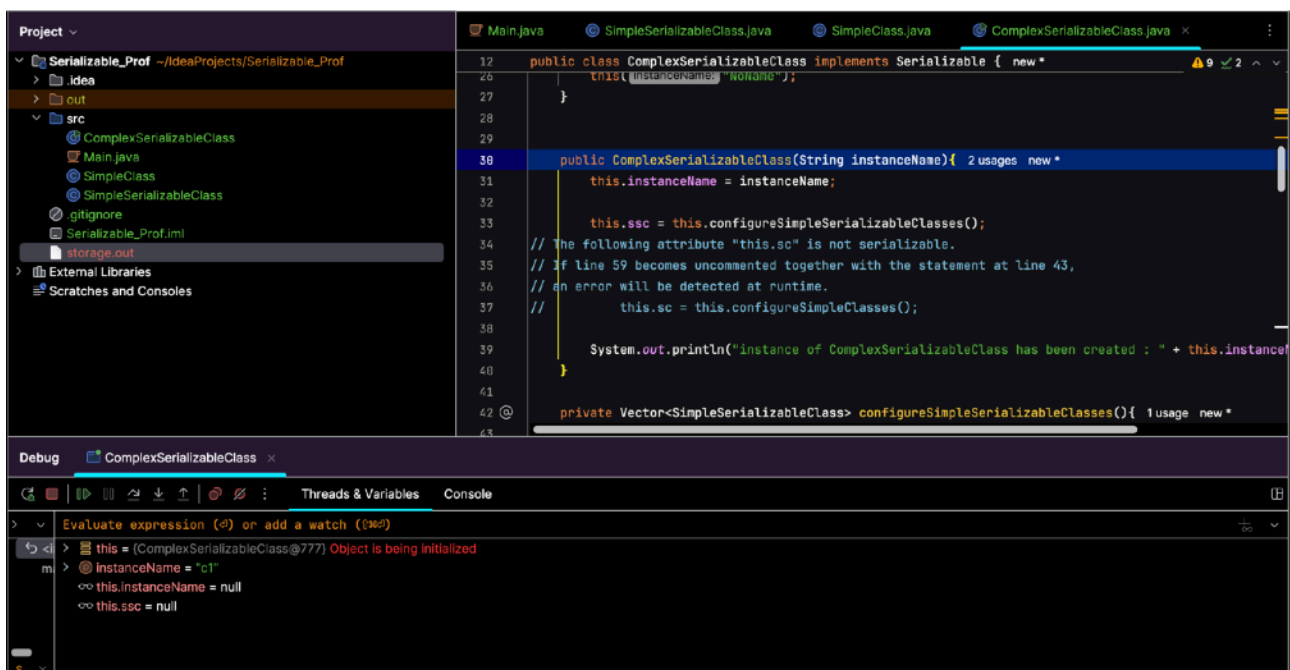
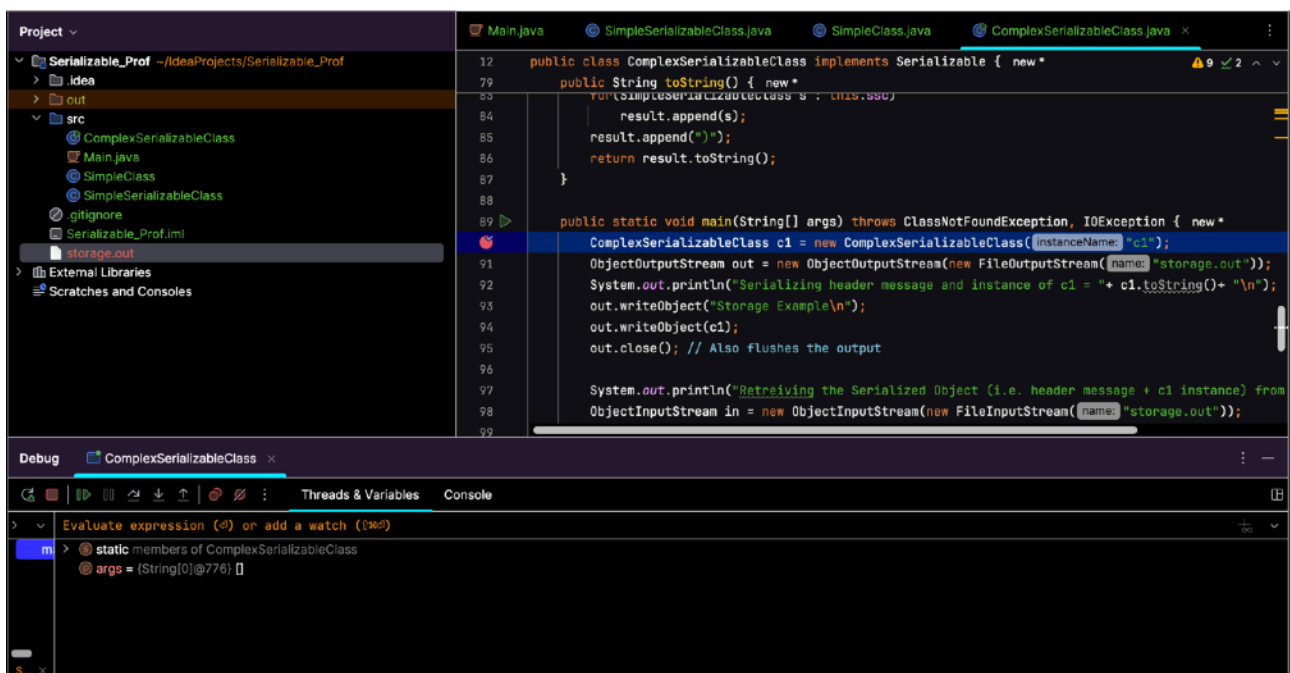
(Un Vector è anch'esso serializzabile, quindi nessun problema qui).

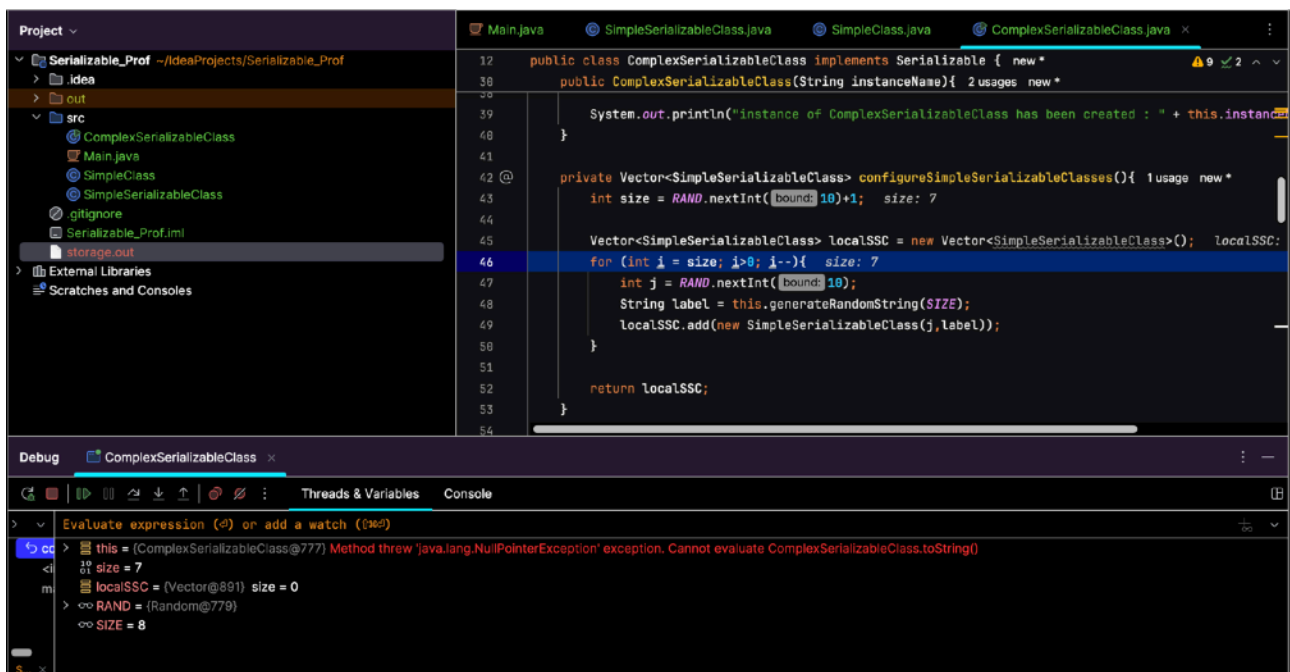
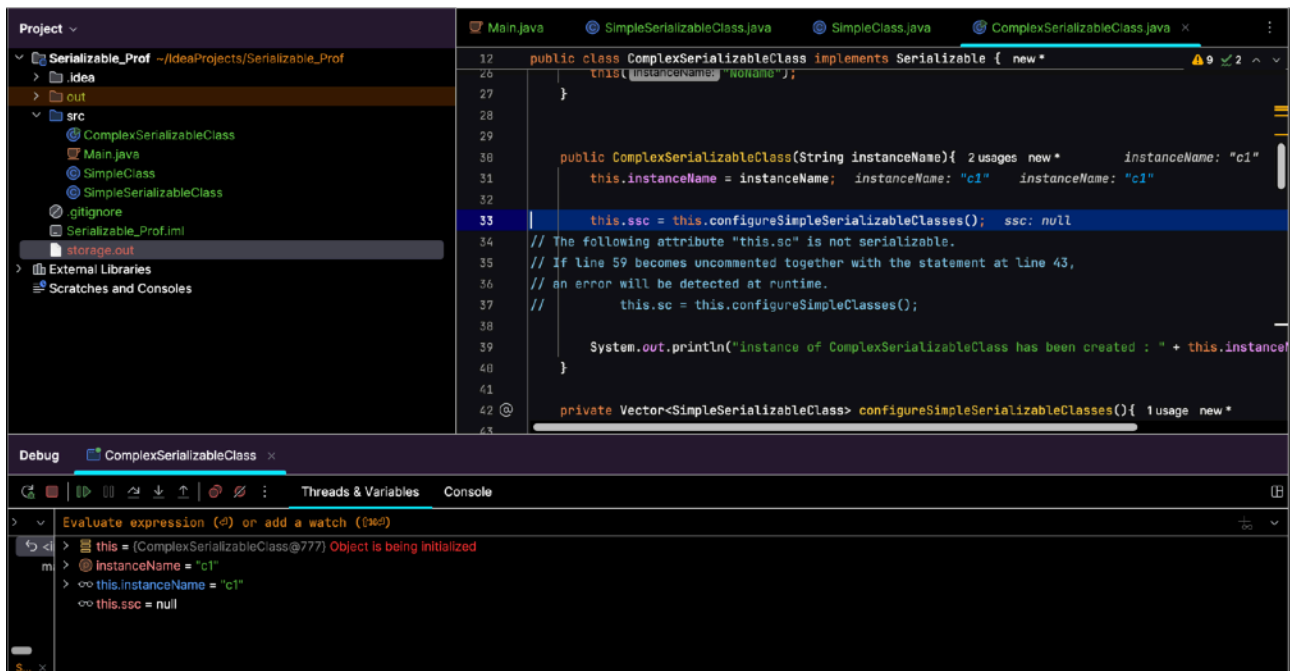
Ha anche un campo commentato:

```
// private Vector<SimpleClass> sc;
```

Se lo decommenti e provi a salvare l'oggetto, otterrai un errore a runtime: `java.io.NotSerializableException: SimpleClass`, perché `SimpleClass` non è serializzabile.

Pronti? Si parte con il debug passo passo.





Questo metodo è **privato**: viene usato solo all'interno della classe `ComplexSerializableClass`.

Restituisce un `Vector<SimpleSerializableClass>`, cioè una collezione di oggetti serializzabili.

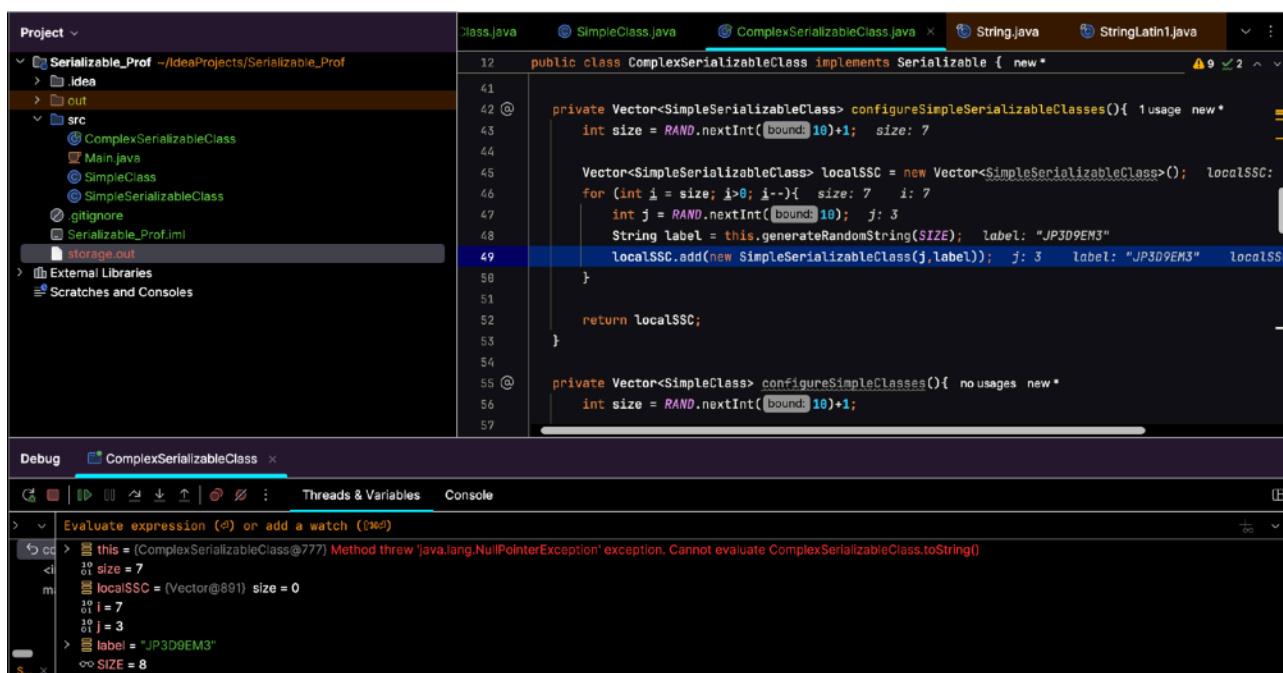
Viene chiamato dal costruttore, quando il prof crea un nuovo oggetto “complesso”. `RAND` è un oggetto `Random` creato come attributo statico della classe. `nextInt(10)` genera un numero casuale da 0 a 9. Quindi `+1` serve a evitare lo 0.

Dopodiché crea un **nuovo vettore vuoto** che conterrà tutti gli oggetti `SimpleSerializableClass` che stiamo per generare.

Il ciclo parte da size e scende fino a 1. Quindi verranno create size istanze.

Ogni ciclo genera un numero intero casuale j (da 0 a 9) e genera una stringa casuale (label) lunga SIZE caratteri (in questo caso 8), usando il metodo `generateRandomString()`.

Dopodiché si crea un nuovo oggetto `SimpleSerializableClass` passando quei due valori: `new SimpleSerializableClass(j, label)` e lo aggiunge al vettore: `localSSC.add(...)`.



Allo stato attuale è stato creato un oggetto `SimpleSerializableClass(3, JP3D9EM3)`.

Finito il ciclo sono state create 7 istanze di questo tipo:

```
localSSC = {Vector@891} size = 7
> 0 = {SimpleSerializableClass@944} "#3 @ JP3D9EM3# "
> 1 = {SimpleSerializableClass@945} "#5 @ 39KS6DO9# "
> 2 = {SimpleSerializableClass@946} "#4 @ AQ69ZMRC# "
> 3 = {SimpleSerializableClass@947} "#6 @ 3DDP2NKR# "
> 4 = {SimpleSerializableClass@1008} "#9 @ 5CWORS8A# "
> 5 = {SimpleSerializableClass@1026} "#0 @ INOG4YI0# "
> 6 = {SimpleSerializableClass@1044} "#2 @ YZYVCSL5# "
```

E sono collezionate dentro **localSSC**.

Nel frattempo, a video vediamo le seguenti scritte:

instance of SimpleSerializableClass has been created

instance of SimpleSerializableClass has been created

instance of SimpleSerializableClass has been created

instance of SimpleSerializableClass has been created

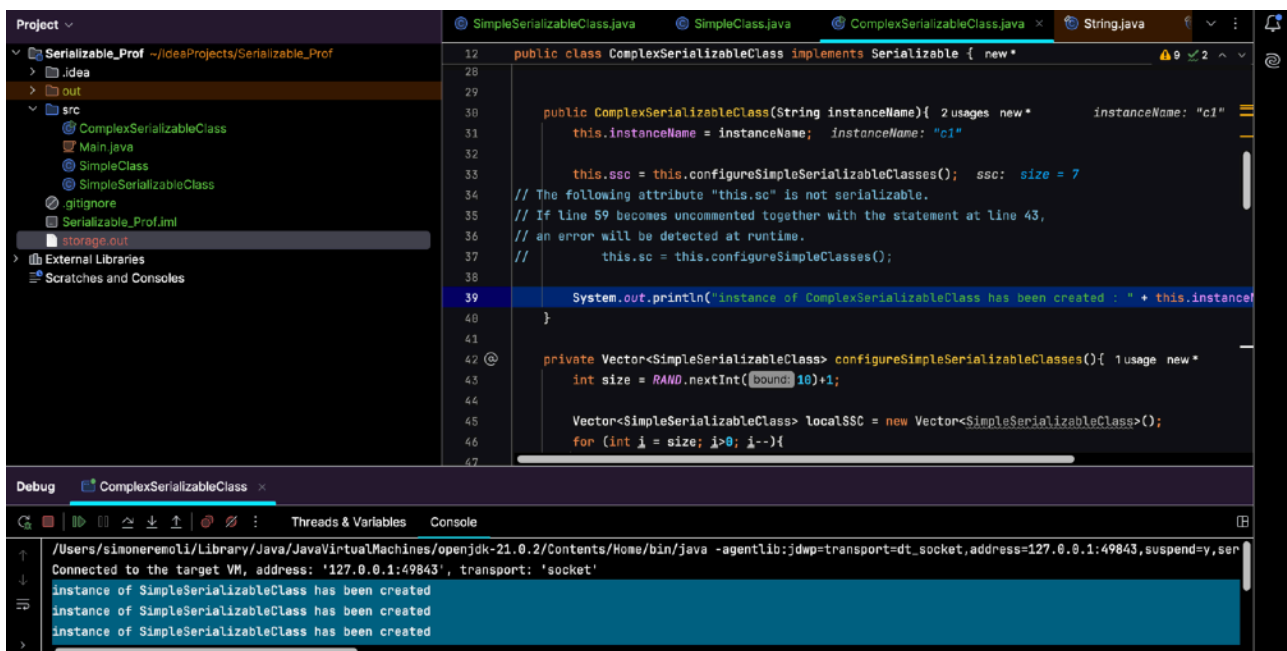
instance of SimpleSerializableClass has been created

instance of SimpleSerializableClass has been created

instance of SimpleSerializableClass has been created

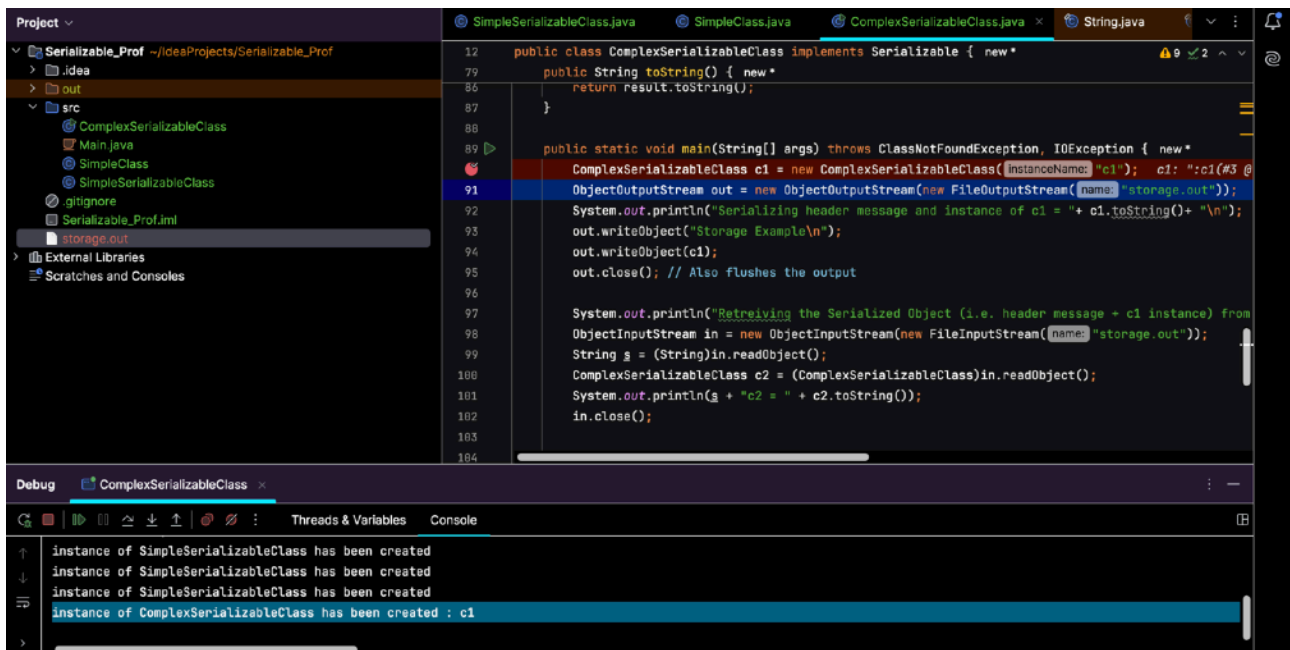
```
public SimpleSerializableClass(int n, String str) { 3 usages new *
    this.integerAttr = n;
    this.strAttr = str;
    System.out.println("instance of SimpleSerializableClass has been created");
}
```

Perché ogni volta che chiamiamo il costruttore succede questo.



Out: instance of ComplexSerializableClass has been created : c1

A questo punto ci prepariamo per la serializzazione:

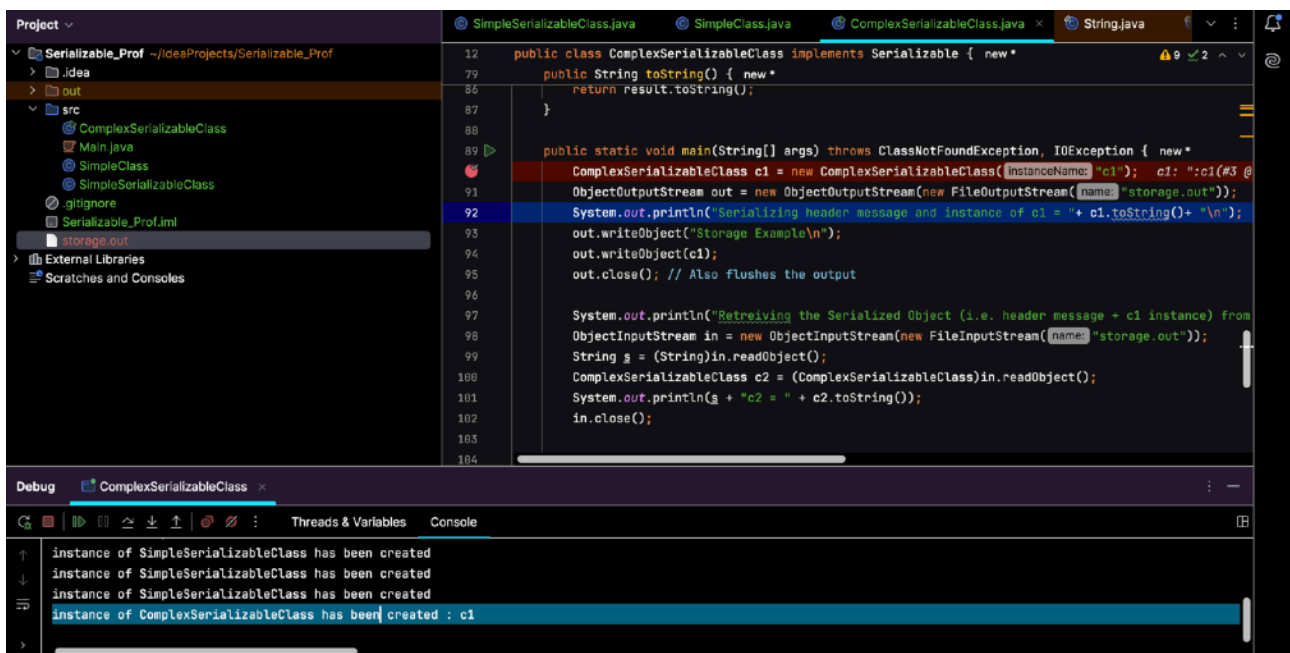


The screenshot shows an IDE with a project named 'Serializable_Prof'. The source code for 'ComplexSerializableClass.java' is displayed. The class implements the 'Serializable' interface and has a 'toString()' method. The 'main' method creates an instance of 'ComplexSerializableClass' with the instance name 'c1'. It then creates an 'ObjectOutputStream' and writes the object to a file named 'storage.out'. The console output shows the creation of instances of 'SimpleSerializableClass' and 'ComplexSerializableClass'.

```
12 public class ComplexSerializableClass implements Serializable { new *
79 public String toString() { new *
86 return result.toString();
87 }
88
89 public static void main(String[] args) throws ClassNotFoundException, IOException { new *
91 ComplexSerializableClass c1 = new ComplexSerializableClass(instanceName: "c1"); c1: ":c1(#3 @
92 ObjectOutputStream out = new ObjectOutputStream(new FileOutputStream(name: "storage.out"));
93 System.out.println("Serializing header message and instance of c1 = " + c1.toString() + "\n");
94 out.writeObject("Storage Example\n");
95 out.writeObject(c1);
96 out.close(); // Also flushes the output
97
98 System.out.println("Retreiving the Serialized Object (i.e. header message + c1 instance) from
99 ObjectInputStream in = new ObjectInputStream(new FileInputStream(name: "storage.out"));
100 String s = (String)in.readObject();
101 ComplexSerializableClass c2 = (ComplexSerializableClass)in.readObject();
102 System.out.println(s + "c2 = " + c2.toString());
103 in.close();
104 }
```

Debug Console:

```
instance of SimpleSerializableClass has been created
instance of SimpleSerializableClass has been created
instance of SimpleSerializableClass has been created
instance of ComplexSerializableClass has been created : c1
```



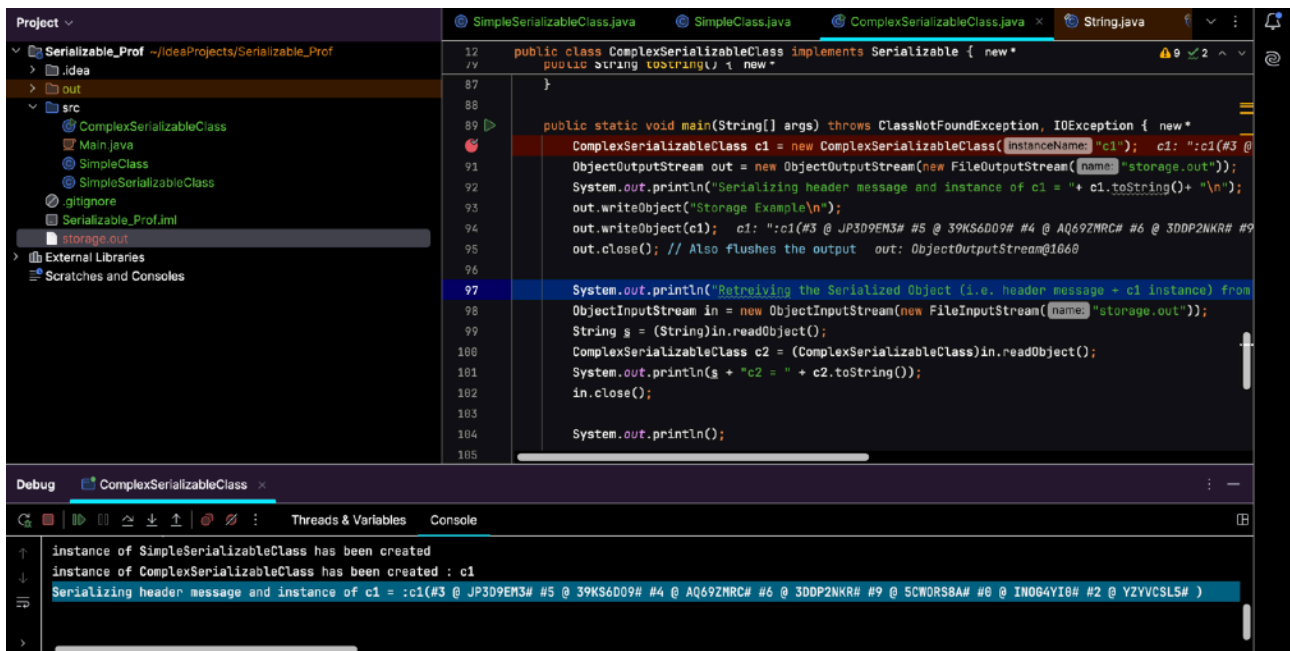
The screenshot shows the same IDE with the 'ComplexSerializableClass.java' file. The 'main' method is modified to read the object from the 'storage.out' file using an 'ObjectInputStream'. The console output shows the creation of instances of 'SimpleSerializableClass' and 'ComplexSerializableClass'.

```
12 public class ComplexSerializableClass implements Serializable { new *
79 public String toString() { new *
86 return result.toString();
87 }
88
89 public static void main(String[] args) throws ClassNotFoundException, IOException { new *
91 ComplexSerializableClass c1 = new ComplexSerializableClass(instanceName: "c1"); c1: ":c1(#3 @
92 ObjectOutputStream out = new ObjectOutputStream(new FileOutputStream(name: "storage.out"));
93 System.out.println("Serializing header message and instance of c1 = " + c1.toString() + "\n");
94 out.writeObject("Storage Example\n");
95 out.writeObject(c1);
96 out.close(); // Also flushes the output
97
98 System.out.println("Retreiving the Serialized Object (i.e. header message + c1 instance) from
99 ObjectInputStream in = new ObjectInputStream(new FileInputStream(name: "storage.out"));
100 String s = (String)in.readObject();
101 ComplexSerializableClass c2 = (ComplexSerializableClass)in.readObject();
102 System.out.println(s + "c2 = " + c2.toString());
103 in.close();
104 }
```

Debug Console:

```
instance of SimpleSerializableClass has been created
instance of SimpleSerializableClass has been created
instance of SimpleSerializableClass has been created
instance of ComplexSerializableClass has been created : c1
```

Out: Serializing header message and instance of c1 = :c1(#3 @ JP3D9EM3# #5 @ 39KS6DO9# #4 @ AQ69ZMRC# #6 @ 3DDP2NKR# #9 @ 5CWORS8A# #0 @ INOG4YI0# #2 @ ZYVCSL5#)

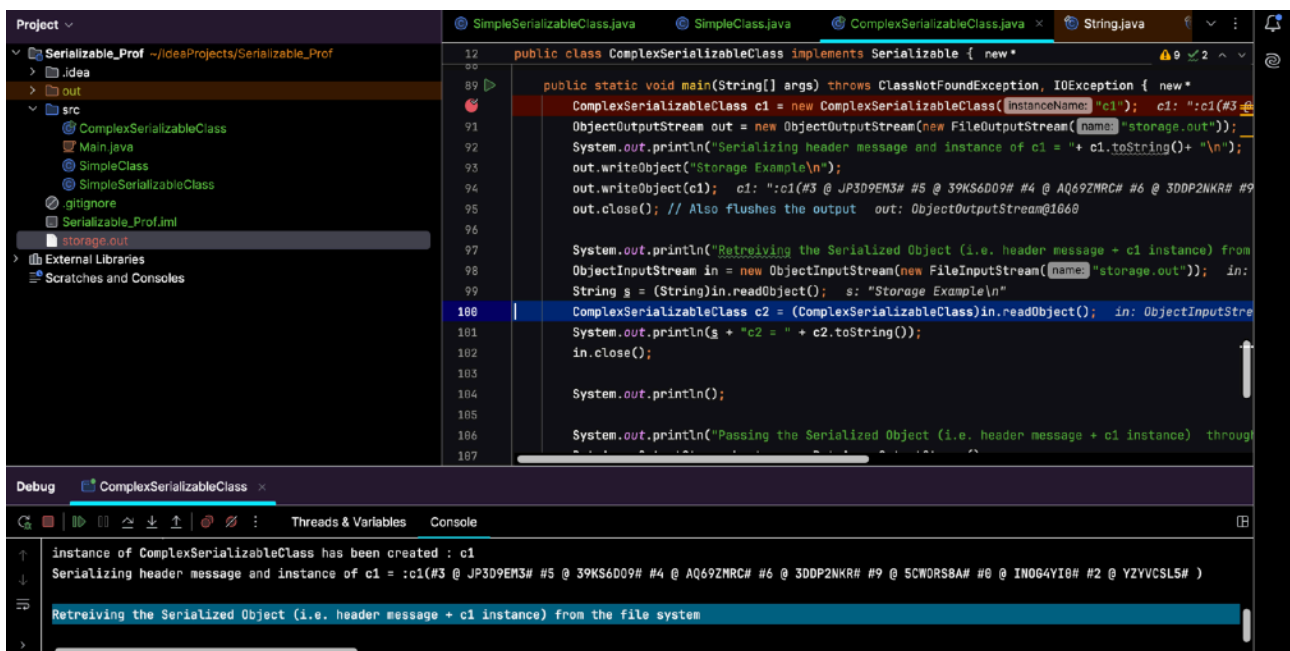


L'oggetto è stato scritto nel file storage.out.

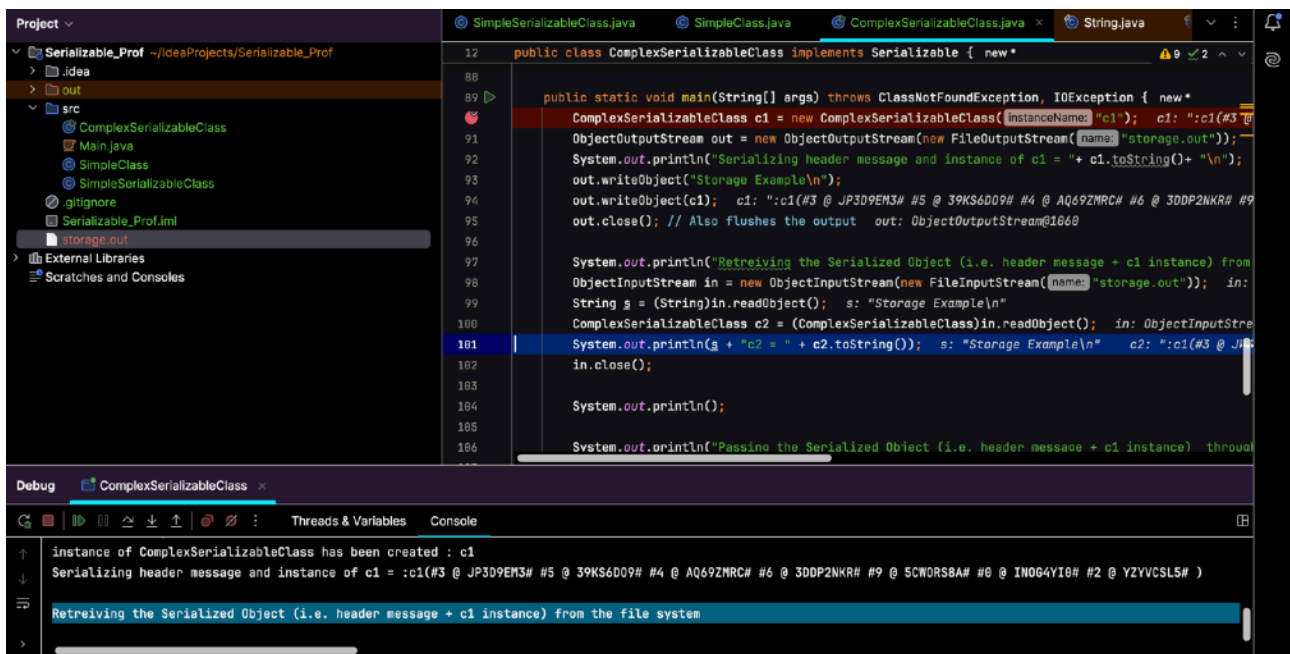
Ora dobbiamo deserializzare.

Out: Retreiving the Serialized Object (i.e. header message + c1 instance) from the file system.

Nota bene: prima ha scritto una stringa e poi l'oggetto, ora preleva prima la stringa e poi l'oggetto (come giusto che sia).



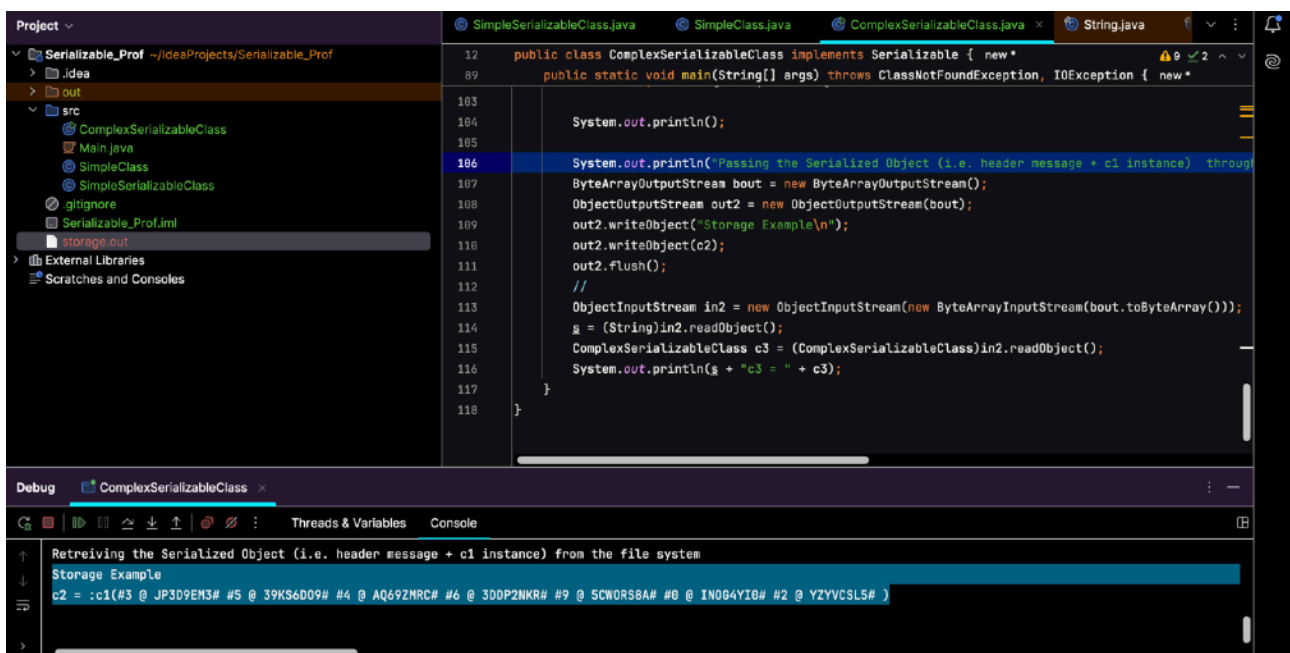
Attenzione, stiamo per vedere il risultato più importante, la stampa della deserializzazione.



Giustamente viene stampato quello che prima aveva serializzato:

Storage Example

c2 = :c1(#3 @ JP3D9EM3# #5 @ 39KS6D09# #4 @ AQ69ZMRC# #6 @ 3DDP2NKR# #9 @ 5CWORS8A# #0 @ INOG4YI0# #2 @ YZYVCSL5#)



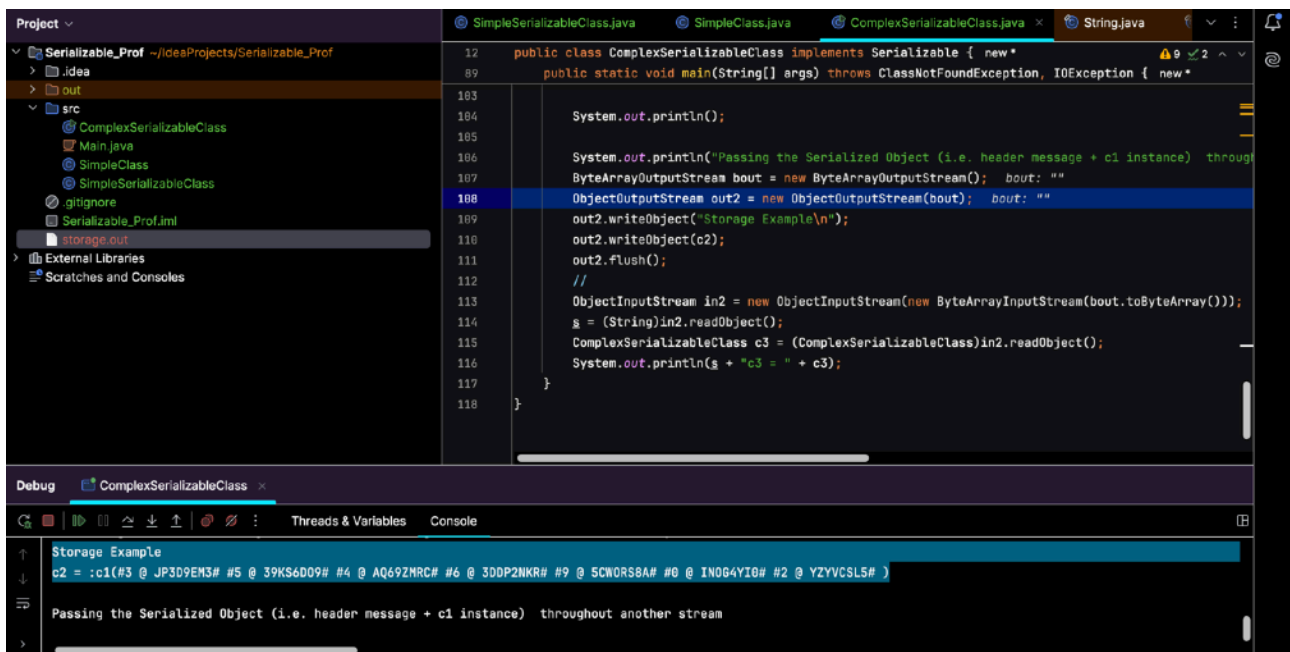
ATTENZIONE: Il Prof è stato molto bravo a farci capire certi concetti.

Ora il prossimo passaggio serve a dimostrare che la serializzazione può avvenire anche interamente in memoria, non solo su file, e che l'oggetto serializzato può viaggiare in un flusso qualunque di byte (come una rete o una pipe tra processi). In pratica, il professore ci sta facendo vedere che la serializzazione è indipendente dal mezzo fisico.

Nella prima parte del programma, il prof ha serializzato su file fisico.

Ora il prof replica la stessa operazione, ma senza passare dal file system. Usa un **ByteArrayOutputStream**, cioè una memoria temporanea (RAM). Collega il flusso degli oggetti (**ObjectOutputStream**) al buffer di memoria (**bout**). Così, quando scrivi un oggetto, esso non va su disco ma **viene serializzato direttamente nel buffer RAM**.

Proseguiamo.



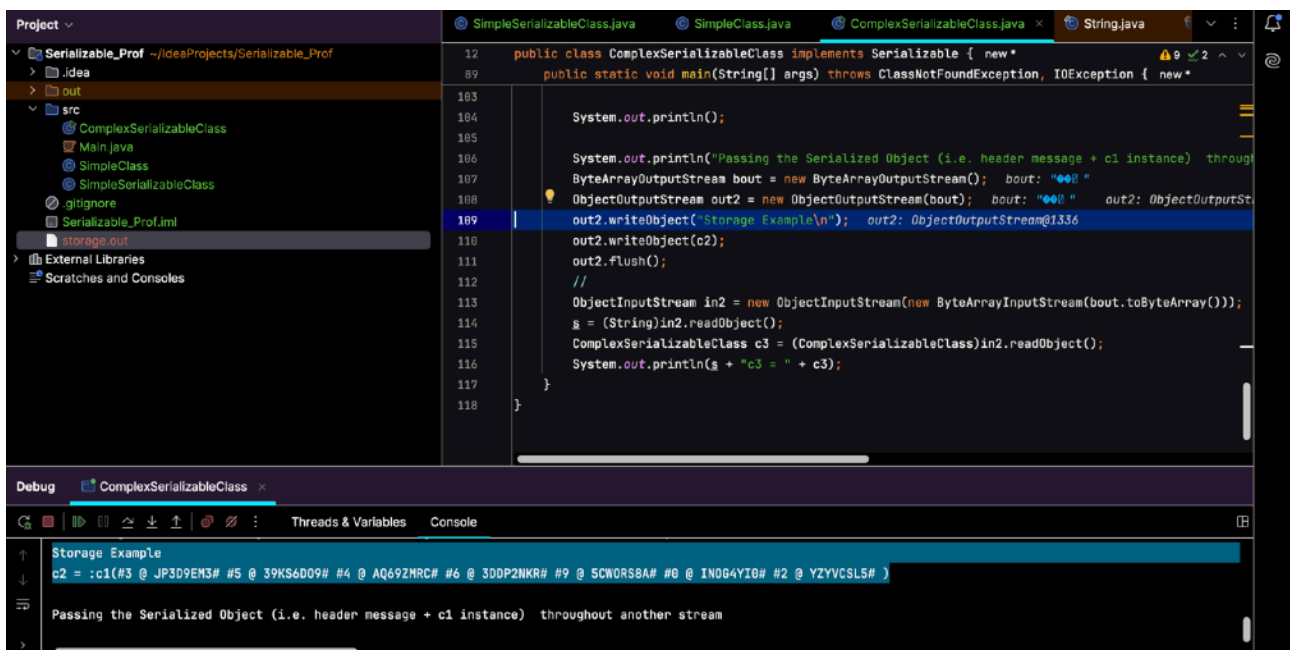
```
Project ▾
  Serializable_Prof ~/ideaProjects/Serializable_Prof
    > .idea
    > > out
    > > src
      > ComplexSerializableClass
      > Main.java
      > SimpleClass
      > SimpleSerializableClass
      > .gitignore
      > Serializable_Prof.iml
      > storage.out
    > External Libraries
    > Scratches and Consoles

12 public class ComplexSerializableClass implements Serializable { new *
89 public static void main(String[] args) throws ClassNotFoundException, IOException { new *
183
184     System.out.println();
185
186     System.out.println("Passing the Serialized Object (i.e. header message + c1 instance) through
187     ByteArrayOutputStream bout = new ByteArrayOutputStream(); bout: ""
188     ObjectOutputStream out2 = new ObjectOutputStream(bout); bout: ""
189     out2.writeObject("Storage Example\n");
190     out2.writeObject(c2);
191     out2.flush();
192     //
193     ObjectInputStream in2 = new ObjectInputStream(new ByteArrayInputStream(bout.toByteArray()));
194     s = (String)in2.readObject();
195     ComplexSerializableClass c3 = (ComplexSerializableClass)in2.readObject();
196     System.out.println(s + "c3 = " + c3);
197 }
198 }
```

Debug ComplexSerializableClass x

Storage Example
c2 = :c1(#3 @ JP3D9EN3# #5 @ 39KS6D09# #4 @ AQ69ZMRC# #6 @ 3DDP2MKR# #9 @ 5CWORS8A# #0 @ IN064YI0# #2 @ YZVCSL5#)

Passing the Serialized Object (i.e. header message + c1 instance) throughout another stream



```
Project ▾
  Serializable_Prof ~/ideaProjects/Serializable_Prof
    > .idea
    > > out
    > > src
      > ComplexSerializableClass
      > Main.java
      > SimpleClass
      > SimpleSerializableClass
      > .gitignore
      > Serializable_Prof.iml
      > storage.out
    > External Libraries
    > Scratches and Consoles

12 public class ComplexSerializableClass implements Serializable { new *
89 public static void main(String[] args) throws ClassNotFoundException, IOException { new *
183
184     System.out.println();
185
186     System.out.println("Passing the Serialized Object (i.e. header message + c1 instance) through
187     ByteArrayOutputStream bout = new ByteArrayOutputStream(); bout: ""
188     ObjectOutputStream out2 = new ObjectOutputStream(bout); bout: "" out2: ObjectOutputStream
189     out2.writeObject("Storage Example\n"); out2: ObjectOutputStream@1336
190     out2.writeObject(c2);
191     out2.flush();
192     //
193     ObjectInputStream in2 = new ObjectInputStream(new ByteArrayInputStream(bout.toByteArray()));
194     s = (String)in2.readObject();
195     ComplexSerializableClass c3 = (ComplexSerializableClass)in2.readObject();
196     System.out.println(s + "c3 = " + c3);
197 }
198 }
```

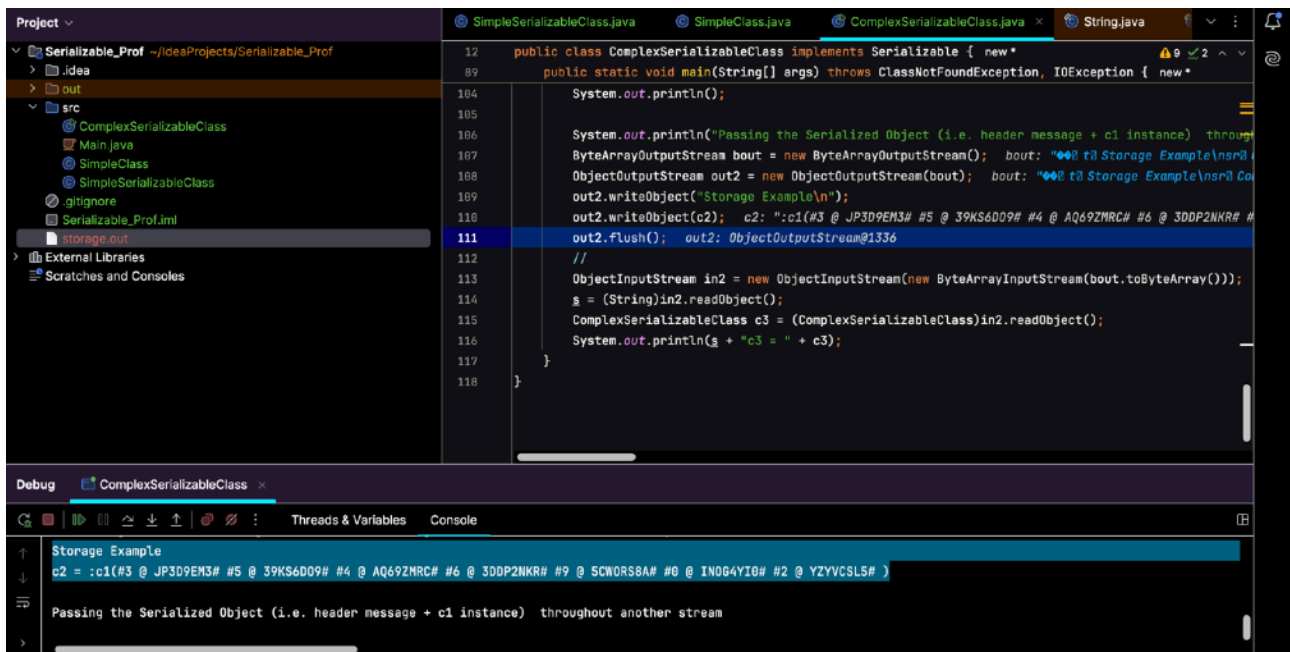
Debug ComplexSerializableClass x

Storage Example
c2 = :c1(#3 @ JP3D9EN3# #5 @ 39KS6D09# #4 @ AQ69ZMRC# #6 @ 3DDP2MKR# #9 @ 5CWORS8A# #0 @ IN064YI0# #2 @ YZVCSL5#)

Passing the Serialized Object (i.e. header message + c1 instance) throughout another stream

Scrivi una stringa dentro al buffer.

Serve come "intestazione" o "header message", esattamente come nel primo esperimento.



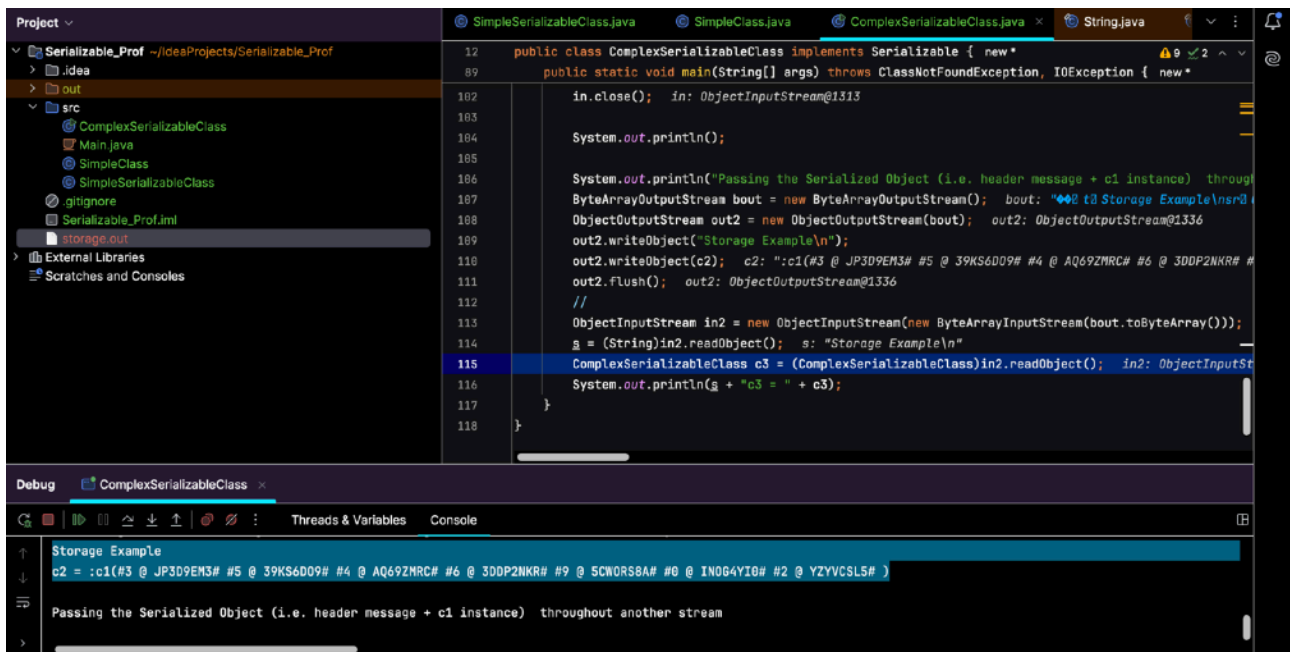
Serializza l'oggetto c2 (una copia di c1) nel buffer.

Ora nel ByteArrayOutputStream ci sono due oggetti serializzati:

1. La stringa "Storage Example\n",
2. L'oggetto ComplexSerializableClass (c2).

out2.flush(): forza l'invio di tutti i byte accumulati nel buffer interno di ObjectOutputStream verso il ByteArrayOutputStream.

Non scrive su disco, ma “chiude” la scrittura in RAM, rendendo i dati pronti per essere letti.



Vado in deserializzazione, nulla da dire.

Out c3:

Storage Example

c3 = :c1(#3 @ JP3D9EM3# #5 @ 39KS6DO9# #4 @ AQ69ZMRC# #6 @ 3DDP2NKR# #9 @ 5CWORS8A# #0 @ INOG4YI0# #2 @ YZYVCSL5#)

Process finished with exit code 0 - fine della tortura.

Lo stream è il vero protagonista. Finora il prof ha usato:

- FileOutputStream: scrittura su disco.
- ByteArrayOutputStream: scrittura in memoria (RAM).

Si potrebbe aggiungere un terzo caso, di cui non parlerò, ma che è molto bello da spiegare nel corso di IIW del Prof.Lo Presti, ossia `SocketOutputStream`: scrittura su rete.

In tutti e tre i casi, il protagonista non cambia: L'`ObjectOutputStream` non si cura *dove* vadano i byte, gli basta che lo stream sottostante li sappia ricevere.

FINE PRIMA PARTE

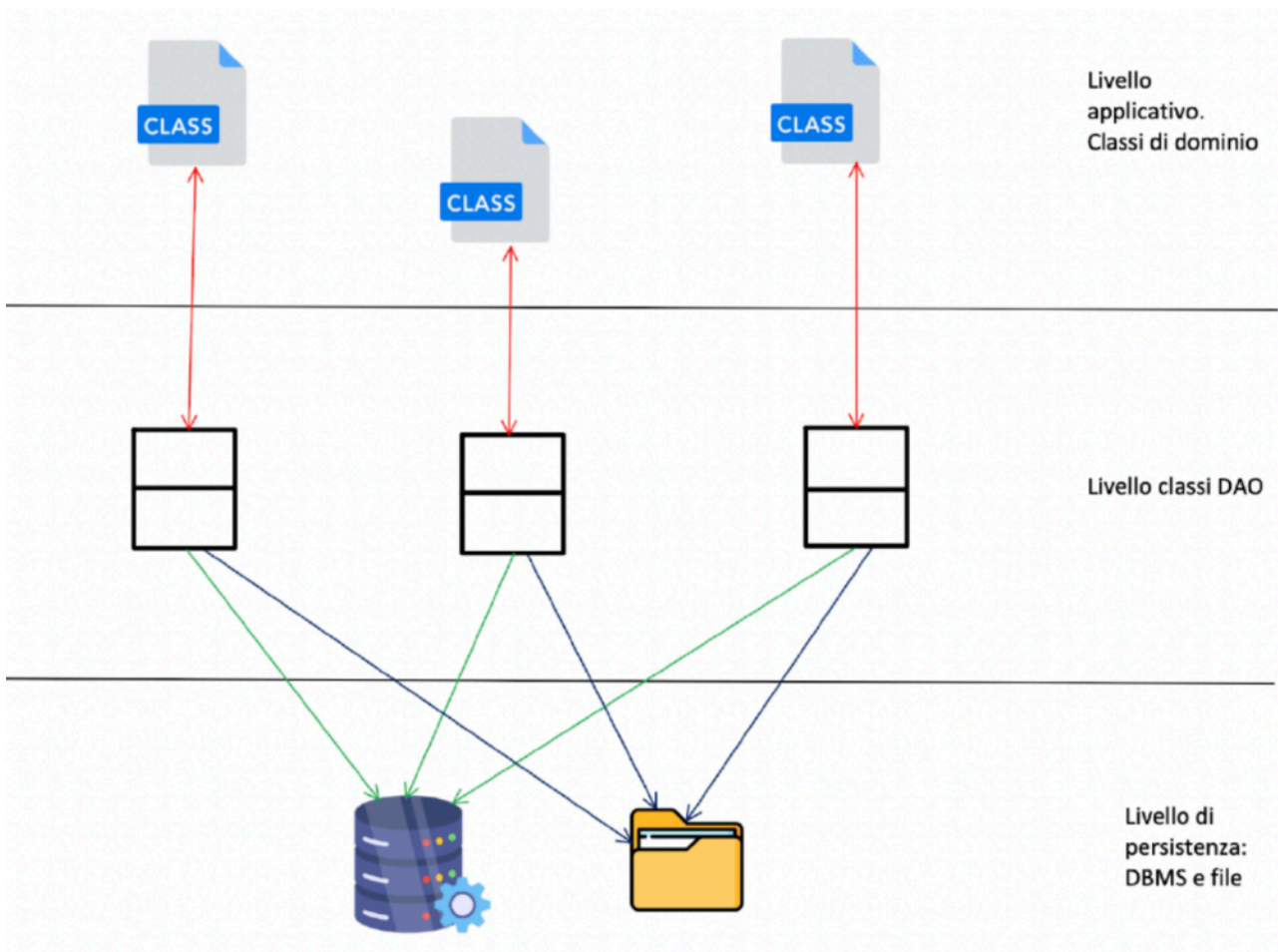
CORSO DI BASI DI DATI - RICHIAMI E INTEGRAZIONE CON DE ANGELIS.

Chi ha superato basi di dati può anche non leggere la seguente trattazione.

Il corso di **Basi di Dati**, in particolare la sezione tenuta dal **professor Alessandro Pellegrini**, riveste un'importanza fondamentale per comprendere la logica che connette il mondo della **persistenza dei dati** con quello **applicativo**.

Durante le lezioni, il professor Pellegrini sottolinea come, per **interagire correttamente con il database** sia per il prelievo sia per la scrittura dei dati sia necessario utilizzare **classi dedicate**, specificamente progettate per tale scopo.

Tali classi, denominate **DAO (Data Access Object)**, costituiscono il livello di accesso ai dati e non devono essere confuse con le classi del **model** o del **controller**, poiché svolgono un ruolo distinto e ben definito nell'architettura applicativa.



Qui i miei appunti sulla DAO nel corso di Basi di Dati.

Una **DAO (Data Access Object)** è una classe che si occupa esclusivamente della comunicazione tra l'applicazione e il database.

Il suo compito è **accedere ai dati, leggerli, scriverli, modificarli o cancellarli**, mantenendo separata la logica di accesso ai dati dalla logica applicativa.

In questo modo, il codice risulta più **organizzato, riutilizzabile e manutenibile**, poiché eventuali modifiche al database non richiedono interventi sulle altre parti dell'applicazione.

Esempio di una semplicissima classe DAO:

```
package it.uniroma2.dicii.bd.Model.DAO;
```

```
import it.uniroma2.dicii.bd.Exception.DAOException;
```

```
import java.sql.CallableStatement;
```

```
import java.sql.Connection;
```

```
import java.sql.ResultSet;
```

```
import java.sql.SQLException;
```

```
public class ReportCompletoDAO {
```

```
    public void Execute(Object... params) throws DAOException {
```

```
        try {
```

```
            int i = 1;
```

```
            Connection conn = ConnectionFactory.getConnection();
```

```
            CallableStatement cs = conn.prepareCall("{call  
report_completo()}");
```

```
            ResultSet rs = cs.executeQuery();
```

```
System.out.println("\t-----  
-----");
```

```
        while(rs.next())
```

```
        {
```

```
            System.out.println("\t " + i++ + ") " +
```

```
"Titolo:" + rs.getString(1) + "\t ISBN:" + rs.getString(2) +
```

```
            "\t Codice Scaffale:" + rs.getInt(3) + "\t Nome
```

```
Scaffale:" + rs.getString(4) + "\t Ripiano:" + rs.getInt(5) + "\n");
```

```
        }
```

```
System.out.println("\t-----  
-----");
```

```
    } catch (SQLException e) {
```

```
        throw new RuntimeException(e);
```

```
    }
```

```
}
```

```
}
```


CallableStatement serve per invocare una stored procedure nel database.

Una stored procedure è un blocco di codice SQL precompilato e salvato all'interno del database. Può contenere istruzioni SQL (come **SELECT**, **INSERT**, **UPDATE**, **DELETE**), ma anche **strutture di controllo** (**IF**, **WHILE**, **CASE**, ecc.), variabili, cursori e parametri in ingresso o in uscita.

Esempio di stored procedure:

```
CREATE PROCEDURE report_completo()

BEGIN

    SELECT

        titolo,

        isbn,

        codice_scaffale,

        nome_scaffale,

        ripiano

    FROM libro

    JOIN scaffale ON libro.id_scaffale = scaffale.id;

END
```

La stored procedure legge dal database titolo, isbn, codice_scaffale, nome_scaffale e ripiano, e li “consegna” alla DAO.

Innanzitutto per incominciare ad utilizzare le DAO bisogna importare JDBC nel progetto. Quindi, a seconda dell'IDE che avete, scaricate JDBC per la connettività:

<https://www.mysql.com/products/connector/>

Dopodiché importatelo sul vostro IDE, su IntelliJ andate su File->Project Structure->Libraries e trovate il .jar.

La libreria che si adotta è **java.sql.***!

Un controller chiama la DAO, non un MODEL!

Brevissimo* RESULT SET.

Quando esegui una query con JDBC, ottieni un `ResultSet` come risultato:

```
Statement stmt = conn.createStatement();  
ResultSet rs = stmt.executeQuery("SELECT * FROM utenti");
```

In questo esempio, `rs` contiene **tutti i record** della tabella utenti.

Un `ResultSet` è un **cursore**: si muove riga per riga tramite il metodo `next()`.

```
while (rs.next()) {  
    int id = rs.getInt("id");  
    String nome = rs.getString("nome");  
    String email = rs.getString("email");  
    System.out.println(id + " - " + nome + " - " + email);  
}
```

`rs.next()`: sposta il cursore alla riga successiva e restituisce `true` se esiste.

`getInt("id")`, `getString("nome")`, ecc. : recuperano i valori della riga corrente, in questo caso sto facendo l'accesso per nome dei campi.

È fondamentale **chiudere** le risorse JDBC per evitare perdite di memoria:

```
rs.close();  
stmt.close();  
conn.close();
```

Riagganciamoci a De Angelis.

Ad ogni classe entità dovrebbe corrispondere un DAO che ne gestisce il salvataggio ed il recupero delle istanze su layer di persistenza.

Di seguito il codice del De Angelis e poi abbiamo finito.

```
import dao.AlbumDAO;
import dao.AlbumDAOCSV;
import dao.AlbumDAOJDBC;
import entities.Album;

import java.io.IOException;
import java.sql.SQLException;
import java.util.List;
import exceptions.DuplicatedRecordException;

public class SimpleJDBCExample {

    private AlbumDAO albumDAO = null;

    public SimpleJDBCExample(TypesOfPersistenceLayer p) throws
IOException {
        switch (p) {
            case JDBC:
                this.albumDAO = new AlbumDAOJDBC();
                break;
            case FileSystem:
                this.albumDAO = new AlbumDAOCSV();
                break;
            default:
                this.albumDAO = new AlbumDAOCSV();
                break;
        }
    }

    public void runTheExample() {
        Album newAlbumInstance = new Album(55, "Pearl Jam", "Ten", 1991);

        try {
            String searchKey = "Made In Japan";
            System.out.println("Looking for albums named: " + searchKey);
            List<Album> list =
this.albumDAO.retrieveByAlbumName(searchKey);

            int i=0;
            for (Album album : list) {
                i++;
            }
        }
    }
}
```

```

        System.out.println("Result " + i + ": "
+album.toString());
    }

    try {
        System.out.println("*****");
        System.out.println("Adding the new album instance named:
" + newAlbumInstance.getTitolo());
        this.albumDAO.saveAlbum(newAlbumInstance);
        System.out.println("... done!");
        // Sperimentate l'uso di eccezioni custom. Rimuovete il commento
dall'istruzione che segue, ed eseguite il codice in modalità debug.
        this.albumDAO.saveAlbum(newAlbumInstance);
    } catch (DuplicatedRecordException dre) {
        System.out.println("... ops! Album already present.\n
Removing the old item (i.e. ID: "+newAlbumInstance.getAlbumId()+")...");
        this.albumDAO.removeAlbumById(newAlbumInstance);
        System.out.println("... removal completed!");
        Album updatedAlbumInstance = new Album(56,
newAlbumInstance.getArtista(), newAlbumInstance.getTitolo(),
newAlbumInstance.getAnno());
        System.out.println("Replacing with an updated item (i.e.
ID: "+updatedAlbumInstance.getAlbumId()+")...");
        this.albumDAO.saveAlbum(updatedAlbumInstance);
        System.out.println("... replacement completed!");
        newAlbumInstance = updatedAlbumInstance;
    }

    System.out.println("*****");
    System.out.println("Removing the item just added ...");
    albumDAO.removeAlbumById(newAlbumInstance);
    System.out.println("... done!");
    System.out.println("*****");
} catch (SQLException se) {
    // Errore durante l'apertura della connessione
    se.printStackTrace();
} catch (ClassNotFoundException driverEx) {
    // Errore nel loading del driver
    driverEx.printStackTrace();
} catch (Exception e) {
    // Errore nel loading del driver o possibilmente nell'accesso
al filesystem
    e.printStackTrace();
}
}

    public static void main(String args[]) throws IOException {

// Alternate il tipo dello strato di persistenza di riferimento.
// Notate che l'implementazione dell'applicazione non cambia.
// La responsabilità dell'interfacciamento verso lo strato di persistenza
è tutta delegata ai DAO
        SimpleJDBCExample toyExample = new
SimpleJDBCExample(TypesOfPersistenceLayer.JDBC);
//         SimpleJDBCExample toyExample = new
SimpleJDBCExample(TypesOfPersistenceLayer.FileSystem);

```

```

        toyExample.runTheExample();
    }
}

enum TypesOfPersistenceLayer{
    JDBC,
    FileSystem
}

```

```

package exceptions;
public class DuplicatedRecordException extends Exception {

    private static final long serialVersionUID = 1L;

    public DuplicatedRecordException(String message) {
        super(message);
    }

}

```

```

package entities;

```

```

public class Album {
    private String artista;
    private String titolo;
    private int anno;
    private int albumId;

    public Album (int albumId){
        this.albumId =albumId;
    }

    public Album (int albumId, String artista, String titolo,
int anno){
        this.albumId = albumId;
        this.artista = artista;
        this.titolo = titolo;
        this.anno = anno;
    }
}

```

```

    public int getAlbumId() {
        return albumId;
    }

    public void setAnno(int anno) {
        this.anno = anno;
    }

    public int getAnno() {
        return anno;
    }

    public void setArtista(String artista) {
        this.artista = artista;
    }

    public String getArtista() {
        return artista;
    }

    public void setTitolo(String titolo) {
        this.titolo = titolo;
    }

    public String getTitolo() {
        return titolo;
    }

    @Override
    public String toString(){
        return this.artista + " \"\" + this.titolo + "\"\" (\" +
this.anno + ")";
    }

}

package entities;
public class Persona {
    private String nome;
    private String cognome;

```

```

    public Persona(String nome, String cognome) {
        this.nome = nome;
        this.cognome = cognome;
    }

    public String getNome() {
        return nome;
    }

    public void setName(String nome) {
        this.nome = nome;
    }

    public String getCognome() {
        return cognome;
    }

    public void setCognome(String cognome) {
        this.cognome = cognome;
    }
}

package dao;

import entities.Album;

import java.util.List;

public interface AlbumDAO {

    List<Album> retrieveByAlbumName(String albumName) throws
Exception;
    void saveAlbum(Album instance) throws Exception;
    void removeAlbumById(Album instance) throws Exception;
}

package dao;

import static org.junit.Assert.assertEquals;
import static org.junit.Assert.assertTrue;

import java.io.BufferedReader;

```

```

import java.io.BufferedWriter;
import java.io.File;
import java.io.FileReader;
import java.io.FileWriter;
import java.io.IOException;
import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;

import static java.nio.file.StandardCopyOption.*;

import java.nio.file.Files;

import com.opencsv.CSVReader;
import com.opencsv.CSVWriter;

import entities.*;
import exceptions.DuplicatedRecordException;

public class AlbumDAOCSV implements AlbumDAO {

    private static final String CSV_FILE_NAME =
"localDBFile.csv";

    private File fd;

    private HashMap<String, Album> localCache;

    public AlbumDAOCSV() throws IOException {
        this.fd = new File(CSV_FILE_NAME);

        if (!fd.exists()) {
            fd.createNewFile();
        }

        this.localCache = new HashMap<String, Album>();
    }

    @Override
    public List<Album> retrieveByAlbumName(String albumName)
throws Exception {

        List<Album> lst = new ArrayList<Album>();

```

```

        synchronized (this.localCache) {
            for (String id : this.localCache.keySet()) {
                Album recordInCache =
this.localCache.get(id);
                boolean recordFound =
recordInCache.getTitolo().equals(albumName);
                if (recordFound) {
                    lst.add(recordInCache);
                }
            }
        }

        if (lst.isEmpty()) {
            lst = retrieveByAlbumName(this.fd, albumName);
            synchronized (this.localCache) {
                for (Album album : lst) {

this.localCache.put(String.valueOf(album.getAlbumId()),
album);

                }
            }
        }

        return lst;
    }

    private static synchronized List<Album>
retrieveByAlbumName(File fd, String albumName) throws
Exception {
        // create csvReader object passing file reader as a
parameter
        CSVReader csvReader = new CSVReader(new
BufferedReader(new FileReader(fd)));
        String[] record;

        List<Album> albumList = new ArrayList<Album>();

        while ((record = csvReader.readNext()) != null) {
            int postTitolo =
AlbumAttributesOrder.getIndex_Titolo();

            boolean recordFound =
record[postTitolo].equals(albumName);
            if (recordFound) {

```



```

        int albumId =
Integer.valueOf(record[AlbumAttributesOrder.getIndex_AlbumID(
)]);

        String artista =
record[AlbumAttributesOrder.getIndex_Artista()];
        String titolo =
record[AlbumAttributesOrder.getIndex_Titolo()];
        int anno =
Integer.valueOf(record[AlbumAttributesOrder.getIndex_Anno()]);
;

        Album album = new Album(albumId, artista,
titolo, anno);
        albumList.add(album);
    }
}

csvReader.close();

if (albumList.isEmpty()) {
    Exception e = new Exception("No Album Found
matching with name: " + albumName);
    throw e;
}

return albumList;
}

private static synchronized List<Album> retrieveById(File
fd, int albumId) throws Exception {
    // create csvReader object passing file reader as a
parameter
    CSVReader csvReader = new CSVReader(new
BufferedReader(new FileReader(fd)));
    String[] record;

    List<Album> albumList = new ArrayList<Album>();

    while ((record = csvReader.readNext()) != null) {
        int posTitolo =
AlbumAttributesOrder.getIndex_AlbumID();

        boolean recordFound =
record[posTitolo].equals(String.valueOf(albumId));
        if (recordFound) {

```

```

        int id =
Integer.valueOf(record[AlbumAttributesOrder.getIndex_AlbumID(
)]);

        String artista =
record[AlbumAttributesOrder.getIndex_Artista()];
        String titolo =
record[AlbumAttributesOrder.getIndex_Titolo()];
        int anno =
Integer.valueOf(record[AlbumAttributesOrder.getIndex_Anno()])
;

        Album album = new Album(id, artista, titolo,
anno);
        albumList.add(album);
    }
}

csvReader.close();

if (albumList.isEmpty()) {
    Exception e = new Exception("No Album Found
matching with ID: " + albumId);
    throw e;
}

return albumList;
}

@Override
public void saveAlbum(Album instance) throws Exception {
    boolean duplicatedRecordId = false;

    synchronized (this.localCache) {
        duplicatedRecordId =
(this.localCache.get(String.valueOf(instance.getAlbumId())) !=
null);
    }

    if (!duplicatedRecordId) {
        try {
            List<Album> albumList = retrieveById(this.fd,
instance.getAlbumId());
            duplicatedRecordId = (albumList.size() != 0);
        } catch (Exception e) {
            duplicatedRecordId = false;

```

```

        }
    }

    if (duplicatedRecordId) {
        DuplicatedRecordException e = new
DuplicatedRecordException(
            "Duplicated Instance ID. Id " +
instance.getAlbumId() + " was already assigned");
        throw e;
    }

    saveAlbum(this.fd, instance);
}

private static synchronized void saveAlbum(File fd, Album
instance) throws Exception {

    // create csvWriter object passing file reader as a
parameter
    CSVWriter csvWriter = new CSVWriter(new
BufferedWriter(new FileWriter(fd, true)));

    String[] record = new String[4];

    record[AlbumAttributesOrder.getIndex_AlbumID()] =
String.valueOf(instance.getAlbumId());
    record[AlbumAttributesOrder.getIndex_Titolo()] =
instance.getTitolo();
    record[AlbumAttributesOrder.getIndex_Artista()] =
instance.getArtista();
    record[AlbumAttributesOrder.getIndex_Anno()] =
String.valueOf(instance.getAnno());

    csvWriter.writeNext(record);
    csvWriter.flush();
    csvWriter.close();
}

@Override
public void removeAlbumById(Album instance) throws
Exception {
    synchronized (this.localCache) {

this.localCache.remove(String.valueOf(instance.getAlbumId()))
;
    }
}

```

```

    }
    removeAlbumById(this.fd, instance);
}

private static synchronized void removeAlbumById(File fd,
Album instance) throws Exception {
    File tmpFD = File.createTempFile("dao", "tmp");
    boolean found = false;

    // create csvReader object passing file reader as a
parameter
    CSVReader csvReader = new CSVReader(new
BufferedReader(new FileReader(fd)));
    String[] record;

    CSVWriter csvWriter = new CSVWriter(new
BufferedWriter(new FileWriter(tmpFD, true)));

    while ((record = csvReader.readNext()) != null) {
        int posId =
AlbumAttributesOrder.getIndex_AlbumID();

        boolean recordFound =
record[posId].equals(String.valueOf(instance.getAlbumId()));
        if (!recordFound) {
            csvWriter.writeNext(record);
        } else {
            found = recordFound;
        }
    }
    csvWriter.flush();

    csvReader.close();
    csvWriter.close();

    if (found) {
        Files.move(tmpFD.toPath(), fd.toPath(),
REPLACE_EXISTING);
    } else {
        tmpFD.delete();
    }
}

private static class AlbumAttributesOrder {
    public static int getIndex_AlbumID() {

```

```

        return 0;
    }

    public static int getIndex_Titolo() {
        return 1;
    }

    public static int getIndex_Artista() {
        return 2;
    }

    public static int getIndex_Anno() {
        return 3;
    }
}

private static void cleanup() {
    File fd = new File(CSV_FILE_NAME);
    fd.delete();
}

public static void main(String[] args) throws Exception {
    cleanup();

    Album a0 = new Album(0, "artista0", "titolo0", 2020);
    Album a1 = new Album(1, "artista1", "titolo1", 2021);
    Album a2 = new Album(2, "artista2", "titolo2", 2022);
    Album a3 = new Album(3, "artista3", "titolo3", 2023);
    Album a4 = new Album(4, "artista4", "titolo1", 2020);

    AlbumDAOCSV dao = new AlbumDAOCSV();
    dao.saveAlbum(a0);
    dao.saveAlbum(a1);
    dao.saveAlbum(a2);
    dao.saveAlbum(a3);
    dao.saveAlbum(a4);

    dao.removeAlbumById(a2);
    List<Album> albumList;
    try {
        albumList = dao.retrieveByAlbumName("titolo2");
    } catch (Exception e) {
        albumList = new ArrayList<Album>();
    }
    assertEquals(0, albumList.size());
}

```

```

        boolean failedSaving = false;
        a3.setArtista("artista5");
        a3.setTitolo("titolo5");
        a3.setAnno(2025);
        try {
            dao.saveAlbum(a3);
        } catch (Exception e) {
            failedSaving = true;
        }
        assertTrue(failedSaving);

        albumList = dao.retreiveByAlbumName("titolo0");
        assertEquals(1, albumList.size());

        albumList = dao.retreiveByAlbumName("titolo1");
        assertEquals(2, albumList.size());

        albumList = dao.retreiveByAlbumName("titolo1");
        assertEquals(2, albumList.size());
    }
}

```

```

package dao;

```

```

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;
import java.util.ArrayList;
import java.util.List;

import dao.queries.*;
import dao.AlbumDAO;
import entities.Album;
import exceptions.DuplicatedRecordException;

```

```

public class AlbumDAOJDBC implements AlbumDAO {

```

```

//Note this is the USER on the DBMS that has proper
privileges in order to access the specific DB

```

```

        private static String USER = "userSimpleMusicCatalog";
        private static String PASS = "thisisfoo";
//      private static String DB_URL = "jdbc:mysql://
localhost:3306/MySimpleMusicCatalog-DB";
//      private static String DB_URL = "jdbc:mysql://saks-
db.iasi.cnr.it:3306/MySimpleMusicCatalog-DB";
private static String DB_URL = "jdbc:mysql://localhost:3306/
MySimpleMusicCatalog-DB?
useSSL=false&allowPublicKeyRetrieval=true";
//      private static String DB_URL = "jdbc:postgresql://
localhost/MySimpleMusicCatalog-DB";
        private static String DRIVER_CLASS_NAME =
"com.mysql.jdbc.Driver";
//      private static String DRIVER_CLASS_NAME =
"org.postgresql.Driver";

        public List<Album> retrieveByAlbumName(String albumName)
throws Exception {
            // STEP 1: dichiarazioni
            Statement stmt = null;
            Connection conn = null;
            List<Album> listOfAlbums = new ArrayList<Album>();

            try {
                // STEP 2: loading dinamico del driver mysql
                Class.forName(DRIVER_CLASS_NAME);

                // STEP 3: apertura connessione
                conn = DriverManager.getConnection(DB_URL, USER,
PASS);

                // STEP 4: creazione ed esecuzione della query
                stmt =
conn.createStatement(ResultSet.TYPE_SCROLL_INSENSITIVE,
                        ResultSet.CONCUR_READ_ONLY);

                // In pratica i risultati delle query possono
essere visti come un Array Associativo o un Map
                ResultSet rs =
SimpleQueries.selectAlbumByName(stmt, albumName);

                if (!rs.first()){ // rs empty
                    Exception e = new Exception("No Album Found
matching with name: "+albumName);
                    throw e;

```

```

    }

    // riposizionamento del cursore
    rs.first();
    do{
        // lettura delle colonne "by name"
        String artist = rs.getString("Artista");
        int albumId = rs.getInt("AlbumId");
        int year = rs.getInt("Anno");

        Album a = new Album(albumId, artist,
albumName, year);

        listOfAlbums.add(a);

    }while(rs.next());

    // STEP 5.1: Clean-up dell'ambiente
    rs.close();
} finally {
    // STEP 5.2: Clean-up dell'ambiente
    try {
        if (stmt != null)
            stmt.close();
    } catch (SQLException se2) {
    }
    try {
        if (conn != null)
            conn.close();
    } catch (SQLException se) {
        se.printStackTrace();
    }
}

return listOfAlbums;
}

public void saveAlbum(Album instance) throws Exception {
    // STEP 1: dichiarazioni
    Statement stmt = null;
    Connection conn = null;

    try {
        // STEP 2: loading dinamico del driver mysql
        Class.forName(DRIVER_CLASS_NAME);

```



```

        // STEP 3: apertura connessione
        conn = DriverManager.getConnection(DB_URL, USER,
PASS);

        // STEP 4.1: creazione ed esecuzione della query
        stmt =
conn.createStatement(ResultSet.TYPE_SCROLL_INSENSITIVE,
        ResultSet.CONCUR_READ_ONLY);

        // In pratica i risultati delle query possono
essere visti come un Array Associativo o un Map
        ResultSet rs =
SimpleQueries.selectAlbumIds(stmt);
        while (rs.next()) {
            // lettura delle colonne "by name"
            int albumId = rs.getInt("AlbumId");
            System.out.println("Found AlbumId: "+
albumId);

            if (albumId == instance.getAlbumId()){
                DuplicatedRecordException e = new
DuplicatedRecordException("Duplicated Instance ID. Id
"+albumId + " was already assigned");
                throw e;
            }
        }

        rs.close();
        stmt.close();

        // STEP 4.2: creazione ed esecuzione della query
        stmt =
conn.createStatement(ResultSet.TYPE_SCROLL_INSENSITIVE,
        ResultSet.CONCUR_READ_ONLY);
        int result = CRUDQueries.inserisciAlbum(stmt,
instance);

        // STEP 5.1: Clean-up dell'ambiente
        rs.close();
    } finally {
        // STEP 5.2: Clean-up dell'ambiente
        if (stmt != null)
            stmt.close();
        if (conn != null)
            conn.close();
    }
}

```

```

    }
}

    public void removeAlbumById(Album instance) throws
Exception {
    // STEP 1: dichiarazioni
    Statement stmt = null;
    Connection conn = null;

    try {
        // STEP 2: loading dinamico del driver mysql
        Class.forName(DRIVER_CLASS_NAME);

        // STEP 3: apertura connessione
        conn = DriverManager.getConnection(DB_URL, USER,
PASS);

        // STEP 4.1: creazione ed esecuzione della query
        stmt =
conn.createStatement(ResultSet.TYPE_SCROLL_INSENSITIVE,
                        ResultSet.CONCUR_READ_ONLY);
        int result = CRUDQueries.eliminaAlbum(stmt,
instance);

    } finally {
        // STEP 5.2: Clean-up dell'ambiente
        if (stmt != null)
            stmt.close();
        if (conn != null)
            conn.close();
    }

}

}

```

```

package dao.queries;
import java.sql.*;
import entities.Album;

public class CRUDQueries {
    public static int inserisciAlbum(Statement stmt, Album
album) throws SQLException {

```

```

        String insertStatement = String.format("INSERT INTO
Album (AlbumId, Titolo, Artista, Anno) VALUES (%s,'%s','%s',
%s)", album.getAlbumId(), album.getTitolo(),
album.getArtista(), album.getAnno());
        System.out.println(insertStatement);
        return stmt.executeUpdate(insertStatement);
    }

    public static int aggiornaAlbum(Statement stmt, Album
album) throws SQLException {
        String updateStatement = String.format("UPDATE Album
set Titolo='%s', Artista='%s', Anno=%s WHERE AlbumId = %s",
album.getArtista(), album.getTitolo(), album.getAnno(),
album.getAlbumId());
        System.out.println(updateStatement);
        return stmt.executeUpdate(updateStatement);
    }

    public static int eliminaAlbum(Statement stmt, Album
album) throws SQLException {
        String deleteStatement = String.format("DELETE FROM
Album WHERE AlbumId = %s", album.getAlbumId());
        System.out.println(deleteStatement);
        return stmt.executeUpdate(deleteStatement);
    }

    public static void stampaResultSet( Statement stmt)
throws SQLException{
        try (ResultSet res = stmt.executeQuery("SELECT * FROM
Album")) {
            while (res.next()) {
                System.out.printf("%s : %s (%s)\n",
res.getString("Artista"), res.getString("Titolo"),
res.getInt("Anno"));
            }
        }
    }
}

```

```

package dao.queries;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;

```

```

public class SimpleQueries {

    public static ResultSet selectAlbumByName(Statement stmt,
String title) throws SQLException {
        String sql = "SELECT * FROM Album where Titolo = '" +
title + "'";
        System.out.println(sql);
        return stmt.executeQuery(sql);
    }

    public static ResultSet selectAlbumIds(Statement stmt)
throws SQLException {
        String sql = "SELECT DISTINCT AlbumId FROM Album ";
        System.out.println(sql);
        return stmt.executeQuery(sql);
    }
}

```

Out con JDBC:

```

Looking for albums named: Made In Japan
SELECT * FROM Album where Titolo = 'Made In Japan';
Result 1: Deep Purple "Made In Japan" (1972)
*****
Adding the new album instance named: Ten
SELECT DISTINCT AlbumId FROM Album ;
Found AlbumId: 1
Found AlbumId: 2
Found AlbumId: 3
Found AlbumId: 4
INSERT INTO Album (AlbumId, Titolo, Artista, Anno) VALUES
(55, 'Ten', 'Pearl Jam', 1991)
... done!
SELECT DISTINCT AlbumId FROM Album ;
Found AlbumId: 1
Found AlbumId: 2
Found AlbumId: 3
Found AlbumId: 4
Found AlbumId: 55
... ops! Album already present.
Removing the old item (i.e. ID: 55)...
DELETE FROM Album WHERE AlbumId = 55
... removal completed!
Replacing with an updated item (i.e. ID: 56)...
SELECT DISTINCT AlbumId FROM Album ;

```

```

Found AlbumId: 1
Found AlbumId: 2
Found AlbumId: 3
Found AlbumId: 4
INSERT INTO Album (AlbumId, Titolo, Artista, Anno) VALUES
(56, 'Ten', 'Pearl Jam', 1991)
... replacement completed!
*****
Removing the item just added ...
DELETE FROM Album WHERE AlbumId = 56
... done!
*****

```

Riconosco che questo codice non è di immediata comprensione; per chiarirne il funzionamento, il metodo più efficace è eseguirlo in modalità debug, avanzando passo dopo passo e osservando l'evoluzione delle variabili.

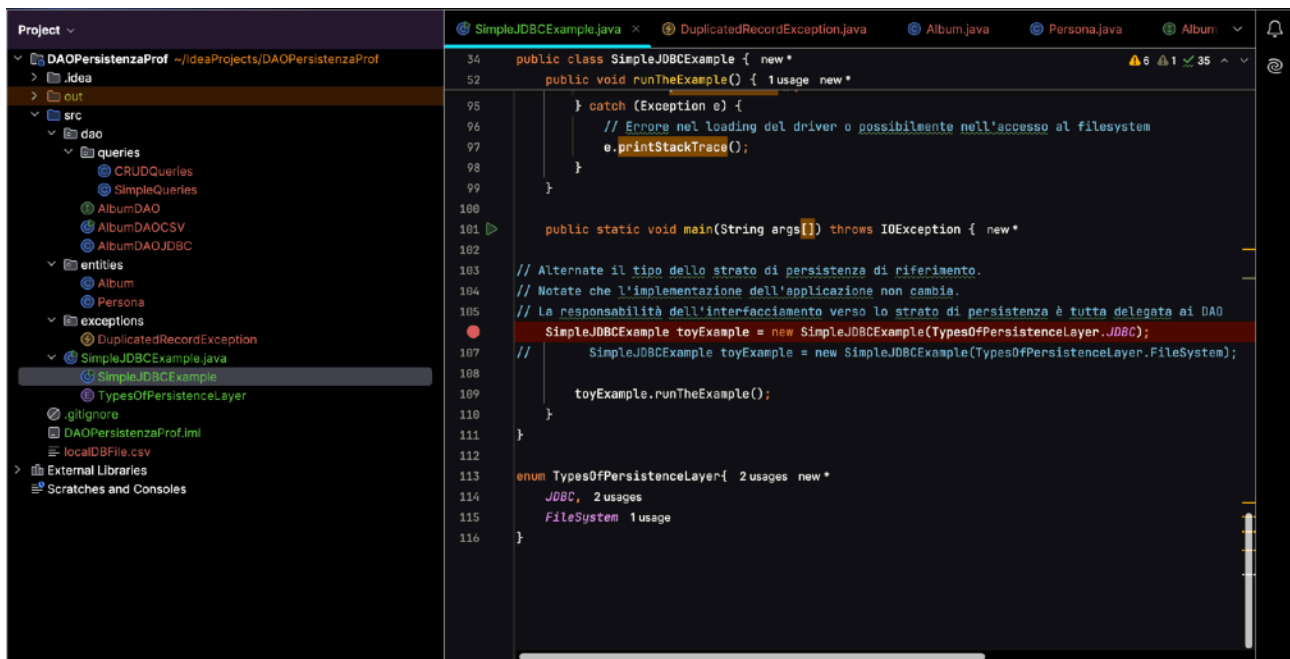
Per eseguire correttamente il progetto “MySimpleMusicCatalog”, è necessario installare tutte le librerie esterne da cui dipendono le varie classi — in particolare quelle usate per JDBC, CSV e test unitari.

Le librerie sono: MySQL Connector/JDBC, OpenCSV, Apache Commons Lang. Tutte le librerie sono incluse nel package del Prof. De Angelis, bisogna solamente importarle con file->project structure->module->dependencies (in IntelliJCE). In libraries->aggiungere il jar di mysql.

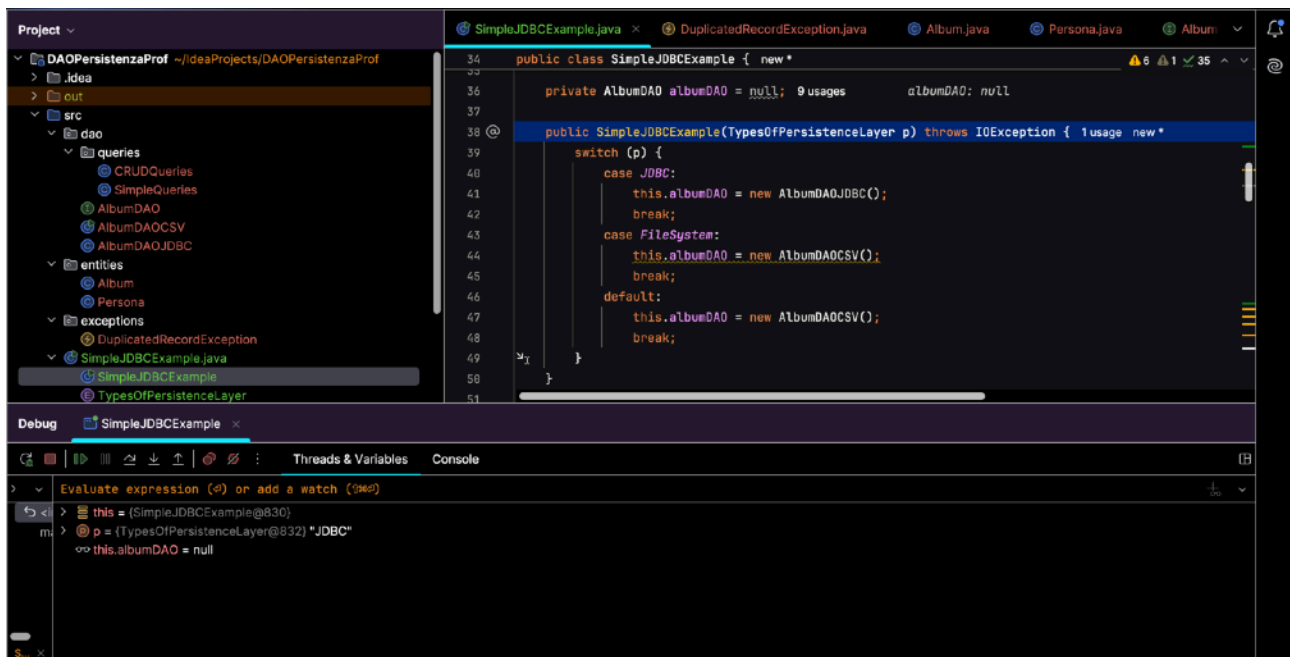
È inoltre fondamentale importare il dump fornito dal ricercatore, ossia il file **.sql** contenente la struttura e i dati del database da utilizzare in locale. Se non si ha familiarità con MySQL, è sufficiente aprire il terminale, accedere come utente root e digitare il seguente comando: `source /percorso/assoluto/al/file.sql`;

Il codice inizialmente è impostato sulla persistenza JDBC non su CSV.

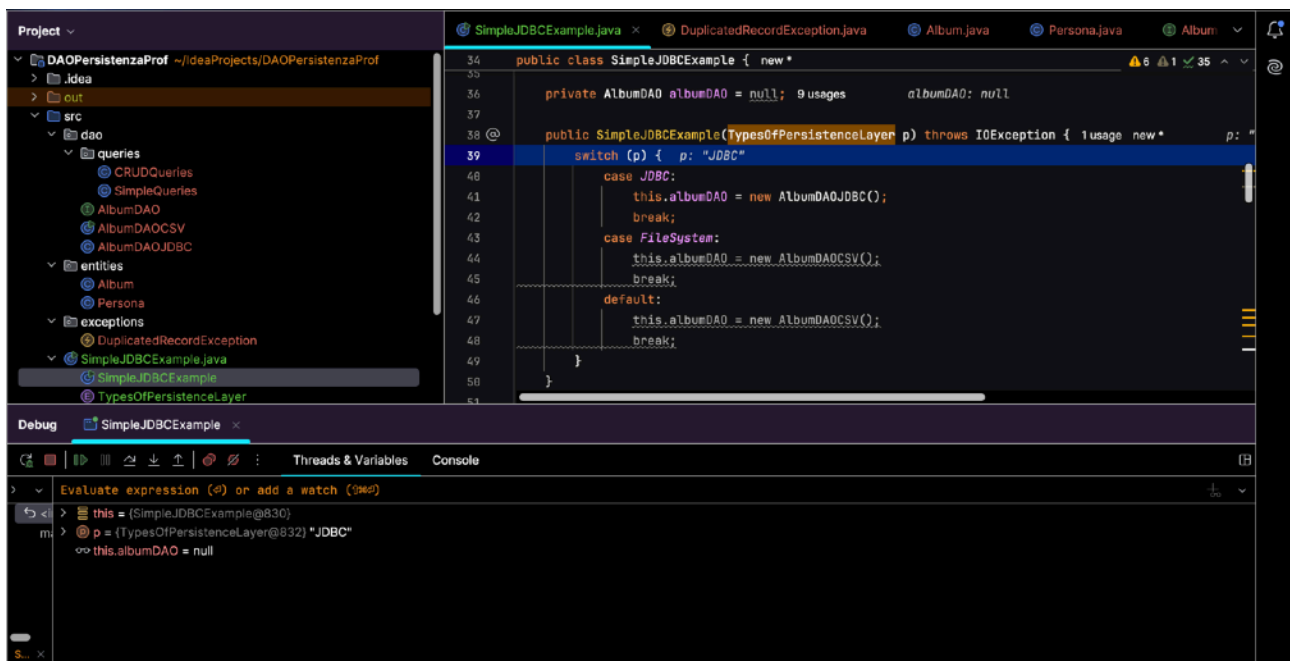
Detto ciò, procediamo: avvio il debug.



Il codice inizia dalla linea 106. Questa linea è molto importante, perché determina quale “strato di persistenza” il programma utilizzerà (cioè da dove verranno letti e scritti i dati). In questo caso utilizzeremo JDBC - persistenza su DBMS (Database). A questo punto viene chiamato il costruttore della classe SimpleJDBCExample.



Al costruttore viene passato il valore `TypesOfPersistenceLayer.JDBC`, cioè una costante dell'enum `TypesOfPersistenceLayer`. Quindi `TypesOfPersistenceLayer.JDBC` indica esplicitamente che vogliamo usare la connessione JDBC (Database MySQL) e non il file CSV.

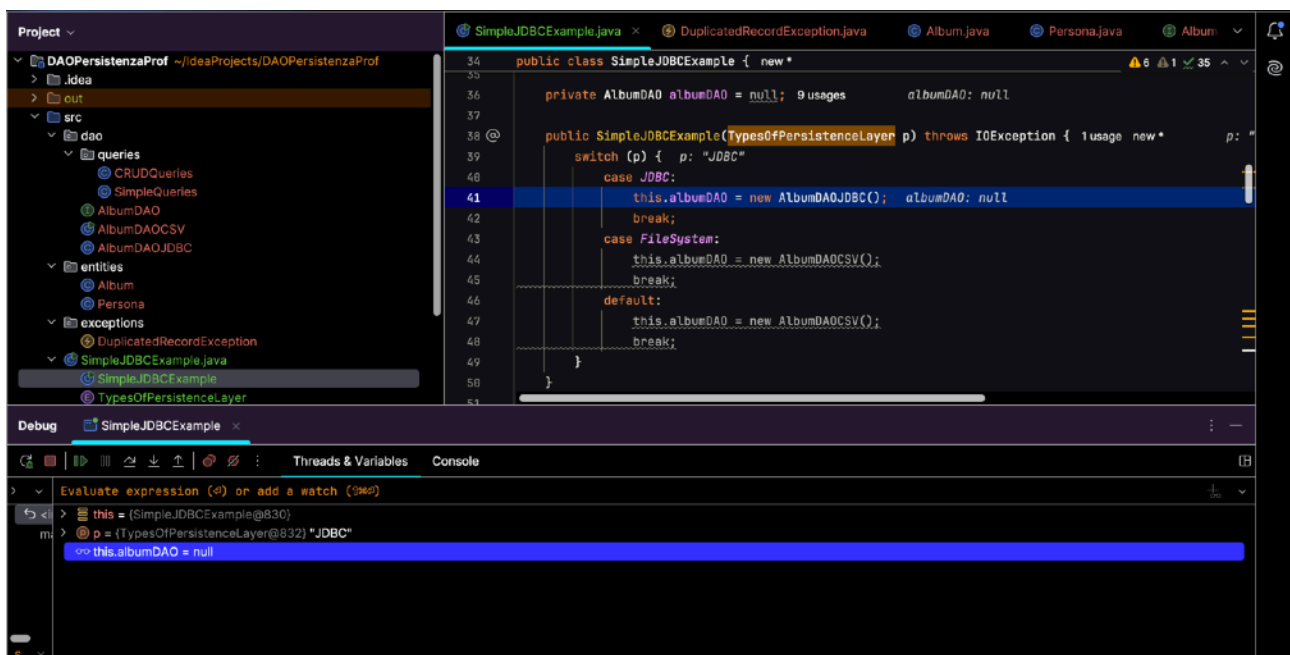


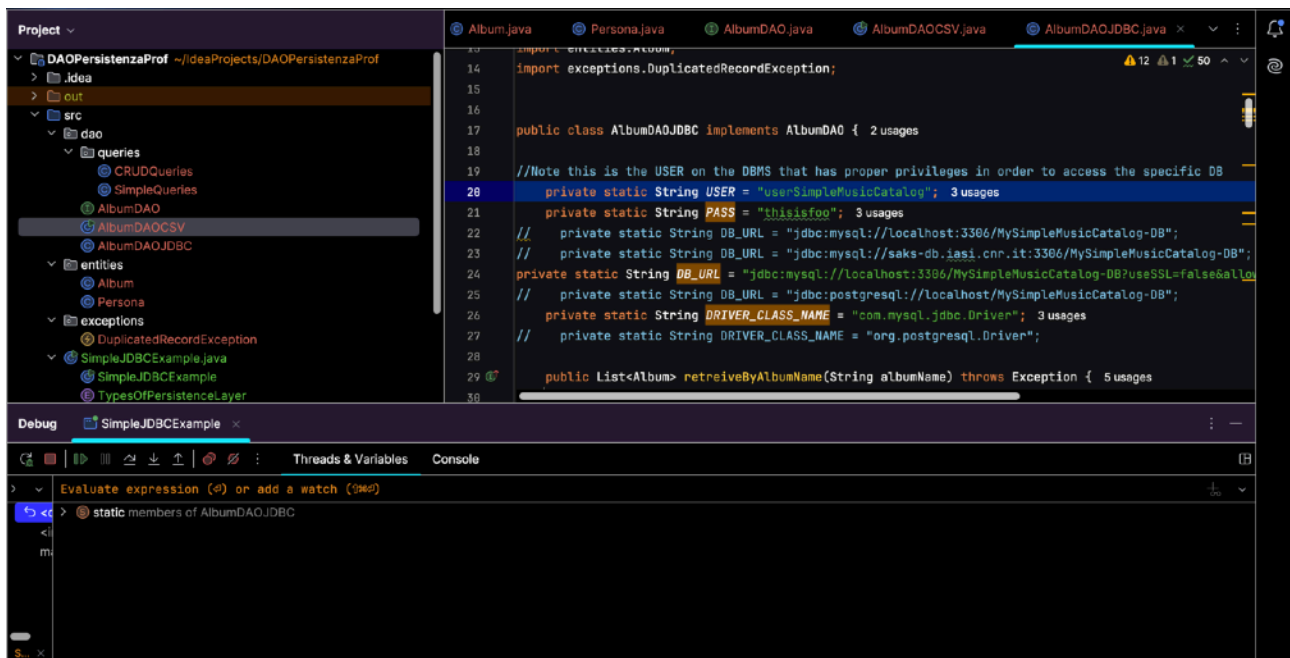
A questo punto del codice si effettua una prima operazione fondamentale: il programma verifica il valore della variabile *p* attraverso una struttura di controllo *switch-case*, per determinare quale tipo di persistenza utilizzare.

È evidente che utilizzeremo la persistenza JDBC.

`albumDAO` è una variabile di istanza di tipo `AlbumDAO` (un'interfaccia).

`new AlbumDAOJDBC()` crea un **oggetto concreto** che implementa quell'interfaccia.

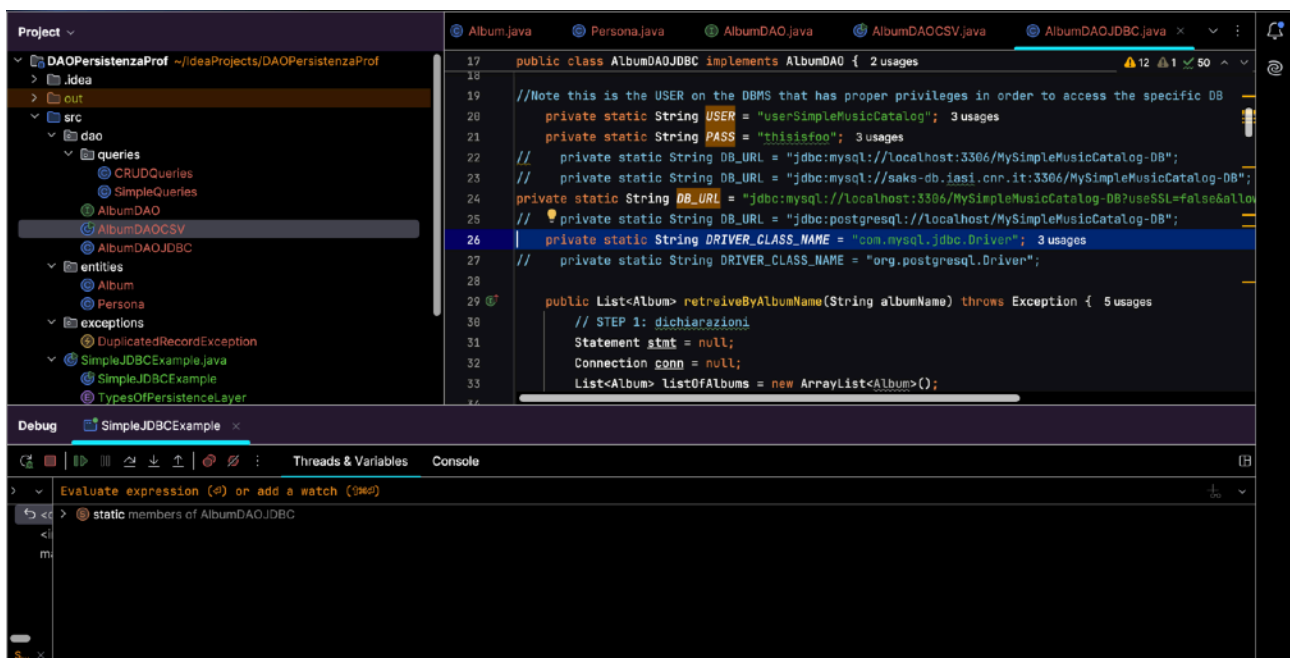




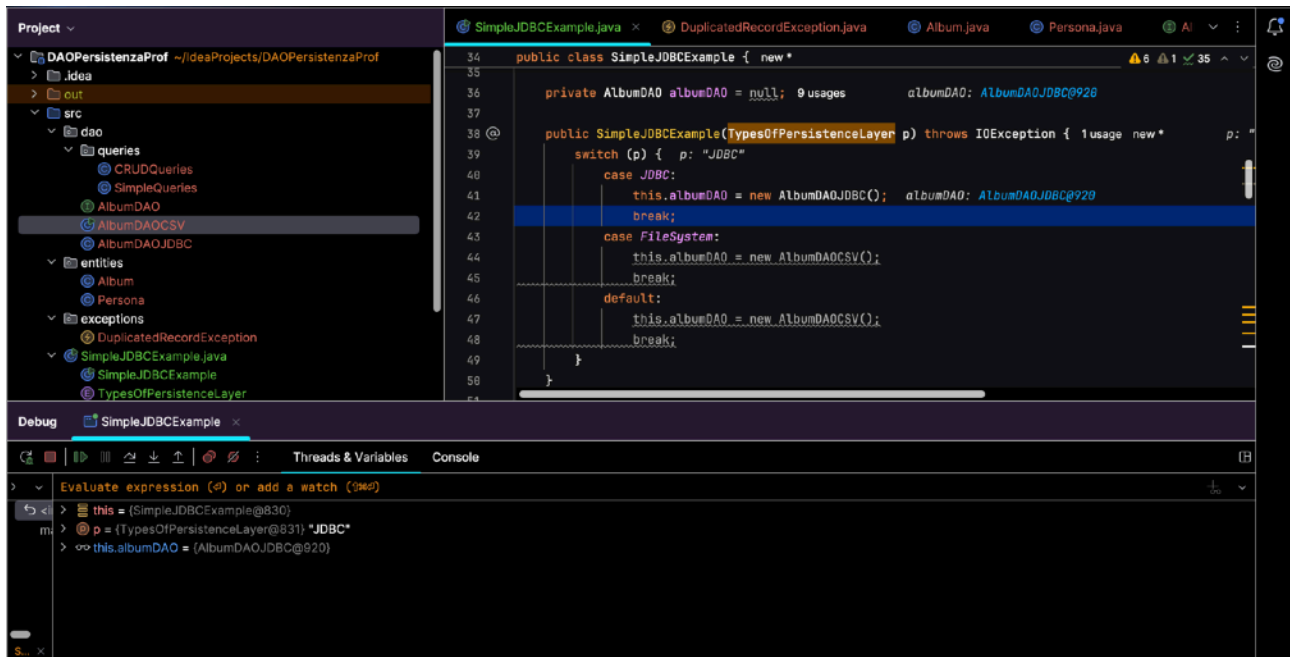
Ora entriamo dentro la classe AlbumDAOJDBC, cioè il cuore dello *strato di persistenza JDBC* (Database Access Layer). Subito dopo, vengono dichiarate alcune costanti: USER, PASS, DB_URL e DRIVER_CLASS_NAME.

Queste costanti verranno utilizzate ogni volta che si apre una connessione al database: DB_URL specifica *dove* si trova il database, USER e PASS indicano *chi* accede e DRIVER_CLASS_NAME serve per caricare dinamicamente il driver.

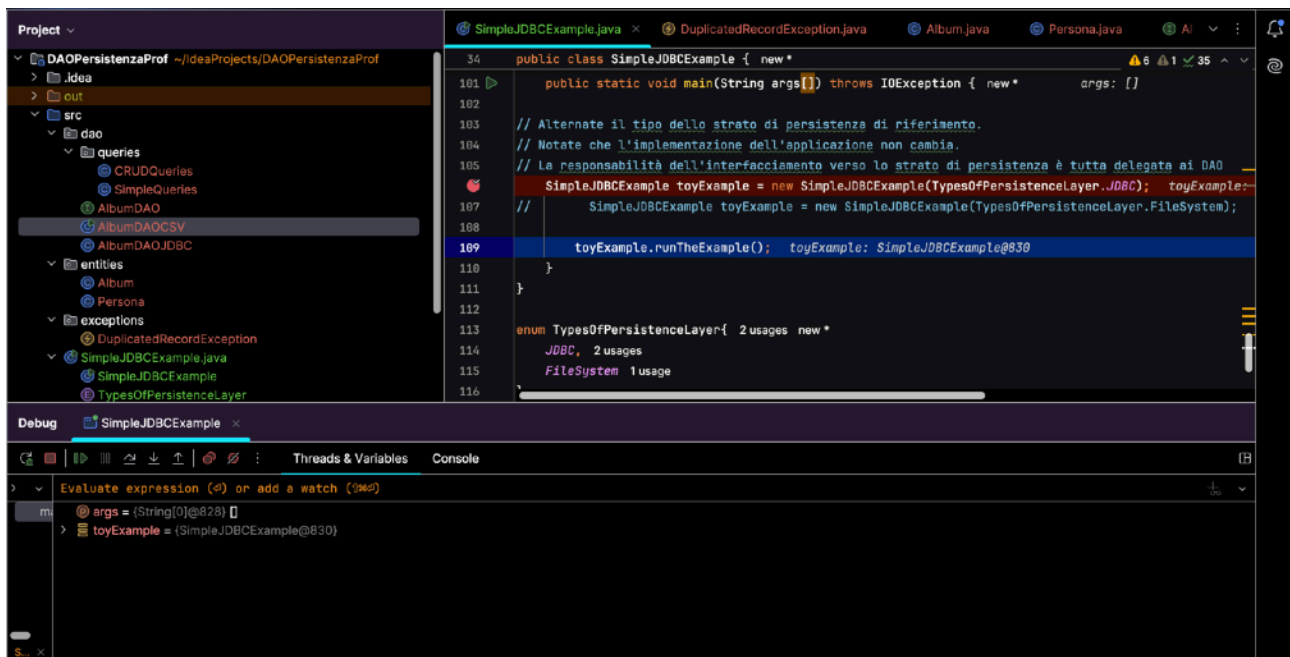
Imparatele a memoria, non servono così tanto.



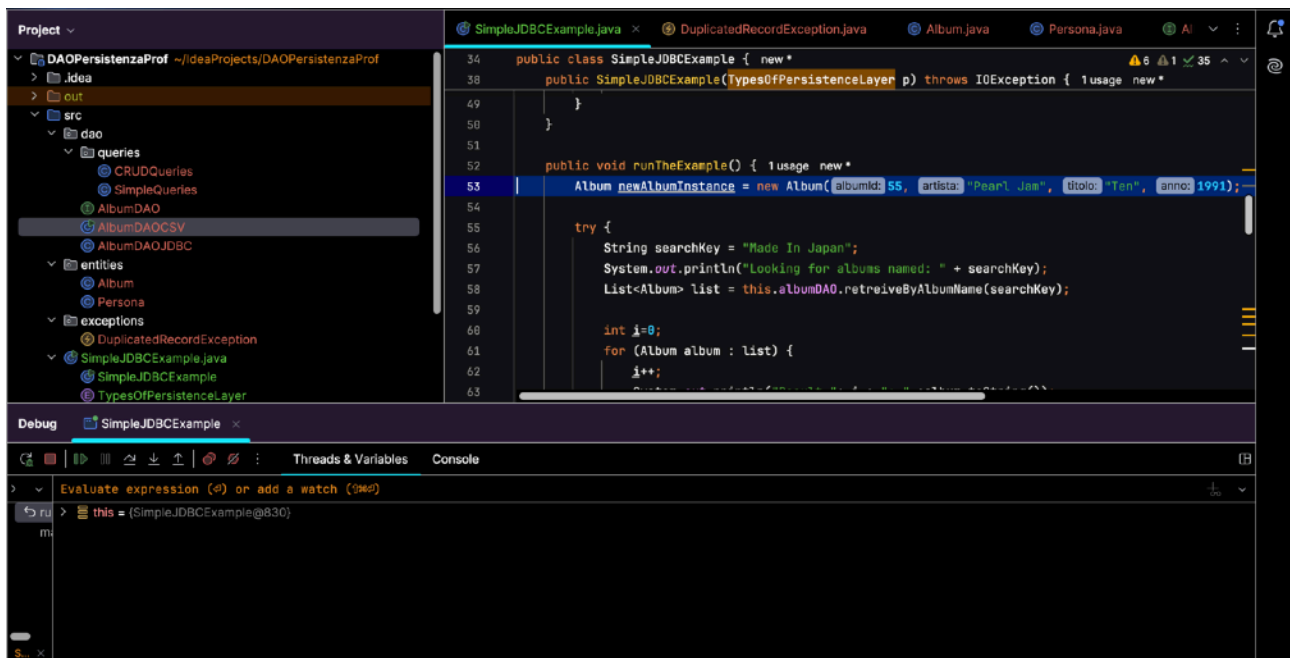
A questo punto torniamo nello switch case.



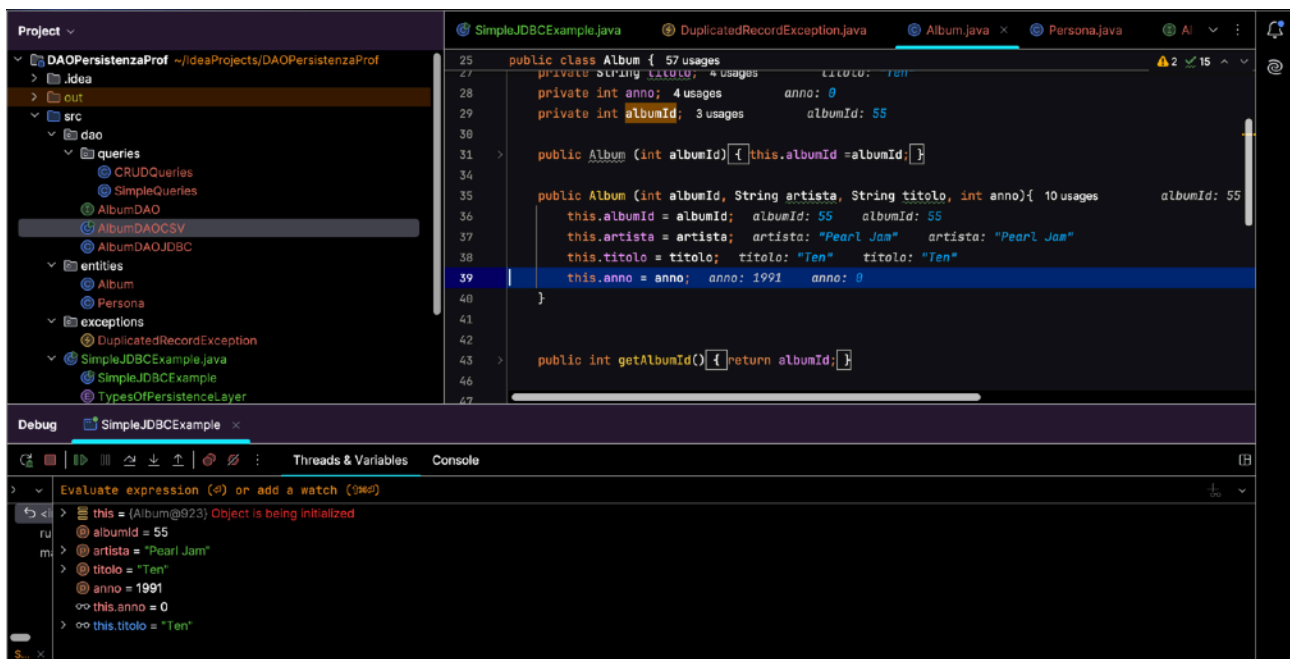
this.albumDAO è diventato un oggetto di tipo AlbumDAOJDBC



Il metodo `runTheExample()` è la **dimostrazione pratica** del funzionamento dell'intero strato di persistenza. Questo metodo effettua le seguenti operazioni: Viene creata un'istanza di `Album` in memoria, ma **non ancora salvata nel DB**.



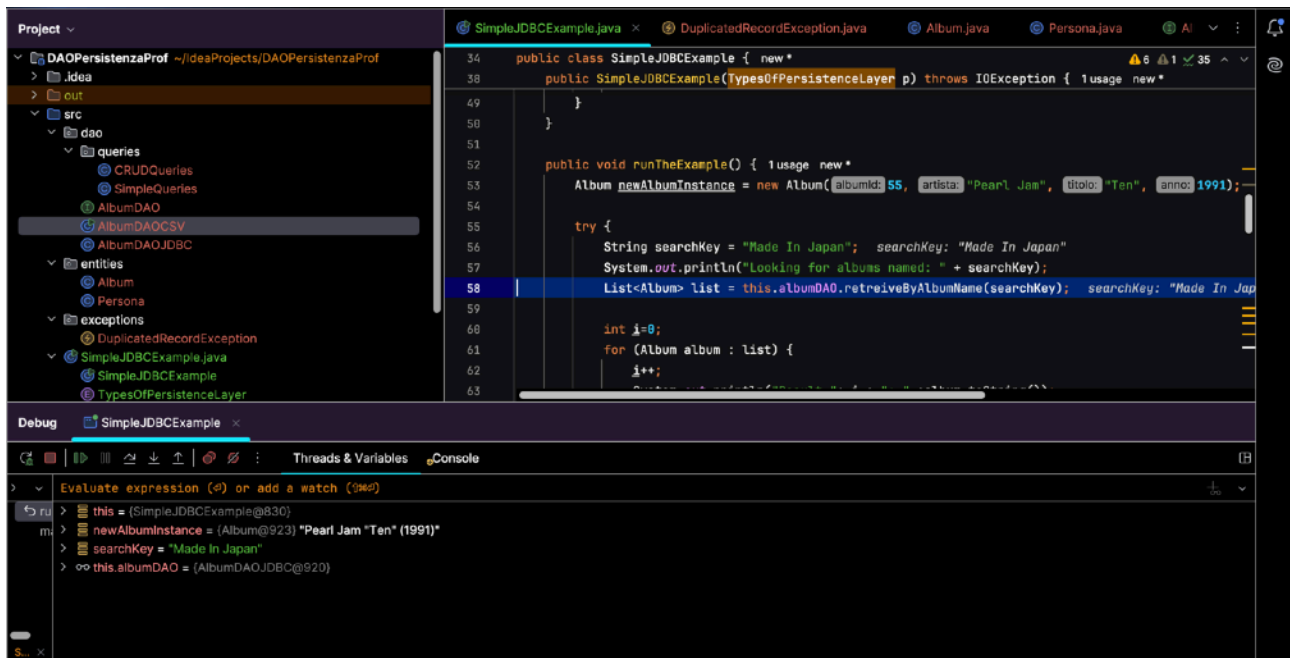
E quindi chiama il costruttore di Album.java e inizializza.



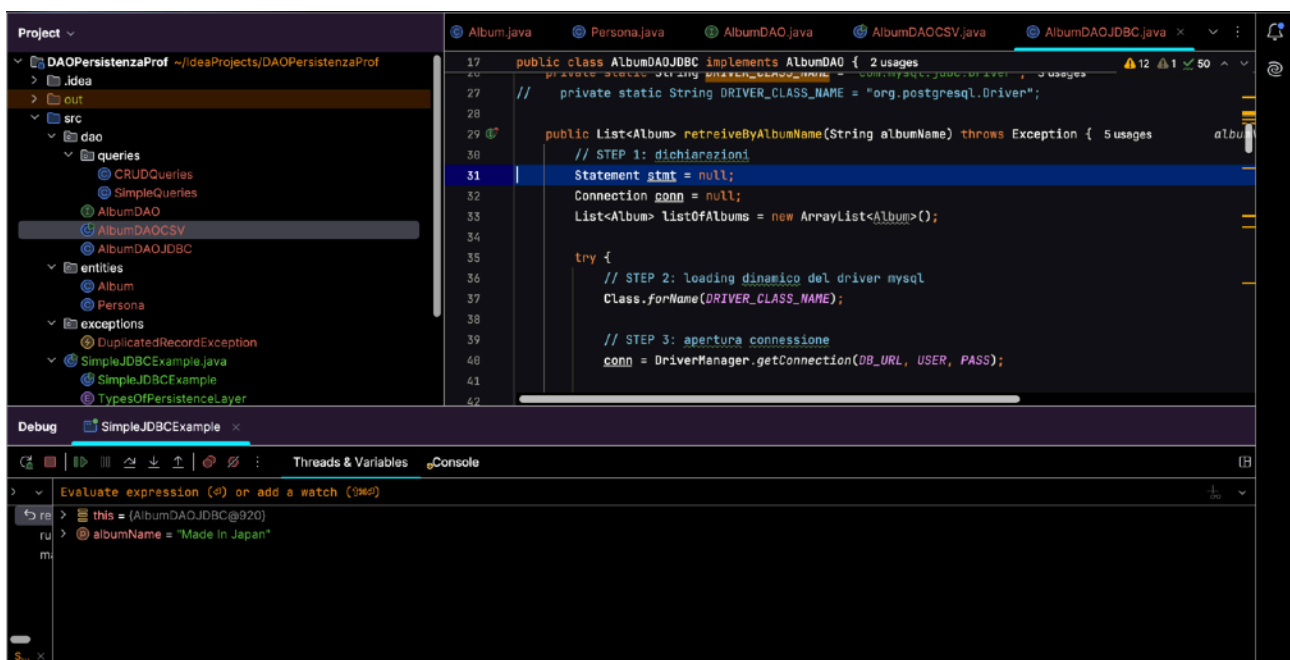
Dopodiché cerca un album nel database.

Chiama il metodo retrieveByAlbumName() sul DAO (che è un AlbumDAOJDBC).

All'interno di AlbumDAOJDBC viene eseguita una query:



E adesso è ovvio che andremo nella classe AlbumDAOJDBC.java per interagire con il database.



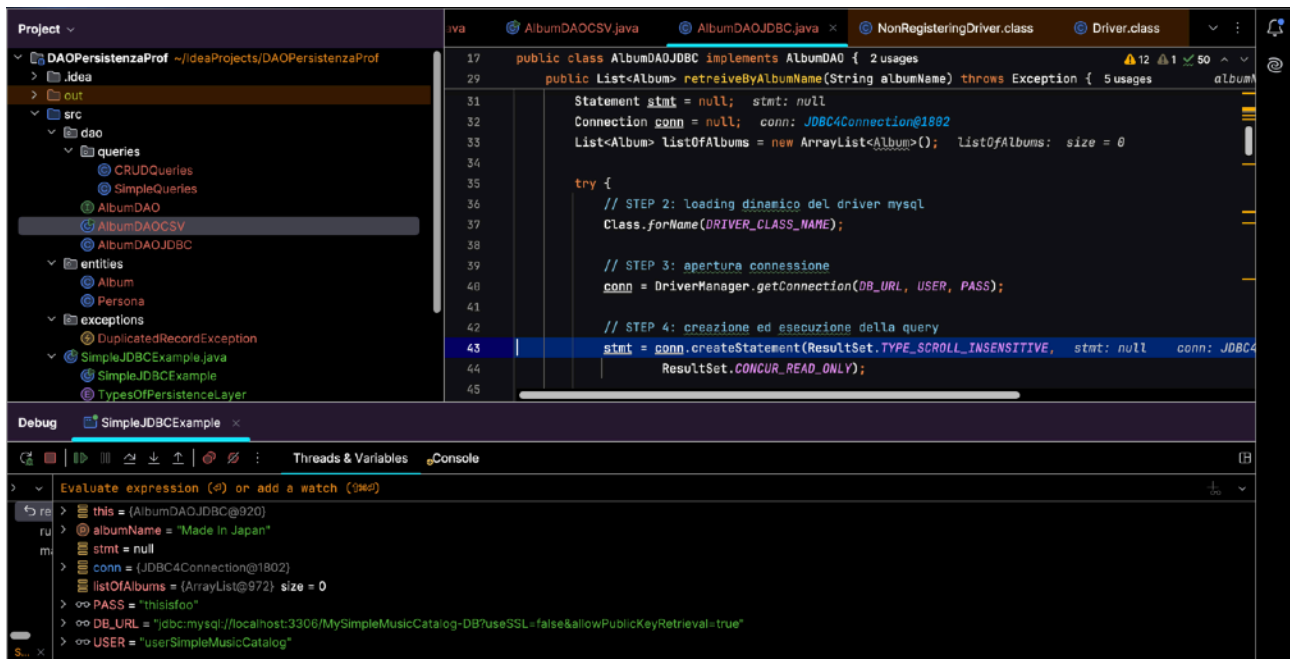
A questo punto viene aperta la connessione passando come parametri al metodo getConnection() le variabili contenenti lo schema del DB, l'utente e la password di accesso.

Dal Connection (conn) appena aperto, viene creato uno Statement, cioè un oggetto che serve a inviare comandi SQL al database.

I due parametri specificano come si comporterà il ResultSet (cioè l'insieme dei risultati della query).

ResultSet.TYPE_SCROLL_INSENSITIVE: Il cursore può scorrere avanti e indietro nei risultati.

ResultSet.CONCUR_READ_ONLY: I dati nel ResultSet non possono essere modificati. È una copia di sola lettura.

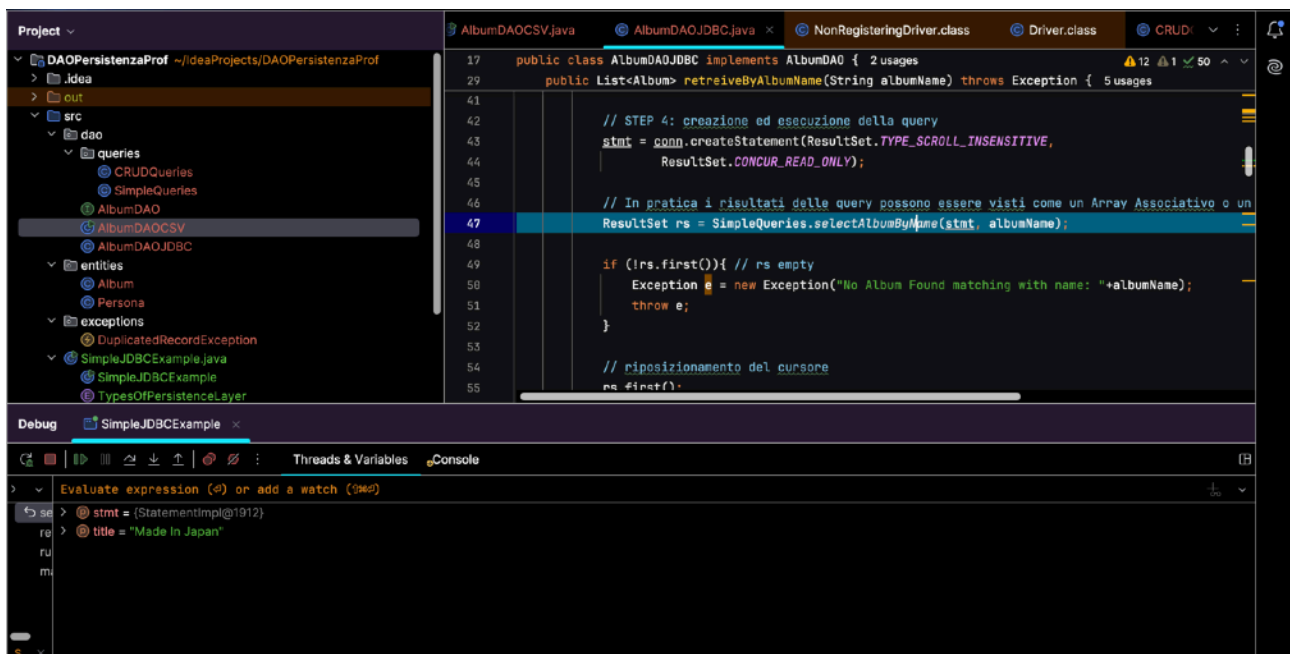


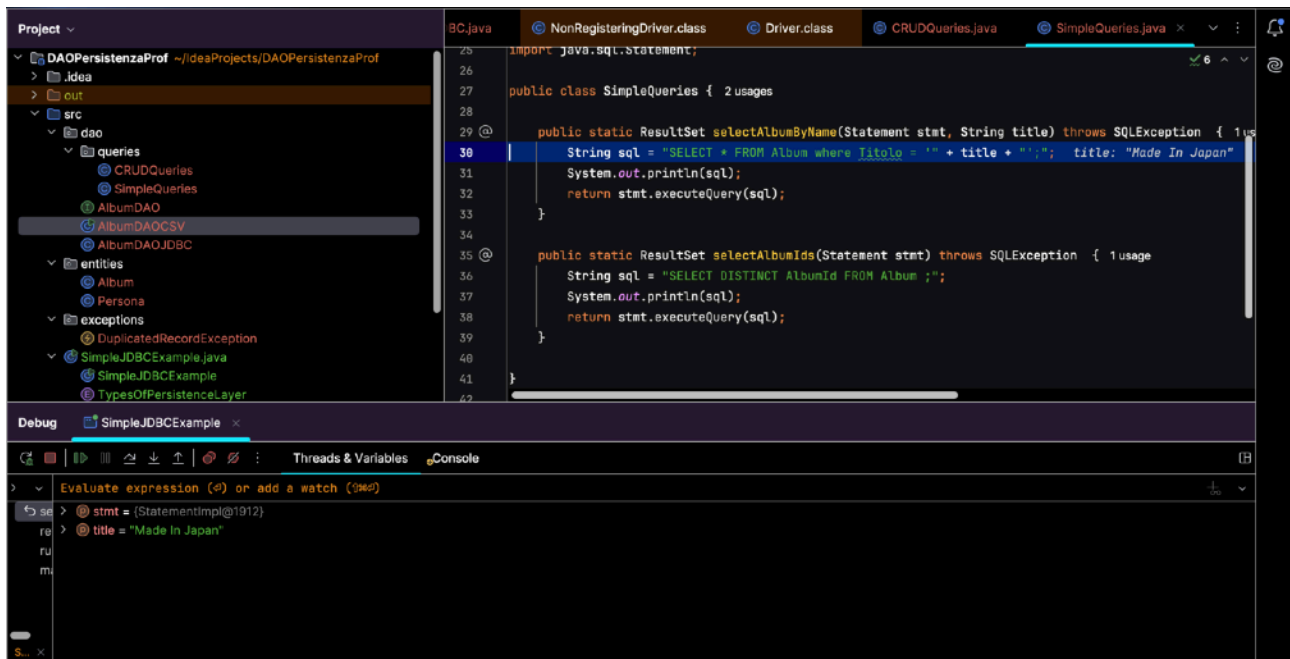
Ora viene chiamato il metodo statico `selectAlbumByName()` della classe `SimpleQueries`.

Gli vengono passati due argomenti:

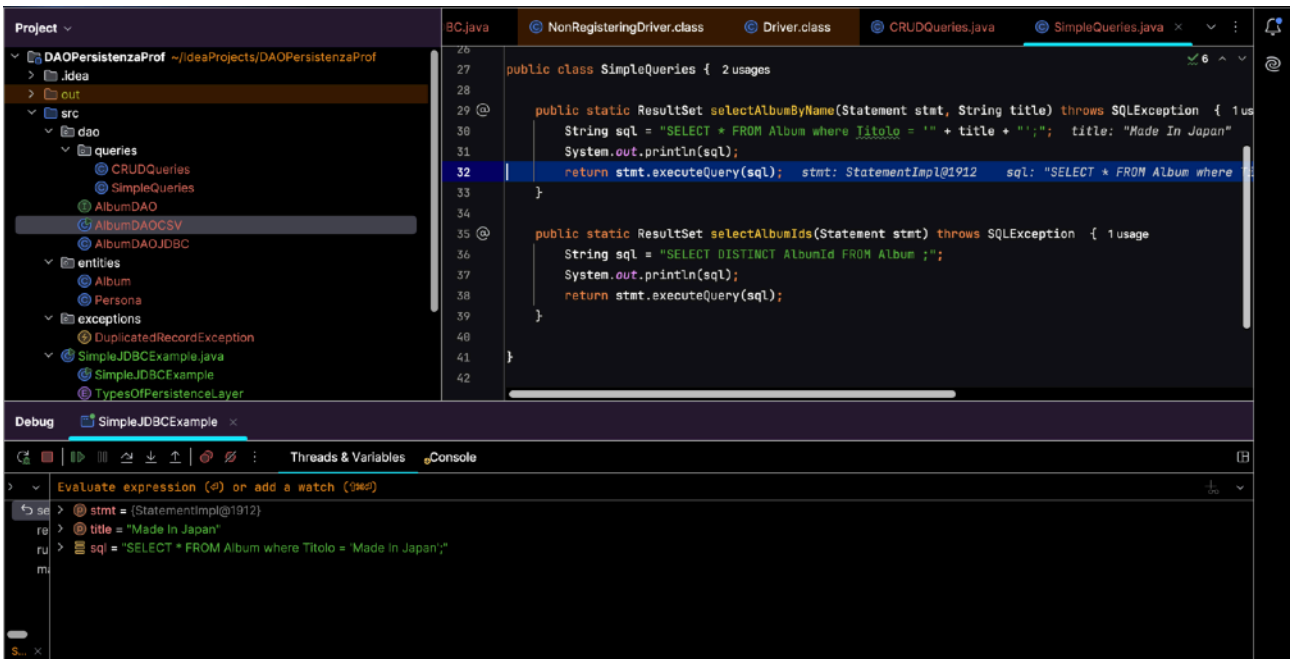
- `stmt`: lo Statement appena creato (collegato al database).
- `albumName`: il valore cercato, ad esempio "Made In Japan".

All'interno del metodo, viene costruita dinamicamente la query SQL (sostanzialmente un'interrogazione al database).





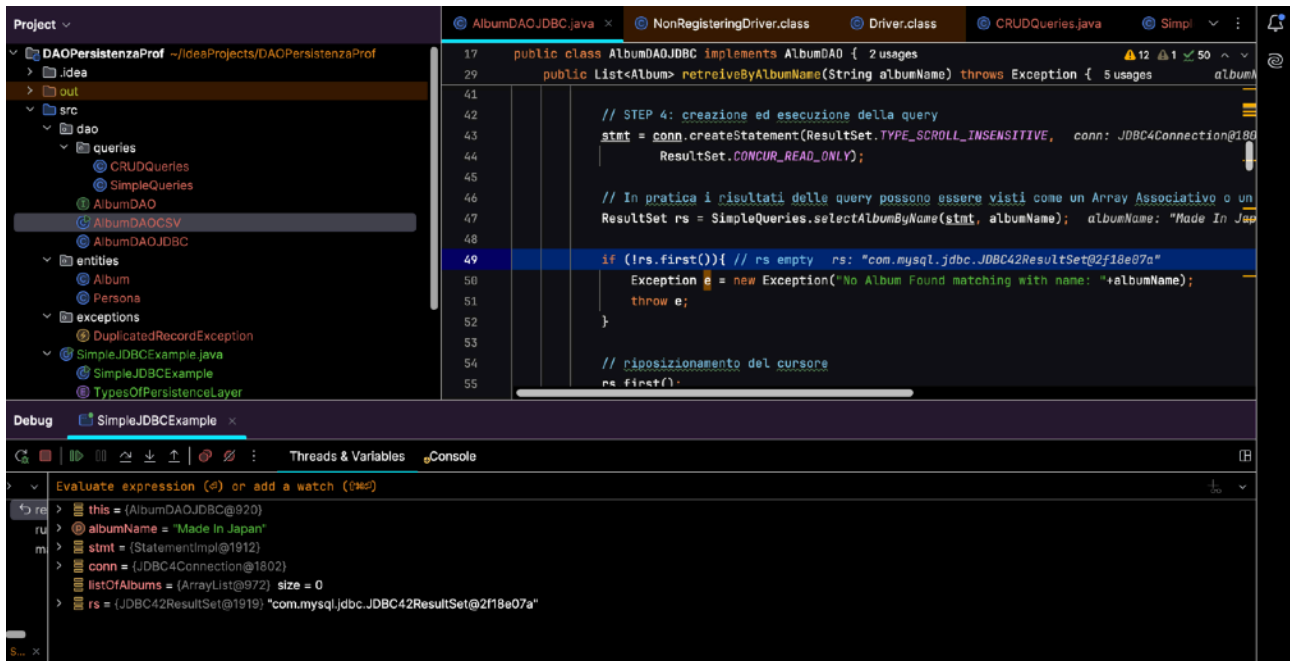
Ci siamo? Uno potrebbe d  “query in chiaro ma che   sta roba” - riga 30.
  un esempio, non siamo al corso di Basi di Dati.
Perlopi  il metodo selectAlbumByName()   anche statico, e lo abbiamo chiamato tramite la classe SimpleQueries.



Adesso la query viene eseguita. All’interno della tabella Album del database stiamo proiettando tutti i campi (*) selezionando la tupla dove il titolo   “Made in Japan”.
Vediamo come   composto il database.

	123 AlbumId	A-Z Titolo	A-Z Artista	123 Anno
1	1	Made In Japan	Deep Purple	1,972
2	2	Be	Pain Of Salvation	2,004
3	3	Images And Words	Dream Theater	1,992
4	4	The Human Equation	Ayreon	2,004

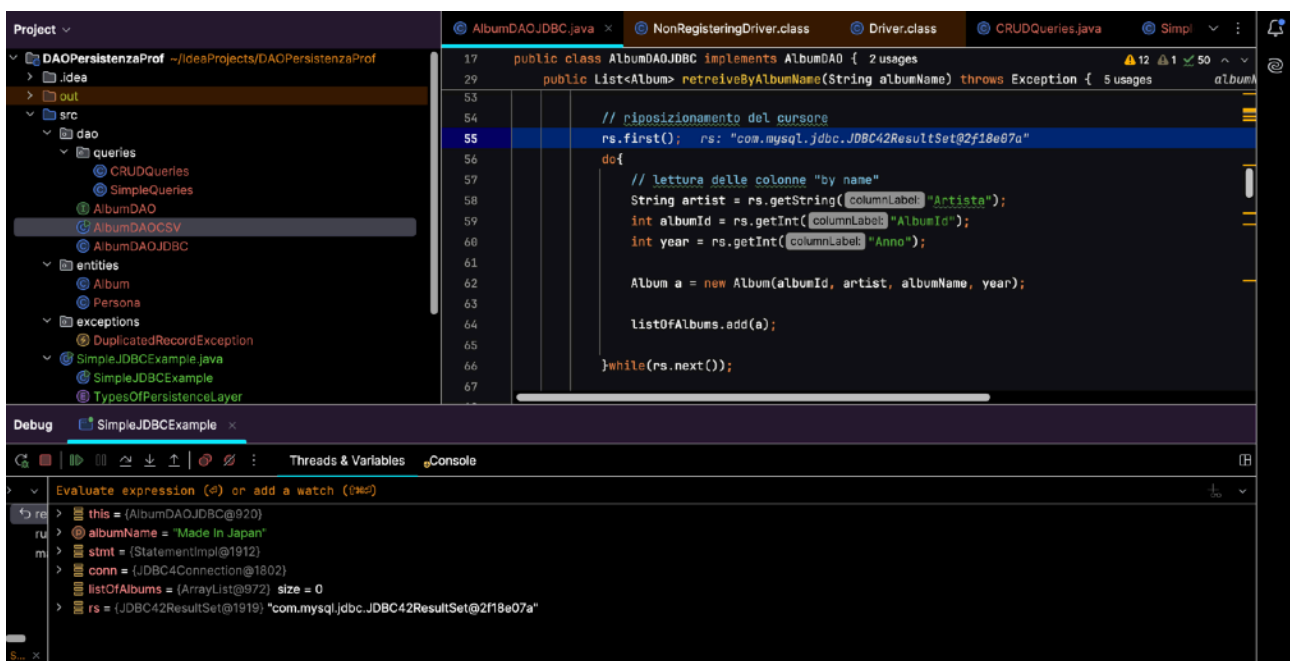
Abbiamo un match. Viene restituita tutta la riga, perché il prof ha scritto “select *”.
Se avesse messo “select titolo from ...” Il risultato sarebbe stato solo “Made in Japan”.
Quindi il valore è presente.



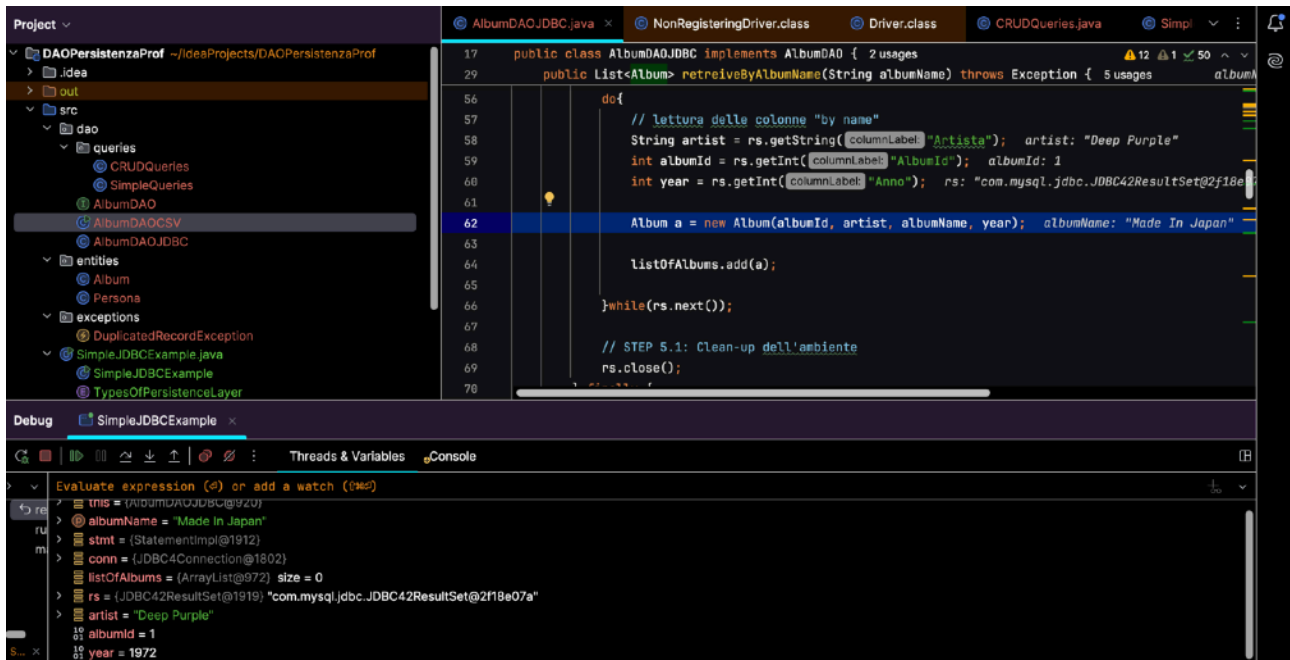
Questo serve per controllare se la query ha restituito risultati:

- `rs.first()` sposta il cursore alla prima riga.
- Se restituisce false, significa che la query non ha trovato nulla.
- In quel caso, viene lanciata un’eccezione con il messaggio “No Album Found...”.

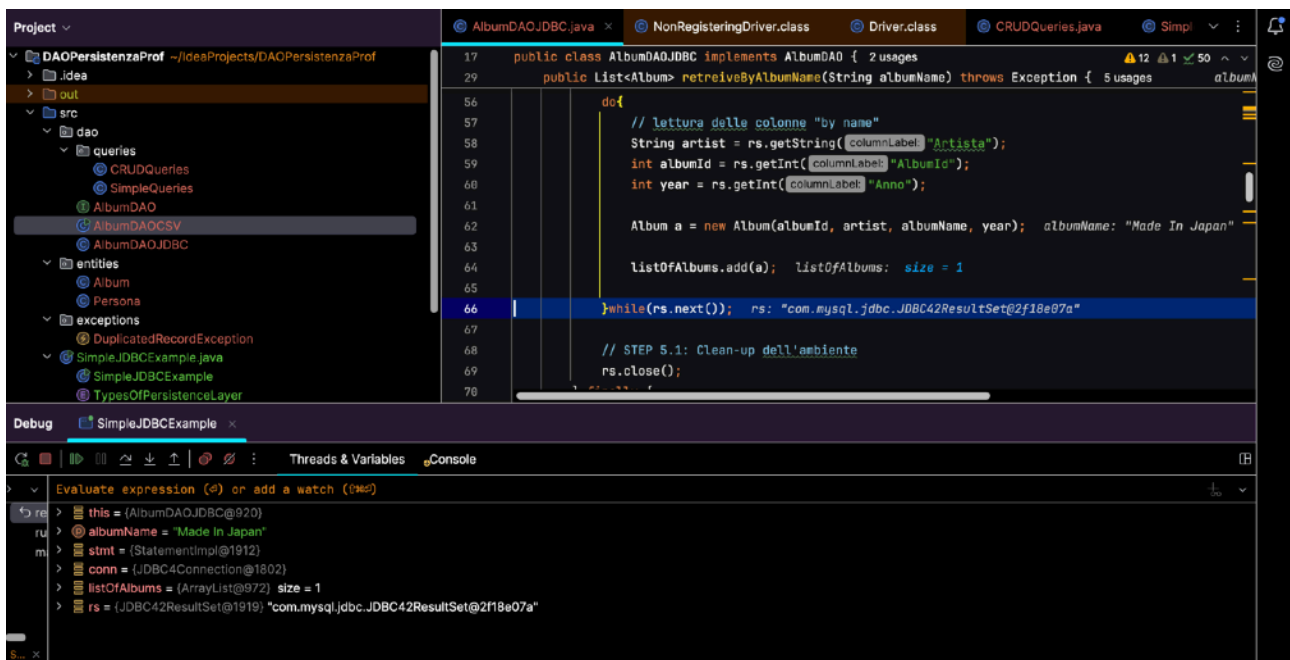
Ovviamente l’istruzione non viene eseguita, perché la query ha trovato qualcosa.



Se la query ha trovato almeno un risultato (`rs.first()` è true), il programma entra nel blocco successivo e legge i campi. Qui viene **letto ogni campo** del risultato e costruito un oggetto `Album`. La lettura avviene per nome delle colonne. Tutti gli album trovati vengono aggiunti a una lista (`listOfAlbums`) che verrà poi restituita al chiamante.



Se notate bene le variabili `artist`, `albumId` e `year` vengono popolate con i valori che sono presenti nel database.

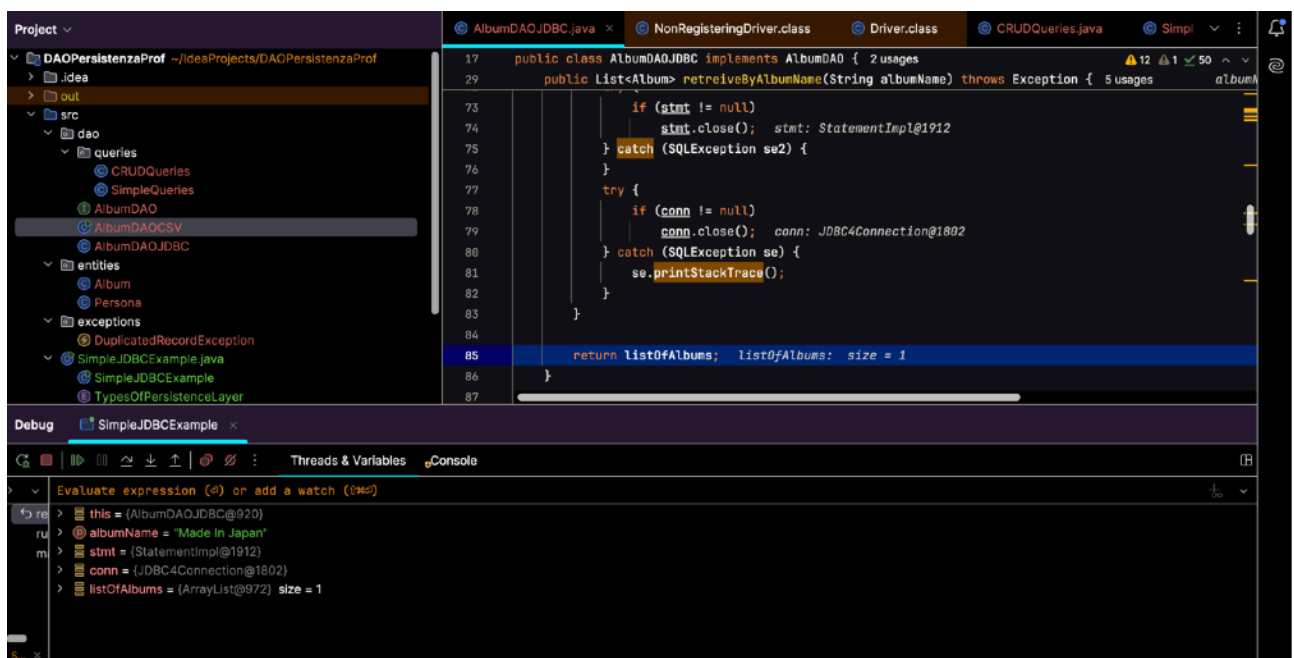
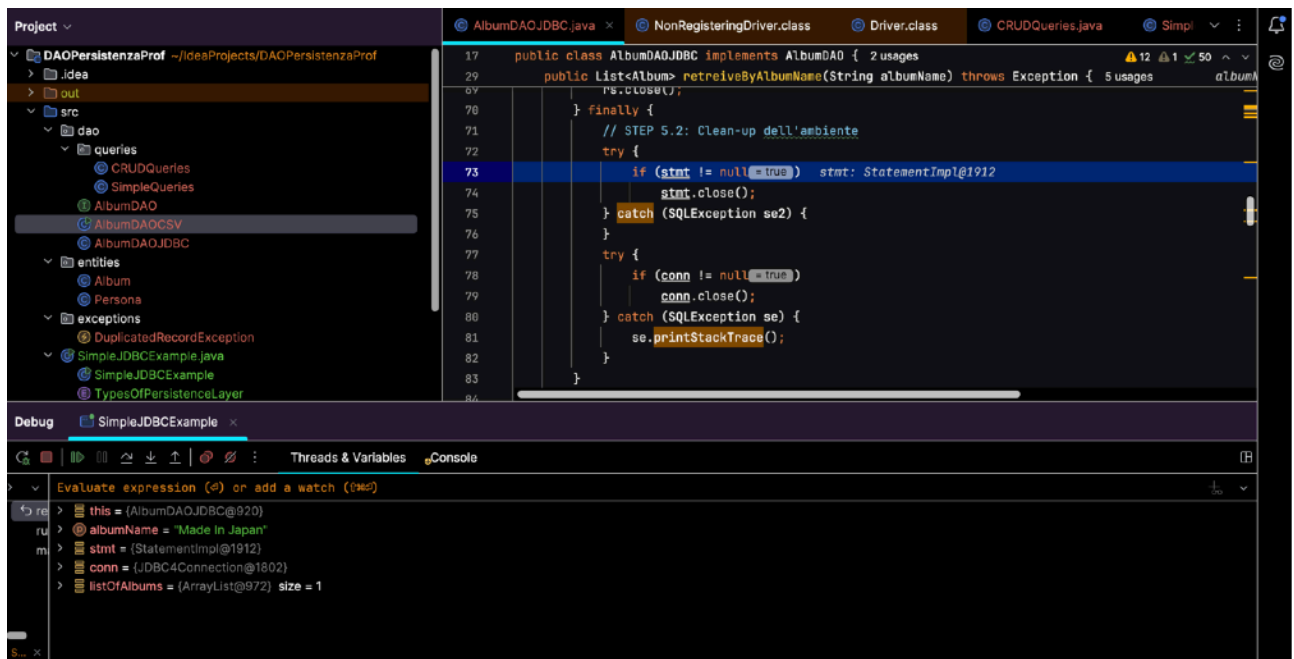


A questo punto chiudiamo lo statement e la connessione.

Il blocco `finally` viene sempre eseguito: verifica se la connessione `conn` è ancora aperta.

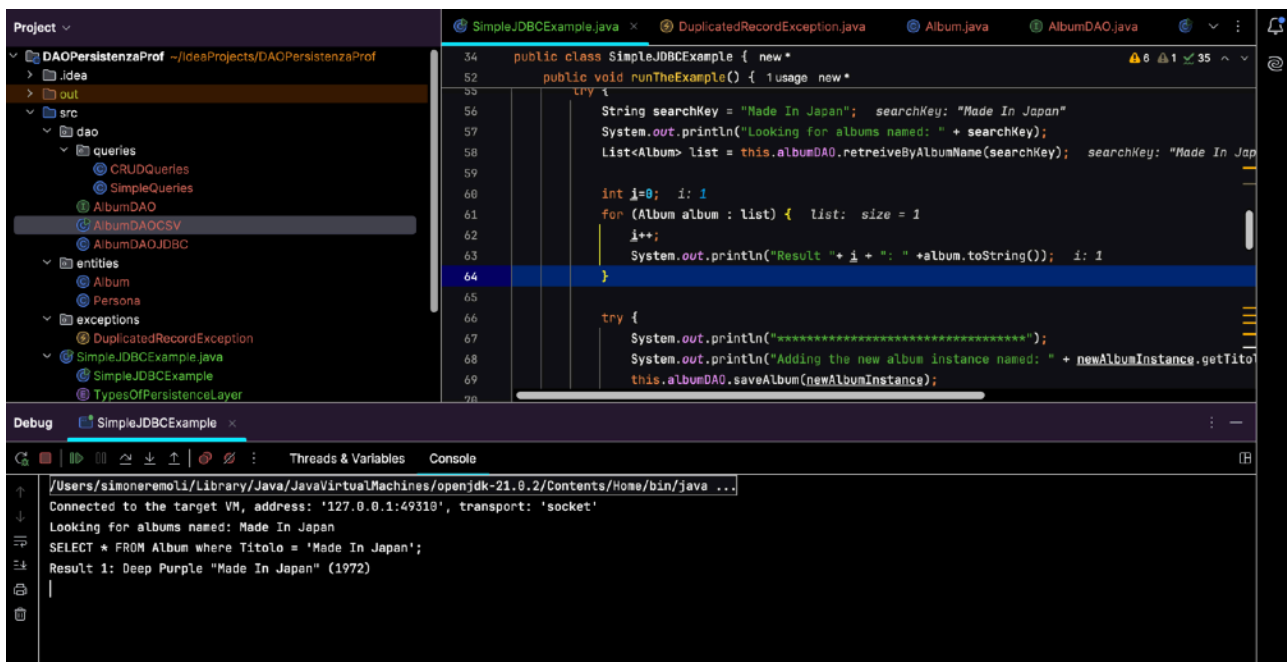
- Se sì, la chiude con `conn.close()`.

Questo è **fondamentale**, perché se non si chiude la connessione, il database mantiene attive sessioni non più utilizzate, con conseguente consumo di risorse o blocco di nuove connessioni.



La collezione di oggetti viene restituita al chiamante.

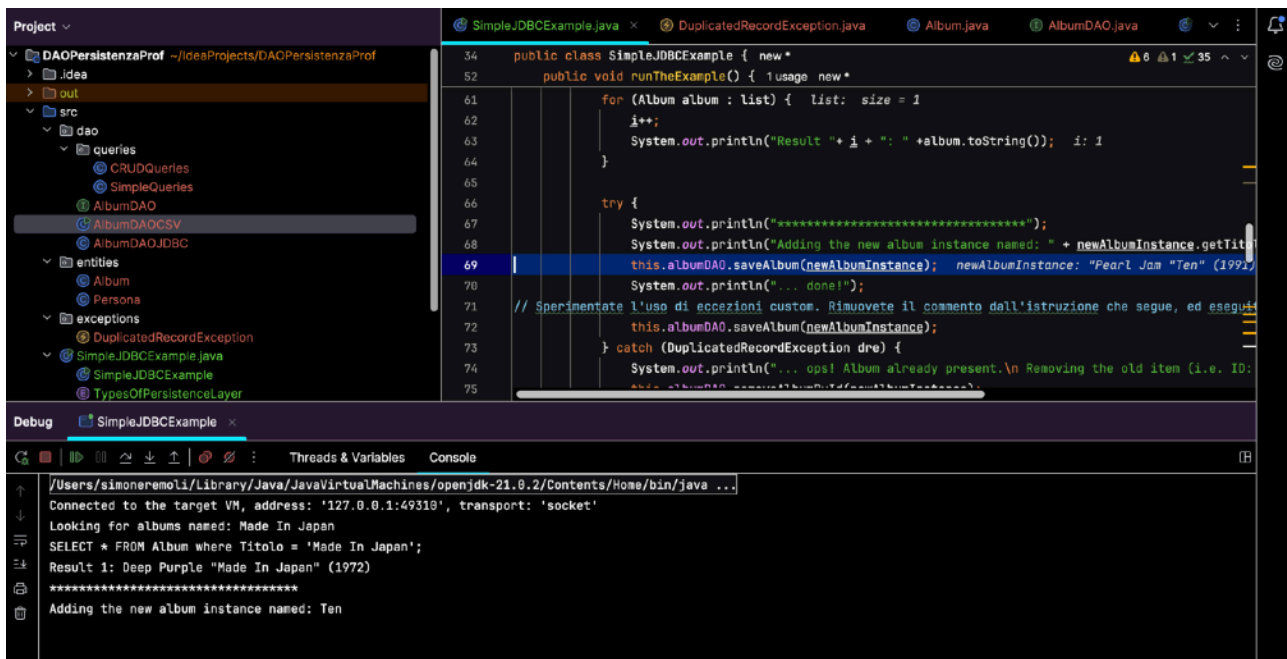
E finalmente i risultati vengono stampati a video, facendo il fetch degli oggetti di tipo album (tipica query del Prof.Pellegrini, ripresa dal De Angelis).



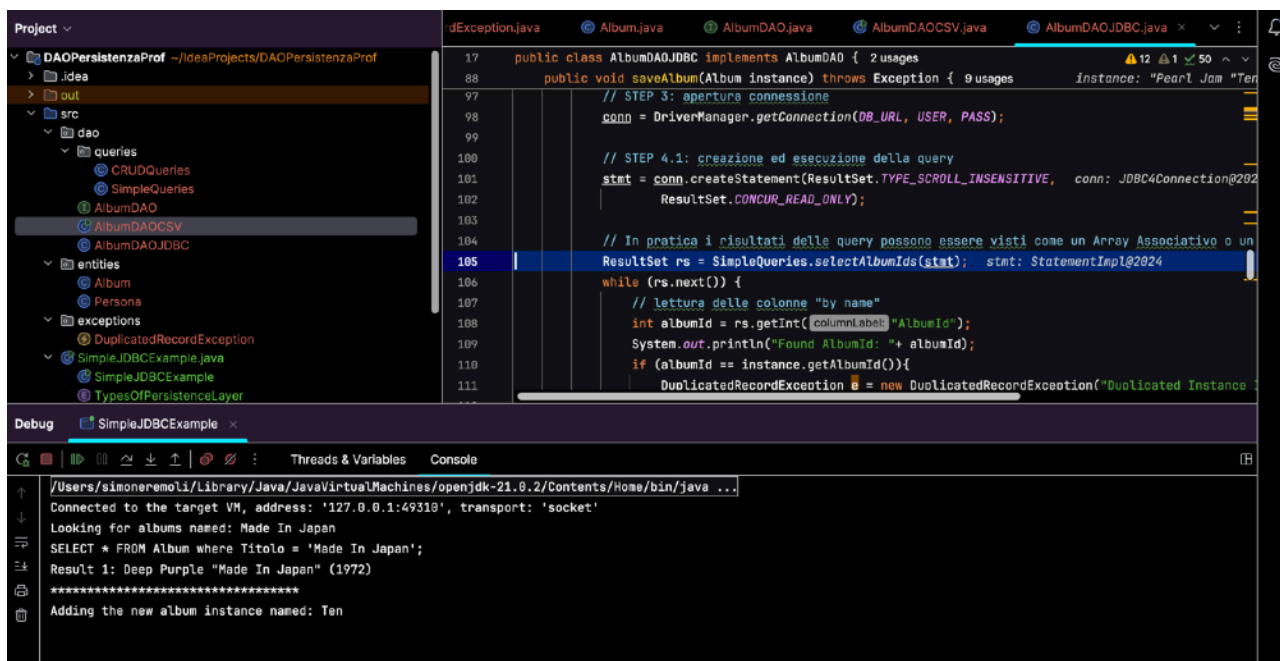
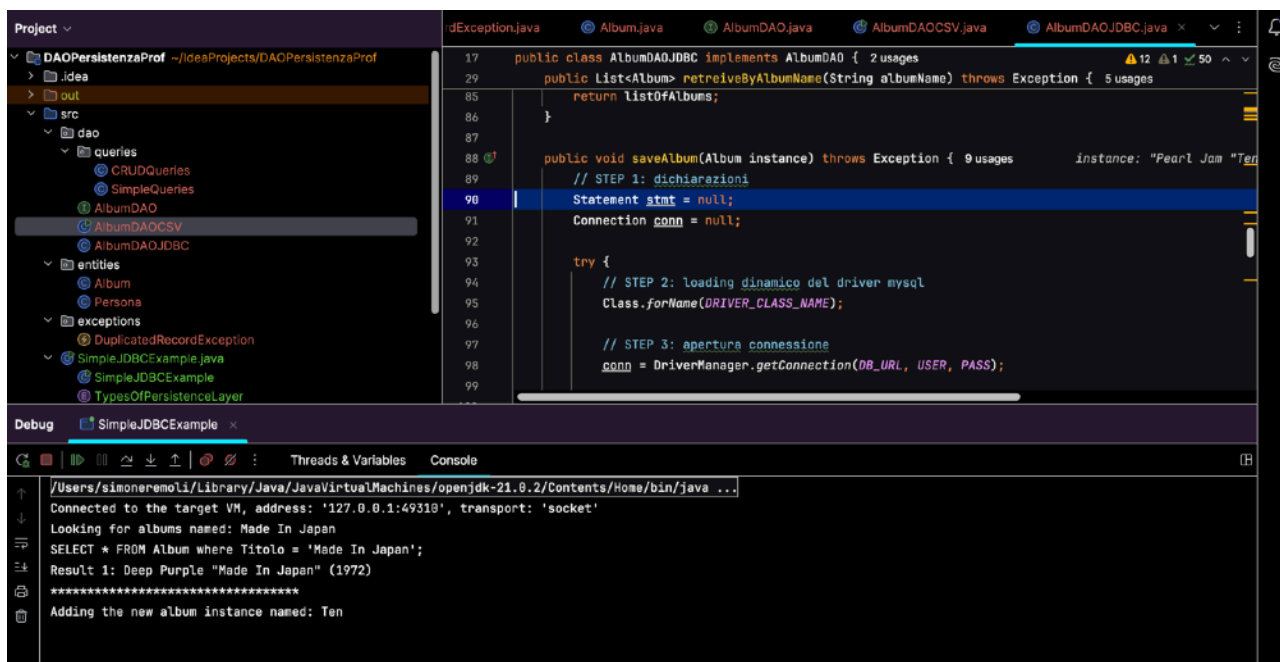
Okay adesso fate una pausa. Respirate. Chi non conosce nulla di basi di dati ha già fatto abbastanza, è praticamente ciò che si consegna come progetto al Prof.Pellegrini.

A questo punto si chiama il metodo `saveAlbum()` sull'oggetto `albumDAO`.

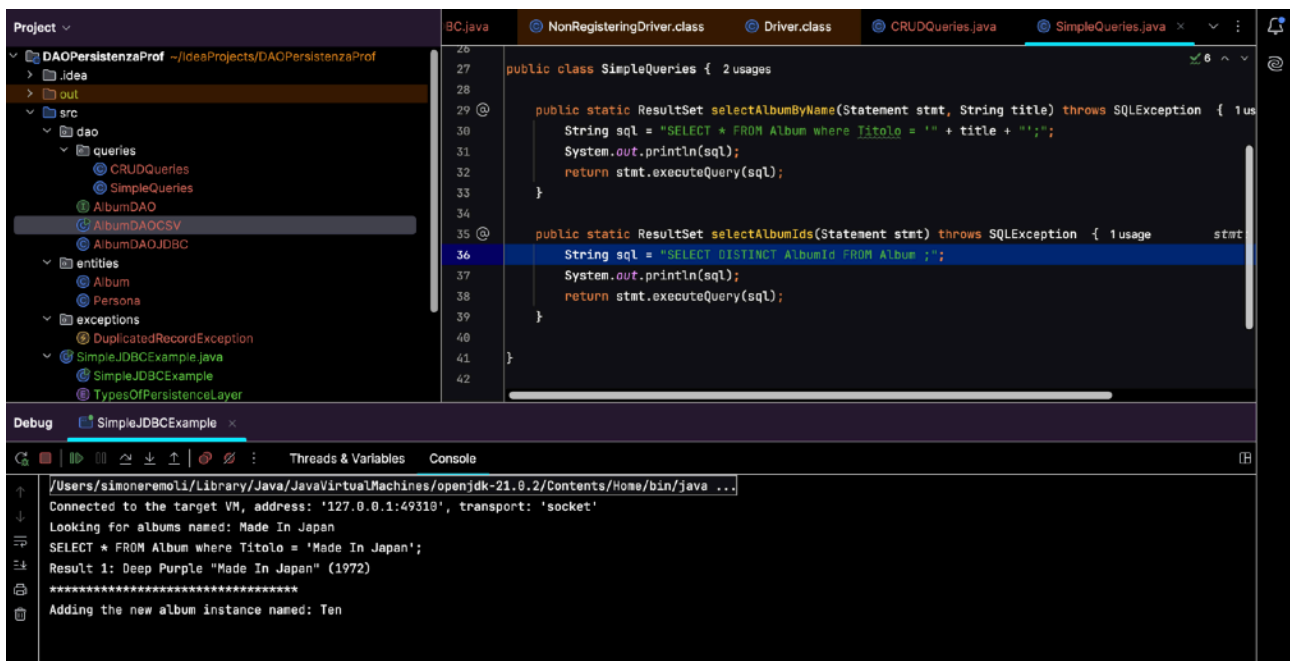
Ora, ricordando che `albumDAO` è di tipo `AlbumDAOJDBC`, questa chiamata corrisponde a eseguire: `AlbumDAOJDBC.saveAlbum(newAlbumInstance);`



Questa funzione della DAO al contrario del report vuole scrivere sul database, quindi è analoga a quella di prima, tranne la query.

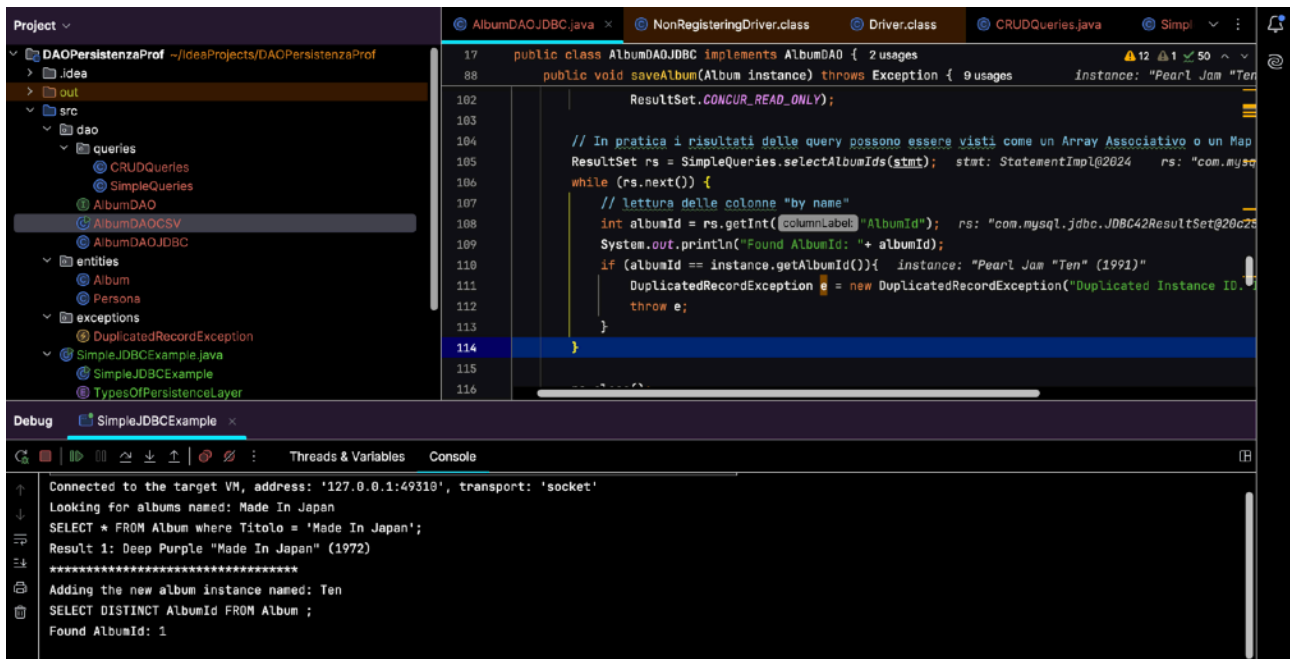


Ora si controlla se esiste già un album con lo stesso ID prima di inserirlo.

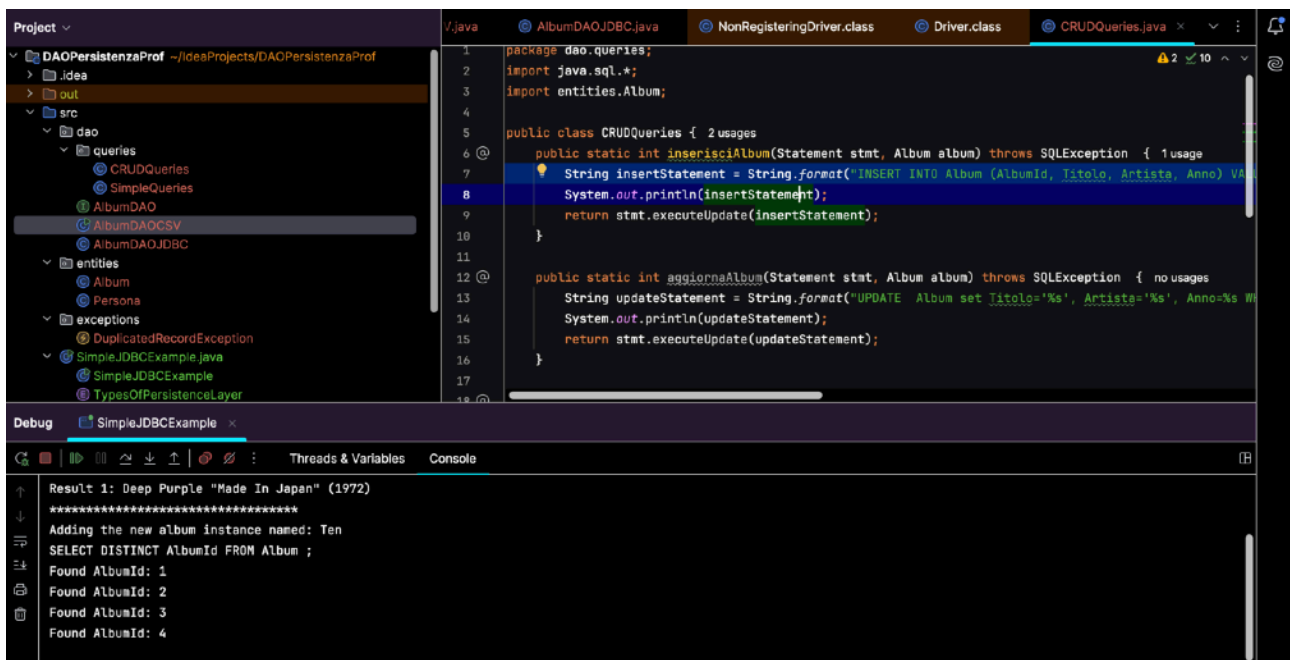


Se trova un ID uguale lancia una
DuplicatedRecordException.

Se non trova duplicati procede.

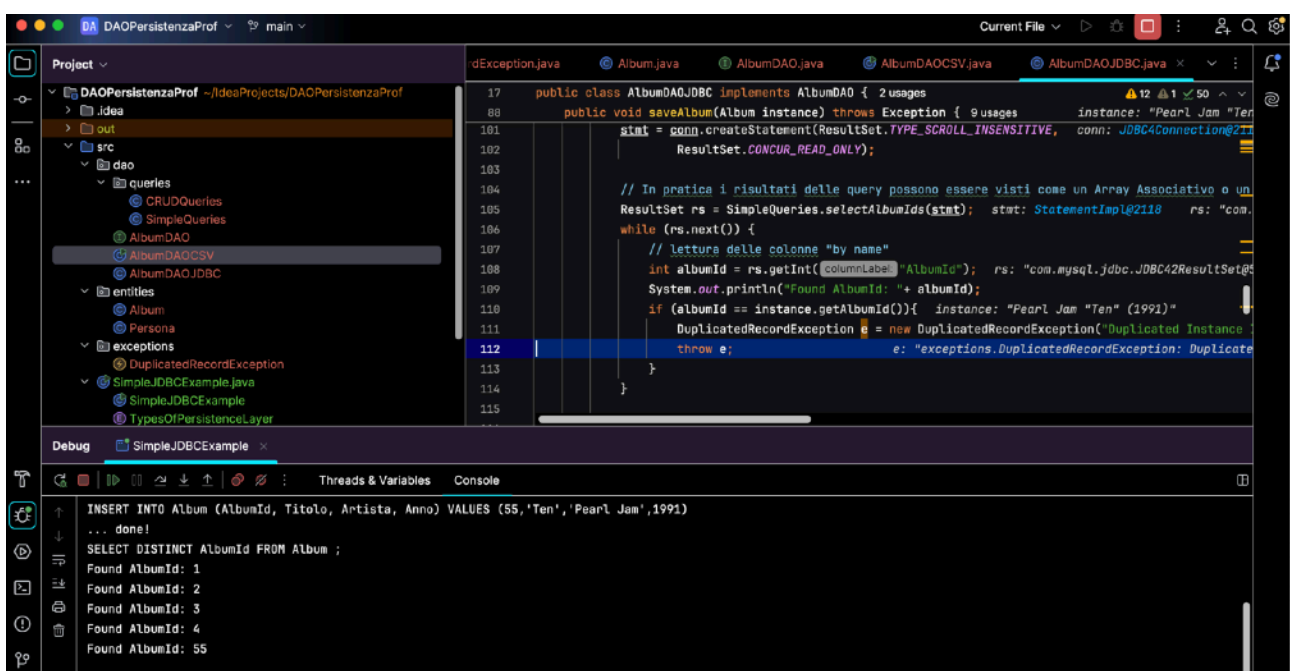


Se non viene sollevata l'eccezione custom,
l'album viene inserito.

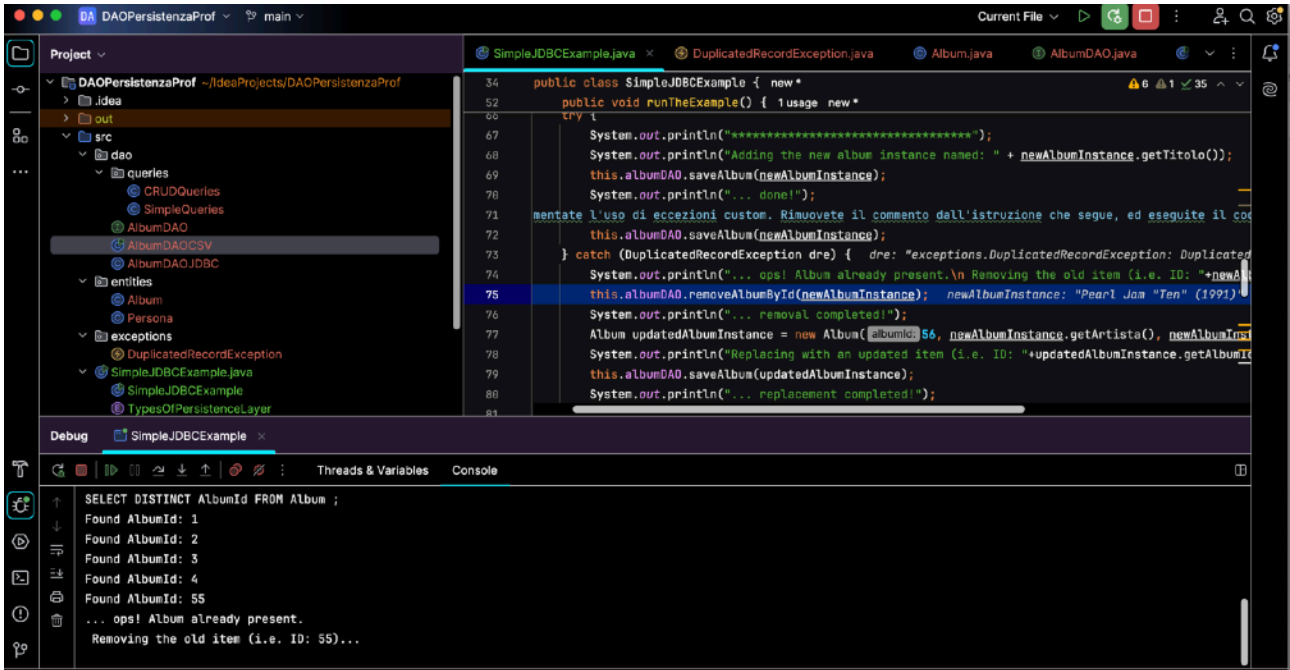


Aggiunta una riga. La numero 5.

	123 AlbumId	A-Z Titolo	A-Z Artista	123 Anno
1	1	Made In Japan	Deep Purple	1,972
2	2	Be	Pain Of Salvation	2,004
3	3	Images And Words	Dream Theater	1,992
4	4	The Human Equation	Ayreon	2,004
5	55	Ten	Pearl Jam	1,991



Alla prossima chiamata del DAO il valore sarà presente e verrà sollevata un'eccezione personalizzata.



Dopodiché c'è la rimozione e il programma è tutto uguale.

Fine del documento.